Computación cuántica con Qiskit: conceptos fundamentales con código comentado

Genesis Silesky

21 de octubre de 2026

Índice

1. Prerrequisitos y notación **2**
1.1. Números complejos . . . 2
1.2. Vectores, norma y producto interno . . . 2
1.3. Notación de Dirac . . . 2
1.4. Matrices unitarias . . . 2
1.5. Producto tensorial (Kronecker) . . . 2
1.6. Probabilidad básica . . . 2
1.7. Conceptos de Qiskit . . . 3
1.8. Puertas usadas en estos ejemplos . . . 3
1.9. Cómo ejecutar el código . . . 3

2. El qubit como vector **4**
2.1. Objetivo . . . 4
2.2. Conceptos previos . . . 4
2.3. Fundamento teórico . . . 4
2.4. Código (completamente comentado) . . . 4
2.5. Explicación celda por celda . . . 5
2.6. Salida obtenida . . . 5
2.7. Interpretación . . . 5
2.8. Ejercicios . . . 6
2.9. Errores comunes . . . 6

3. Las puertas cuánticas son matrices unitarias **7**
3.1. Objetivo . . . 7
3.2. Conceptos previos . . . 7
3.3. Fundamento teórico . . . 7
3.4. Código (completamente comentado) . . . 7
3.5. Explicación celda por celda . . . 8
3.6. Salida obtenida . . . 8
3.7. Interpretación . . . 8
3.8. Ejercicios . . . 8
3.9. Errores comunes . . . 9

4. Crecimiento exponencial: 2^n amplitudes **10**
4.1. Objetivo . . . 10
4.2. Conceptos previos . . . 10
4.3. Fundamento teórico . . . 10
4.4. Código (completamente comentado) . . . 10

4.5. Explicación celda por celda . . . 11
4.6. Salida obtenida . . . 11
4.7. Interpretación . . . 11
4.8. Ejercicios . . . 11

5. Medición y regla de Born **12**
5.1. Objetivo . . . 12
5.2. Conceptos previos . . . 12
5.3. Fundamento teórico . . . 12
5.4. Código (completamente comentado) . . . 12
5.5. Explicación celda por celda . . . 13
5.6. Salida obtenida . . . 13
5.7. Interpretación . . . 13
5.8. Ejercicios . . . 13

6. Interferencia cuántica **14**
6.1. Objetivo . . . 14
6.2. Conceptos previos . . . 14
6.3. Fundamento teórico . . . 14
6.4. Código (completamente comentado) . . . 14
6.5. Explicación celda por celda . . . 15
6.6. Salida obtenida . . . 15
6.7. Interpretación . . . 15
6.8. Ejercicios . . . 15

7. Entrelazamiento: el estado de Bell **16**
7.1. Objetivo . . . 16
7.2. Conceptos previos . . . 16
7.3. Fundamento teórico . . . 16
7.4. Código (completamente comentado) . . . 17
7.5. Explicación celda por celda . . . 17
7.6. Salida obtenida . . . 17
7.7. Interpretación . . . 17
7.8. Ejercicios . . . 18

8. Algoritmo de Grover: búsqueda cuadrática **19**
8.1. Objetivo . . . 19
8.2. Conceptos previos . . . 19
8.3. Fundamento teórico . . . 19
8.4. Código (completamente comentado) . . . 19
8.5. Explicación celda por celda . . . 20
8.6. Salida obtenida . . . 20
8.7. Interpretación . . . 21
8.8. Ejercicios . . . 21

9. Práctica guiada: Bell, variantes y GHZ **22**
9.1. Objetivo . . . 22
9.2. Conceptos previos . . . 22
9.3. Fundamento teórico . . . 22
9.4. Código (completamente comentado) . . . 22
9.5. Explicación celda por celda . . . 23
9.6. Salida obtenida . . . 23

9.7. Interpretación . . . 23
9.8. Ejercicios . . . 24

10. Ruido y la era NISQ **25**
10.1. Objetivo . . . 25
10.2. Conceptos previos . . . 25
10.3. Fundamento teórico . . . 25
10.4. Código (completamente comentado) . . . 25
10.5. Explicación celda por celda . . . 26
10.6. Salida obtenida . . . 26
10.7. Interpretación . . . 26
10.8. Ejercicios . . . 26

11. Generador de números aleatorios cuántico (QRNG) **27**
11.1. Objetivo . . . 27
11.2. Conceptos previos . . . 27
11.3. Fundamento teórico . . . 27
11.4. Código (completamente comentado) . . . 27
11.5. Explicación celda por celda . . . 28
11.6. Salida obtenida . . . 28
11.7. Interpretación . . . 28
11.8. Ejercicios . . . 28

12. La esfera de Bloch: un qubit como punto de una esfera **29**
12.1. Objetivo . . . 29
12.2. Conceptos previos . . . 29
12.3. Fundamento teórico . . . 29
12.4. Código (completamente comentado) . . . 30
12.5. Explicación celda por celda . . . 31
12.6. Salida obtenida . . . 31
12.7. Interpretación . . . 31
12.8. Ejercicios . . . 31
12.9. Errores comunes . . . 32

13. Oscilación de Rabi: cómo un pulso cambia el estado de un qubit **33**
13.1. Objetivo . . . 33
13.2. Conceptos previos . . . 33
13.3. Fundamento teórico . . . 33
13.4. Código (completamente comentado) . . . 34
13.5. Explicación celda por celda . . . 35
13.6. Salida obtenida . . . 36
13.7. Interpretación . . . 36
13.8. Ejercicios . . . 36
13.9. Errores comunes . . . 36

14. Control cuántico: Hamiltoniano, rotaciones y acoplamiento **37**
14.1. Objetivo . . . 37
14.2. Conceptos previos . . . 37
14.3. Fundamento teórico . . . 37
14.4. Código (completamente comentado) . . . 38
14.5. Explicación celda por celda . . . 40
14.6. Salida obtenida . . . 40

14.7. Interpretación . . . 40
14.8. Ejercicios . . . 41
14.9. Errores comunes . . . 41

15. Tomografía de estados: reconstruir un estado a partir de mediciones 42
15.1. Objetivo . . . 42
15.2. Conceptos previos . . . 42
15.3. Fundamento teórico . . . 42
15.4. Código (completamente comentado) . . . 43
15.5. Explicación celda por celda . . . 45
15.6. Salida obtenida . . . 45
15.7. Interpretación . . . 45
15.8. Ejercicios . . . 45
15.9. Errores comunes . . . 45

16. Puertas universales y lógica clásica reversible 46
16.1. Objetivo . . . 46
16.2. Conceptos previos . . . 46
16.3. Fundamento teórico . . . 46
16.4. Código (completamente comentado) . . . 47
16.5. Explicación celda por celda . . . 48
16.6. Salida obtenida . . . 49
16.7. Interpretación . . . 49
16.8. Ejercicios . . . 49
16.9. Errores comunes . . . 49

17. Algoritmo de Deutsch 50
17.1. Objetivo . . . 50
17.2. Conceptos previos . . . 50
17.3. Fundamento teórico . . . 50
17.4. Código (completamente comentado) . . . 51
17.5. Explicación celda por celda . . . 52
17.6. Salida obtenida . . . 52
17.7. Interpretación . . . 53
17.8. Ejercicios . . . 53
17.9. Errores comunes . . . 53

18. Algoritmo de Bernstein–Vazirani 54
18.1. Objetivo . . . 54
18.2. Conceptos previos . . . 54
18.3. Fundamento teórico . . . 54
18.4. Código (completamente comentado) . . . 55
18.5. Explicación celda por celda . . . 56
18.6. Salida obtenida . . . 56
18.7. Interpretación . . . 56
18.8. Ejercicios . . . 57
18.9. Errores comunes . . . 57

19. Transformada de Fourier cuántica y estimación de fase **58**
19.1. Objetivo . . . 58
19.2. Conceptos previos . . . 58
19.3. Fundamento teórico . . . 58
19.4. Código (completamente comentado) . . . 59
19.5. Explicación celda por celda . . . 61
19.6. Salida obtenida . . . 61
19.7. Interpretación . . . 61
19.8. Ejercicios . . . 61
19.9. Errores comunes . . . 61

20. Algoritmo HHL: resolver un sistema lineal con dos qubits **62**
20.1. Objetivo . . . 62
20.2. Conceptos previos . . . 62
20.3. Fundamento teórico . . . 62
20.4. Código (completamente comentado) . . . 63
20.5. Explicación celda por celda . . . 64
20.6. Salida obtenida . . . 65
20.7. Interpretación . . . 65
20.8. Ejercicios . . . 65
20.9. Errores comunes . . . 65

21. VQE: eigensolver cuántico variacional **67**
21.1. Objetivo . . . 67
21.2. Conceptos previos . . . 67
21.3. Fundamento teórico . . . 67
21.4. Código (completamente comentado) . . . 68
21.5. Explicación celda por celda . . . 69
21.6. Salida obtenida . . . 70
21.7. Interpretación . . . 70
21.8. Ejercicios . . . 70
21.9. Errores comunes . . . 70

22. QAOA: algoritmo de optimización cuántica aproximada **71**
22.1. Objetivo . . . 71
22.2. Conceptos previos . . . 71
22.3. Fundamento teórico . . . 71
22.4. Código (completamente comentado) . . . 72
22.5. Explicación celda por celda . . . 74
22.6. Salida obtenida . . . 74
22.7. Interpretación . . . 75
22.8. Ejercicios . . . 75
22.9. Errores comunes . . . 75

23. Decoherencia y spin echo: perder y recuperar la fase de un qubit **76**
23.1. Objetivo . . . 76
23.2. Conceptos previos . . . 76
23.3. Fundamento teórico . . . 76
23.4. Código (completamente comentado) . . . 77
23.5. Explicación celda por celda . . . 79
23.6. Salida obtenida . . . 80
23.7. Interpretación . . . 80

23.8. Ejercicios . . . 80
23.9. Errores comunes . . . 80

24. Desacople dinámico: apagar el acoplamiento con pulsos π 81
24.1. Objetivo . . . 81
24.2. Conceptos previos . . . 81
24.3. Fundamento teórico . . . 81
24.4. Código (completamente comentado) . . . 82
24.5. Explicación celda por celda . . . 83
24.6. Salida obtenida . . . 83
24.7. Interpretación . . . 84
24.8. Ejercicios . . . 84
24.9. Errores comunes . . . 84

25. Pulsos optimizados con GRAPE: control como problema de optimización 85
25.1. Objetivo . . . 85
25.2. Conceptos previos . . . 85
25.3. Fundamento teórico . . . 85
25.4. Código (completamente comentado) . . . 86
25.5. Explicación celda por celda . . . 88
25.6. Salida obtenida . . . 88
25.7. Interpretación . . . 88
25.8. Ejercicios . . . 89
25.9. Errores comunes . . . 89

Referencias 90

1. Prerrequisitos y notación

Este capítulo reúne las herramientas matemáticas y de software que usan todos los demás. Cada capítulo de concepto indica, en sus cajas azules, qué secciones de aquí necesita; si algo no se entiende, conviene volver a la sección correspondiente antes de continuar.

1.1. Números complejos

Un número complejo es $z = a + bi$ con $a, b \in \mathbb{R}$ e $i^2 = -1$.

- **Conjugado:** $\bar{z} = a - bi$.
- **Módulo:** $|z| = \sqrt{z\bar{z}} = \sqrt{a^2 + b^2}$. Es el que importa para las probabilidades.
- **Forma polar y fórmula de Euler:** $z = |z|e^{i\varphi}$, con $e^{i\varphi} = \cos\varphi + i\sin\varphi$. El ángulo φ es la *fase*.

En Python un complejo se escribe `1+1j` (la unidad imaginaria es `1j`) y `np.abs(z)` devuelve su módulo.

1.2. Vectores, norma y producto interno

Los estados de n qubits viven en $\mathbb{C}^{2^n}$. Para vectores columna $u, v \in \mathbb{C}^d$ el producto interno es $\langle u|v\rangle = \sum_k \bar{u}_k v_k$ y la norma es $\|v\| = \sqrt{\langle v|v\rangle}$. Un estado cuántico válido siempre tiene $\|v\| = 1$ ("normalizado").

1.3. Notación de Dirac

Símbolo	Significado	En Python/numpy
$\|\psi\rangle$	vector columna ("ket")	`np.array([a, b])`
$\langle\psi\|$	su transpuesta conjugada ("bra")	`v.conj()`
$\langle\varphi\|\psi\rangle$	producto interno	`np.vdot(phi, psi)`
$\|0\rangle = \binom{1}{0},\ \|1\rangle = \binom{0}{1}$	base computacional	`Statevector.from_label("0")`

1.4. Matrices unitarias

La adjunta de U es $U^\dagger = \bar{U}^T$ (transpuesta conjugada). U es *unitaria* si $U^\dagger U = UU^\dagger = I$. Las matrices unitarias conservan la norma: $\|Uv\| = \|v\|$, y son invertibles con $U^{-1} = U^\dagger$. Toda puerta cuántica es una matriz unitaria.

1.5. Producto tensorial (Kronecker)

Para combinar sistemas se usa $A \otimes B$. En vectores, $|a\rangle \otimes |b\rangle$ se escribe $|ab\rangle$; por ejemplo $|0\rangle \otimes |1\rangle = |01\rangle = (0, 1, 0, 0)^T$. Con n qubits la base tiene 2^n vectores $|x\rangle$, $x \in \{0, 1\}^n$. Un estado que *no* puede escribirse como $|a\rangle \otimes |b\rangle$ está entrelazado (capítulo 05b).

1.6. Probabilidad básica

- **Regla de Born:** si el estado es $\sum_x c_x |x\rangle$, medir da x con probabilidad $|c_x|^2$ (y esas probabilidades suman 1 por la normalización).
- **Frecuencia y ley de los grandes números:** al repetir N veces un experimento con probabilidad p, la frecuencia observada tiene error típico $\sqrt{p(1-p)/N}$ y tiende a p.

1.7. Conceptos de Qiskit

Circuito (`QuantumCircuit`) Lista ordenada de puertas y mediciones sobre qubits y bits clásicos.

`Statevector` Calcula el vector de estado *exacto* de un circuito sin medir; solo es posible en simulación.

Simulador (`AerSimulator`) Ejecuta circuitos en el computador local, con o sin ruido.

Shot Una ejecución completa del circuito con su medición. Se repite muchas veces para estimar probabilidades.

Conteos (`get_counts`) Diccionario cadena-de-bits → número de veces que salió.

Orden de los bits en Qiskit (little-endian)

Qiskit escribe las cadenas con el qubit 0 a la **derecha**: la cadena `'10'` significa $q_1 = 1$, $q_0 = 0$. En estos ejemplos se señala cada vez que importa.

1.8. Puertas usadas en estos ejemplos

Puerta	Matriz	Efecto	Método Qiskit
H	$\frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}$	superposición	`qc.h(q)`
X	$\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$	NOT: $\|0\rangle \leftrightarrow \|1\rangle$	`qc.x(q)`
Z	$\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$	cambia el signo de $\|1\rangle$	`qc.z(q)`
$R_y(\theta)$	$\begin{pmatrix}\cos\frac{\theta}{2} & -\sin\frac{\theta}{2}\\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2}\end{pmatrix}$	rotación real	`qc.ry(theta, q)`
CNOT	(4×4)	invierte el objetivo si el control es $\|1\rangle$	`qc.cx(c, t)`

1.9. Cómo ejecutar el código

Cada ejemplo existe como `.py` (terminal) y como `.ipynb` (Jupyter/Colab). En el `.py`, las líneas `#%%` separan "celdas": en un notebook cada una es una celda; en la terminal no tienen efecto. Instalación (versiones con las que se probó todo):

```
pip install qiskit==2.5.0 qiskit-aer==0.17.2 matplotlib pylatexenc
python 01_qubit_vector.py
```

La API de Qiskit ha cambiado bastante entre versiones; si se usa otra versión algún ejemplo podría fallar.

2. El qubit como vector

2.1. Objetivo

Entender que un qubit es un vector de dos amplitudes complejas normalizado, y cómo esas amplitudes determinan las probabilidades de medición.

2.2. Conceptos previos

Concepto previo: lo que hace falta para este capítulo

Ver el capítulo de prerrequisitos (sección 1): números complejos (módulo y fase), vectores y norma, notación de Dirac y regla de Born. Si solo se recuerda una cosa: el *módulo al cuadrado* de una amplitud es una probabilidad.

2.3. Fundamento teórico

Un bit clásico vale 0 o 1. Un qubit es un vector

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = \begin{pmatrix}\alpha\\ \beta\end{pmatrix}, \qquad \alpha,\beta\in\mathbb{C}, \quad |\alpha|^2+|\beta|^2 = 1. \tag{1}$$

Por la regla de Born, al medir se obtiene 0 con probabilidad $|\alpha|^2$ y 1 con probabilidad $|\beta|^2$. La condición de normalización asegura que las probabilidades sumen 1. Dos observaciones:

- **Fase global:** $e^{i\gamma}\,|\psi\rangle$ y $|\psi\rangle$ son físicamente indistinguibles.
- **Fase relativa:** la fase de β respecto a α no altera las probabilidades de esta medición, pero sí cómo interfiere el qubit con otros después (capítulo 05a).

Salvo fase global, todo qubit se parametriza como $\cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}\,|1\rangle$ (esfera de Bloch).

Ejemplo del código. Se toma $\alpha = 1/\sqrt{3}$ y $\beta = \sqrt{2/3}\,e^{i\pi/4} = (1+i)/\sqrt{3}$. Entonces $|\alpha|^2 = 1/3$ y $|\beta|^2 = 2/3$, por lo que $P(0) = 1/3$, $P(1) = 2/3$.

2.4. Código (completamente comentado)

```
# %% [markdown]
# # 01 - El qubit como vector
# |psi> = alpha|0> + beta|1>, con |alpha|^2 + |beta|^2 = 1

# %% Celda 1: importaciones
# numpy nos da numeros complejos y funciones matematicas (sqrt, exp, pi).
import numpy as np

# Statevector representa el estado de un sistema cuantico como un vector
# de amplitudes complejas (el "vector de estado").
from qiskit.quantum_info import Statevector

# %% Celda 2: los estados de la base computacional
# from_label("0") construye |0>, que en forma de vector columna es (1, 0).
# Estos dos vectores son la base canonica: cualquier qubit es una
# combinacion lineal de ellos.
print("|0> =", Statevector.from_label("0").data)  # [1+0j, 0+0j]
print("|1> =", Statevector.from_label("1").data)  # [0+0j, 1+0j]

# %% Celda 3: un qubit arbitrario
# Elegimos dos amplitudes complejas alpha y beta.
```

```python
# alpha es real; beta tiene modulo sqrt(2/3) y una fase de pi/4 radianes.
# La fase e^{i*pi/4} NO cambia la probabilidad de medir, pero si cambia
# como interfiere el qubit con otros (ver el ejemplo 05a).
alpha = 1 / np.sqrt(3)
beta = np.sqrt(2 / 3) * np.exp(1j * np.pi / 4)

# Construimos el estado a partir de la lista de amplitudes [alpha, beta].
psi = Statevector([alpha, beta])

# .data devuelve el arreglo de numpy con las amplitudes.
print("Vector:", psi.data)

# Condicion de normalizacion: la suma de los modulos al cuadrado debe ser 1.
# np.abs de un complejo es su modulo; el 1.0000000000000002 que se imprime
# es error de redondeo de punto flotante, no un error del estado.
print("Norma |a|^2+|b|^2 =", np.sum(np.abs(psi.data) ** 2))

# %% Celda 4: de amplitudes a probabilidades (regla de Born)
# probabilities_dict() aplica la regla de Born: P(k) = |amplitud_k|^2.
# Esperamos P(0) = 1/3 y P(1) = 2/3. Convertimos a float para imprimir limpio.
probs = {str(k): round(float(v), 4) for k, v in psi.probabilities_dict().items()}
print("Probabilidades:", probs)

# %% Celda 5: un vector NO normalizado no es un estado valido
# (1, 1) tiene norma^2 = 2, asi que no representa un qubit fisico.
# is_valid() comprueba justamente que la norma sea 1.
print("(1,1) es valido?:", Statevector([1, 1]).is_valid())  # False
print("(1,1)/sqrt(2) es valido?:", Statevector([1, 1] / np.sqrt(2)).is_valid())  # True
```

2.5. Explicación celda por celda

Celda 1 Importa `numpy` (complejos y funciones) y `Statevector`, la clase que guarda las amplitudes.

Celda 2 Construye $|0\rangle = (1, 0)$ y $|1\rangle = (0, 1)$ con `from_label`. Son la base canónica.

Celda 3 Define α, β y crea el estado. La suma $|\alpha|^2 + |\beta|^2$ sale 1.0000000000000002 y no exactamente 1: es error de redondeo de punto flotante (aritmética de doble precisión).

Celda 4 `probabilities_dict()` aplica la regla de Born. Se convierte a `float` y `str` para que la impresión sea legible (NumPy 2 imprime sus tipos propios).

Celda 5 Muestra que $(1, 1)$ no es válido (norma$^2 = 2$) y que $(1, 1)/\sqrt{2}$ sí lo es.

2.6. Salida obtenida

```
|0> = [1.+0.j 0.+0.j]
|1> = [0.+0.j 1.+0.j]
Vector: [0.57735027+0.j         0.57735027+0.57735027j]
Norma |a|^2+|b|^2 = 1.0000000000000002
Probabilidades: {'0': 0.3333, '1': 0.6667}
(1,1) es valido?: False
(1,1)/sqrt(2) es valido?: True
```

Esta salida es determinista (no hay medición).

2.7. Interpretación

Las amplitudes son números complejos, pero las probabilidades salen reales y suman 1. La fase de β ($\pi/4$) no aparece en las probabilidades: es información que solo se revela mediante interferencia.

2.8. Ejercicios

1. Cambie la fase de β a π y verifique que las probabilidades no cambian.
2. Construya un estado con $P(1) = 0.9$ y compruébelo.
3. ¿Es $(\frac{3}{5}, \frac{4}{5}i)$ un estado válido? Verifíquelo con `is_valid()`.

2.9. Errores comunes

- Olvidar normalizar: Qiskit no lo hace automáticamente y `Statevector` podría ser inválido.
- Confundir amplitud con probabilidad: la amplitud puede ser negativa o compleja; la probabilidad es su módulo al cuadrado.

3. Las puertas cuánticas son matrices unitarias

3.1. Objetivo

Ver que aplicar una puerta equivale a multiplicar una matriz por el vector de estado, y por qué esa matriz debe ser unitaria.

3.2. Conceptos previos

Concepto previo: multiplicación matriz–vector y unitariedad

Ver sección 1: matrices unitarias ($U^\dagger U = I$) y notación de Dirac. Recordatorio: el producto $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)\left(\begin{smallmatrix} x \\ y \end{smallmatrix}\right) = \left(\begin{smallmatrix} ax+by \\ cx+dy \end{smallmatrix}\right)$.

3.3. Fundamento teórico

Una puerta de un qubit es una matriz $U \in \mathbb{C}^{2\times 2}$ unitaria y el estado evoluciona como $|\psi'\rangle = U|\psi\rangle$. Las más usadas:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{2}$$

Con $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) =: |+\rangle$ y $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) =: |-\rangle$. *(Aquí se usa la notación $|\pm\rangle$ solo como abreviatura.)*

Por qué unitarias. Si $U^\dagger U = I$ entonces $\|U|\psi\rangle\|^2 = \langle\psi|U^\dagger U|\psi\rangle = \||\psi\rangle\|^2$: la normalización (probabilidades que suman 1) se conserva. Además U es invertible con $U^{-1} = U^\dagger$, es decir, la evolución cuántica es *reversible*. Para H se cumple $H^\dagger = H$ y $H^2 = I$.

3.4. Código (completamente comentado)

```python
# %% [markdown]
# # 02 - Las puertas cuanticas son matrices unitarias
# H = 1/sqrt(2) * [[1, 1], [1, -1]]

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit  # un circuito es una lista de puertas
from qiskit.quantum_info import Operator, Statevector
# Operator convierte un circuito (o una puerta) en su matriz unitaria.

# %% Celda 2: la matriz de cada puerta
# Para cada puerta construimos un circuito de 1 qubit con solo esa puerta
# y le pedimos a Qiskit su matriz.
for nombre, puerta in [("H", "h"), ("X", "x"), ("Z", "z")]:
    qc = QuantumCircuit(1)
    # getattr(qc, "h")(0) es lo mismo que qc.h(0): aplica la puerta al qubit 0.
    getattr(qc, puerta)(0)
    # np.round limpia el ruido numerico (0.707106781186547 -> 0.707).
    print(nombre, "=\n", np.round(Operator(qc).data, 3))
# H: crea superposicion. X: "NOT" cuantico (intercambia |0> y |1>).
# Z: deja |0> igual y cambia el signo de |1> (puerta de fase).

# %% Celda 3: aplicar una puerta = multiplicar matriz por vector
H = Operator.from_label("H").data  # matriz 2x2 de Hadamard
ket0 = np.array([1, 0])            # el vector |0>
# El operador @ es el producto matricial de numpy.
```

```python
print("H @ |0> =", H @ ket0)       # (1/sqrt2, 1/sqrt2): superposicion igual

# Qiskit hace exactamente lo mismo con .evolve(): aplica el operador al estado.
evolucionado = Statevector.from_label("0").evolve(Operator.from_label("H"))
print("Statevector:", evolucionado.data)

# %% Celda 4: las puertas son unitarias
# Una matriz U es unitaria si U^dagger U = I, donde U^dagger es la
# transpuesta conjugada (.conj().T). Esto garantiza que la norma del
# vector se conserve: una puerta nunca rompe la normalizacion.
print("H^dagger H = I:", np.allclose(H.conj().T @ H, np.eye(2)))

# Como consecuencia, toda puerta es reversible: su inversa es U^dagger.
# En particular H es su propia inversa (H H = I), lo que se usa en 05a.
print("H H = I:", np.allclose(H @ H, np.eye(2)))
```

3.5. Explicación celda por celda

Celda 1 `Operator` convierte un circuito en su matriz.

Celda 2 Para cada puerta crea un circuito de un qubit y extrae su matriz. `getattr(qc, "h")(0)` equivale a `qc.h(0)`; se usa para recorrer varias puertas en un ciclo.

Celda 3 Calcula $H|0\rangle$ con `@` (producto matricial de NumPy) y con `.evolve()` de Qiskit; ambos dan $(1/\sqrt{2},\, 1/\sqrt{2}) \approx (0.707,\, 0.707)$.

Celda 4 Verifica numéricamente $H^\dagger H = I$ y $H^2 = I$. `np.allclose` compara con tolerancia numérica (no se usa `==` con decimales).

3.6. Salida obtenida

```
H =
 [[ 0.707+0.j  0.707+0.j]
 [ 0.707+0.j -0.707+0.j]]
X =
 [[0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j]]
Z =
 [[ 1.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j]]
H @ |0> = [0.70710678+0.j 0.70710678+0.j]
Statevector: [0.70710678+0.j 0.70710678+0.j]
H^dagger H = I: True
H H = I: True
```

3.7. Interpretación

Cada puerta es una matriz concreta y calculable. El resultado $H|0\rangle = (0.707, 0.707)$ confirma una superposición con $P(0) = P(1) = \frac{1}{2}$.

3.8. Ejercicios

1. Calcule a mano $XZ|0\rangle$ y $ZX|0\rangle$. ¿Conmutan X y Z?
2. Verifique en el código que X y Z son unitarias.
3. Calcule $H|1\rangle$ y compruebe que sus probabilidades son $\frac{1}{2}, \frac{1}{2}$ aunque una amplitud sea negativa.

3.9. Errores comunes

- Con varios qubits, la matriz de un circuito usa el orden little-endian de Qiskit; para un solo qubit no hay ambigüedad.
- Comparar matrices con `==`: use `np.allclose`.

4. Crecimiento exponencial: 2^n amplitudes

4.1. Objetivo

Cuantificar por qué simular n qubits en una computadora clásica requiere memoria que crece como 2^n.

4.2. Conceptos previos

Concepto previo: producto tensorial y potencias de 2

Ver sección 1. Idea clave: al combinar sistemas con $\otimes$, las dimensiones se *multiplican*: $2 \cdot 2 \cdots 2 = 2^n$. Además, $1\,\text{GiB} = 2^{30}$ bytes.

4.3. Fundamento teórico

El espacio de n qubits es $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$; un estado general es

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} c_x \, |x\rangle \,, \qquad \sum_x |c_x|^2 = 1, \tag{3}$$

con 2^n amplitudes complejas independientes (salvo normalización y fase global). Con `complex128` cada amplitud ocupa 16 bytes, de modo que

$$\text{memoria} = 16 \cdot 2^n \text{ bytes}. \tag{4}$$

Para $n = 30$: $16 \cdot 2^{30}$ B $= 16$ GiB; para $n = 50$: $16 \cdot 2^{50}$ B $= 2^{24}$ GiB $= 16$ PiB, y hay $\approx 1.13 \times 10^{15}$ amplitudes.

Un caso concreto. Aplicar H a cada uno de n qubits partiendo de $|0 \cdots 0\rangle$ da $H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$. Con $n = 4$: 16 amplitudes iguales a $1/4$.

Matiz importante

Esto aplica a la simulación de circuitos *generales* con el vector de estado. Hay circuitos con estructura (p. ej. solo puertas Clifford, o con poco entrelazamiento) que se simulan eficientemente con otros métodos. Tampoco significa que un computador cuántico "guarde" 2^n números accesibles: al medir solo se obtienen n bits.

4.4. Código (completamente comentado)

```python
# %% [markdown]
# # 03 - Con n qubits hay 2^n amplitudes

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

# %% Celda 2: memoria necesaria para simular n qubits
# Un estado de n qubits tiene 2^n amplitudes complejas.
# Cada complejo de doble precision (complex128) ocupa 16 bytes
# (8 para la parte real y 8 para la imaginaria).
for n in [1, 2, 10, 20, 30, 40, 50]:
    amplitudes = 2 ** n                 # numero de amplitudes
    gib = amplitudes * 16 / 2 ** 30     # bytes -> GiB (1 GiB = 2^30 bytes)
    print(f"n={n:>2}  amplitudes={amplitudes:>18,}  memoria ~ {gib:>14,.4f} GiB")
```

```
# Con 30 qubits ya son 16 GiB (una laptop buena); con 50, millones de GiB.
# Este crecimiento exponencial es la razon por la que simular es inviable
# a gran escala, y una de las motivaciones de la computacion cuantica.

# %% Celda 3: ver las 2^n amplitudes de un caso pequeno
n = 4
qc = QuantumCircuit(n)
# qc.h(range(n)) aplica Hadamard a cada uno de los n qubits.
# El resultado es la superposicion uniforme de las 2^n cadenas de bits.
qc.h(range(n))
sv = Statevector(qc)  # calcula el vector de estado del circuito
# Todas las amplitudes valen 1/sqrt(2^n) = 1/4 cuando n = 4.
print(f"{len(sv.data)} amplitudes, cada una = {sv.data[0].real:.4f}")
```

4.5. Explicación celda por celda

Celda 2 Recorre varios n, calcula 2^n y convierte a GiB dividiendo entre 2^{30}. Los formatos `:>18,` alinean y ponen separadores de miles.

Celda 3 Construye el estado $H^{\otimes 4}\,|0000\rangle$ y comprueba que hay 16 amplitudes, todas iguales a 0.25.

4.6. Salida obtenida

```
n= 1  amplitudes=                 2  memoria ~         0.0000 GiB
n= 2  amplitudes=                 4  memoria ~         0.0000 GiB
n=10  amplitudes=             1,024  memoria ~         0.0000 GiB
n=20  amplitudes=         1,048,576  memoria ~         0.0156 GiB
n=30  amplitudes=     1,073,741,824  memoria ~        16.0000 GiB
n=40  amplitudes= 1,099,511,627,776  memoria ~    16,384.0000 GiB
n=50  amplitudes=1,125,899,906,842,624  memoria ~ 16,777,216.0000 GiB
16 amplitudes, cada una = 0.2500
```

4.7. Interpretación

Cada qubit extra *duplica* la memoria. A 30 qubits ya se necesitan 16 GiB; unos pocos más y ninguna laptop alcanza. Este es el motivo de que simular sea difícil y, a la vez, parte de la motivación de usar hardware cuántico.

4.8. Ejercicios

1. ¿Cuántos qubits se pueden simular con 64 GiB de RAM?
2. Repita el cálculo con `complex64` (8 bytes por amplitud).
3. Verifique con `Statevector` que para $n = 3$ las amplitudes valen $1/\sqrt{8} \approx 0.354$.

5. Medición y regla de Born

5.1. Objetivo

Comprobar experimentalmente que $P(0) = |\alpha|^2$ y $P(1) = |\beta|^2$, y ver cómo la frecuencia observada converge a esas probabilidades al aumentar los *shots*.

5.2. Conceptos previos

Concepto previo: regla de Born, colapso y frecuencia relativa

Ver sección 1. Dos ideas: (i) medir produce un resultado clásico aleatorio con las probabilidades de Born; (ii) tras medir, el estado *colapsa* al resultado observado, y la superposición original se pierde.

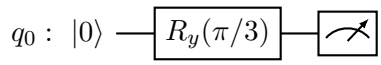
5.3. Fundamento teórico

Para $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$, la medición en la base computacional da 0 con probabilidad $|\alpha|^2$ y 1 con $|\beta|^2$. No se puede conocer α, β con una sola medición; hay que repetir el experimento N veces (cada repetición es un *shot*). La frecuencia $\hat{p}$ de "1" tiene error típico

$$\sigma = \sqrt{\frac{p(1-p)}{N}}. \tag{5}$$

Para $p = 0.25$: $\sigma \approx 0.137$ ($N = 10$), 0.043 ($N = 100$), 0.014 ($N = 1000$), 0.0043 ($N = 10000$).

Preparación del estado. La rotación $R_y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$. Para que $P(1) = \sin^2\frac{\theta}{2} = 0.25$ se necesita $\theta = 2\arcsin(0.5) = \pi/3$.



5.4. Código (completamente comentado)

```python
# %% [markdown]
# # 04 - Medicion: P(0) = |alpha|^2 y P(1) = |beta|^2

# %% Celda 1: importaciones
import sys

import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator  # simulador local de circuitos
from qiskit.visualization import plot_histogram

# La consola de Windows (cp1252) no imprime el dibujo del circuito sin esto.
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

# %% Celda 2: preparar un qubit con P(1) = 0.25
# La rotacion Ry(theta) lleva |0> a cos(theta/2)|0> + sin(theta/2)|1>.
# Queremos sin^2(theta/2) = 0.25, es decir theta = 2*arcsin(sqrt(0.25)).
theta = 2 * np.arcsin(np.sqrt(0.25))

# QuantumCircuit(1, 1): 1 qubit y 1 bit clasico donde guardar la medicion.
qc = QuantumCircuit(1, 1)
qc.ry(theta, 0)     # prepara el estado con las amplitudes deseadas
```

```python
qc.measure(0, 0)    # mide el qubit 0 y guarda el resultado en el bit clasico 0
print(qc.draw())    # dibujo del circuito en texto

# %% Celda 3: repetir la medicion (shots)
# Una sola medicion da un unico 0 o 1 y destruye la superposicion.
# Para estimar las probabilidades hay que repetir el experimento muchas
# veces: cada repeticion se llama "shot".
sim = AerSimulator()
for shots in [10, 100, 1000, 10000]:
    # run() lanza el circuito; result().get_counts() devuelve cuantas veces
    # salio cada resultado, por ejemplo {'0': 749, '1': 251}.
    counts = sim.run(qc, shots=shots).result().get_counts()
    print(shots, "shots ->", counts, f"| freq(1) = {counts.get('1', 0) / shots:.3f}")
# Ley de los grandes numeros: con mas shots, freq(1) se acerca a 0.25.

# %% Celda 4: histograma
# En Colab o Jupyter la figura se muestra sola; en terminal se puede
# guardar con plot_histogram(...).savefig("hist.png").
plot_histogram(sim.run(qc, shots=1000).result().get_counts())
```

5.5. Explicación celda por celda

Celda 1 Además de las importaciones, reconfigura la salida estándar a UTF-8: sin eso la consola de Windows no puede imprimir el dibujo del circuito (error `UnicodeEncodeError`).

Celda 2 Calcula θ, crea un circuito con 1 qubit y 1 bit clásico, aplica R_y y mide.

Celda 3 Ejecuta $10, 100, 1000, 10000$ shots y calcula la frecuencia de "1". `counts.get('1', 0)` evita un error si "1" nunca salió.

Celda 4 Dibuja el histograma (en script puede guardarse con `savefig`).

5.6. Salida obtenida

Los números cambian en cada ejecución

La medición es aleatoria: los conteos exactos varían. Lo que debe reproducirse es la *tendencia*: la frecuencia de "1" se acerca a 0.25 al aumentar los shots.

```
10 shots -> {'1': 3, '0': 7} | freq(1) = 0.300
100 shots -> {'0': 73, '1': 27} | freq(1) = 0.270
1000 shots -> {'0': 728, '1': 272} | freq(1) = 0.272
10000 shots -> {'0': 7501, '1': 2499} | freq(1) = 0.250
```

5.7. Interpretación

Con 10 shots la frecuencia puede alejarse mucho de 0.25 (error típico ≈ 0.14); con 10 000 queda cerca (error ≈ 0.004). La incertidumbre estadística disminuye como $1/\sqrt{N}$.

5.8. Ejercicios

1. Prepare un qubit con $P(1) = 0.9$ y mida 2000 shots.
2. Repita 20 veces la corrida de 100 shots y grafique la dispersión de la frecuencia; compárela con $\sigma = \sqrt{p(1-p)/N}$.
3. ¿Qué obtiene si mide dos veces seguidas el mismo qubit? (Pista: colapso.)

6. Interferencia cuántica

6.1. Objetivo

Mostrar que las amplitudes se suman y pueden cancelarse, y que por eso el resultado de un circuito depende de las *fases* y no solo de las probabilidades intermedias.

6.2. Conceptos previos

Concepto previo: amplitudes con signo y suma de amplitudes

Ver sección 1. Una amplitud puede ser negativa o compleja; una probabilidad no. Al encadenar puertas, las amplitudes que llegan al mismo resultado se *suman* (con su signo/fase) *antes* de elevar al cuadrado.

6.3. Fundamento teórico

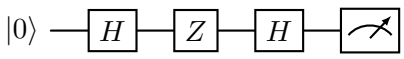
Aplicando H dos veces a $|0\rangle$:

$$H|0\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \tag{6}$$

$$HH|0\rangle = \tfrac{1}{\sqrt{2}}\left[\tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + \tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right] = |0\rangle\,. \tag{7}$$

La amplitud de $|1\rangle$ es $\frac{1}{2} - \frac{1}{2} = 0$ (interferencia *destructiva*); la de $|0\rangle$ es $\frac{1}{2} + \frac{1}{2} = 1$ (*constructiva*). Un modelo clásico probabilístico (50/50 y luego 50/50) daría de nuevo 50/50, porque las probabilidades nunca se cancelan; las amplitudes sí.

Con una fase en medio, Z cambia el signo de $|1\rangle$: $ZH|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$ y $H|-\rangle = |1\rangle$. En general $HZH = X$.



Este es el principio de los algoritmos cuánticos: diseñar fases para que las respuestas incorrectas se cancelen y la correcta se refuerce (capítulo 06).

6.4. Código (completamente comentado)

```python
# %% [markdown]
# # 05a - Interferencia
# Las amplitudes se suman o se cancelan. Como H H = I, el |1> se cancela.

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector

# %% Celda 2: una H frente a dos H
# Una sola H: superposicion 50/50.
una = QuantumCircuit(1)
una.h(0)
p1 = {str(k): round(float(v), 4) for k, v in Statevector(una).probabilities_dict().items()}
print("1 H :", Statevector(una).data.round(3), p1)

# Dos H seguidas: la segunda H combina las dos amplitudes de la primera.
# La contribucion hacia |1> es (+1/2) + (-1/2) = 0: interferencia
# DESTRUCTIVA. La contribucion hacia |0> es (+1/2) + (+1/2) = 1:
# interferencia CONSTRUCTIVA. El qubit vuelve a |0> con certeza.
dos = QuantumCircuit(1)
```

```
dos.h(0)
dos.h(0)
p2 = {str(k): round(float(v), 4) for k, v in Statevector(dos).probabilities_dict().items()}
print("2 H :", Statevector(dos).data.round(3), p2)

# %% Celda 3: la fase decide hacia donde interfiere
# La puerta Z cambia el signo de la amplitud de |1> (fase de pi).
# Entre dos H, ese signo invierte la interferencia: H Z H = X,
# asi que el resultado pasa de |0> a |1> con certeza.
fase = QuantumCircuit(1, 1)
fase.h(0)
fase.z(0)
fase.h(0)
fase.measure(0, 0)
# Esperado: {'1': 1000}. Solo cambiamos una fase y cambio el resultado,
# aunque la Z "no hace nada" a las probabilidades por si sola.
print(AerSimulator().run(fase, shots=1000).result().get_counts())
```

6.5. Explicación celda por celda

Celda 2 Compara un circuito con una H (50/50) y otro con dos H (solo $|0\rangle$). Se usa `Statevector` para ver las amplitudes exactas, sin aleatoriedad.

Celda 3 Inserta Z entre las dos H y mide 1000 shots: sale siempre "1". Solo se cambió una fase.

6.6. Salida obtenida

```
1 H : [0.707+0.j 0.707+0.j] {'0': 0.5, '1': 0.5}
2 H : [1.+0.j 0.+0.j] {'0': 1.0}
{'1': 1000}
```

Es determinista porque la medición final tiene probabilidad 1.

6.7. Interpretación

La puerta Z "no hace nada" a las probabilidades de $|+\rangle$, pero cambia por completo el resultado final. Eso solo es posible porque la información de fase se almacena en el signo de las amplitudes.

6.8. Ejercicios

1. Reemplace Z por la puerta S (fase i) y prediga el resultado antes de ejecutarlo.
2. Pruebe $H\,X\,H$: ¿a qué puerta equivale?
3. Explique por qué un "coin flip" clásico repetido no reproduce el resultado de HH.

7. Entrelazamiento: el estado de Bell

7.1. Objetivo

Construir el estado de Bell $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, demostrar que no se factoriza en dos qubits independientes y observar las correlaciones al medir.

7.2. Conceptos previos

Concepto previo: producto tensorial, estado reducido y entropía

Ver sección 1. Tres ideas nuevas:

- **Estado producto:** $|a\rangle \otimes |b\rangle$; cada qubit tiene su propio estado.
- **Matriz densidad ρ:** describe estados puros y mezclados; para un estado puro $\rho = |\psi\rangle\langle\psi|$.
- **Traza parcial:** "olvidar" un subsistema. El estado del qubit restante es $\rho_0 = \mathrm{Tr}_1\,\rho$. Si ρ_0 es mezclado, el sistema total estaba entrelazado.

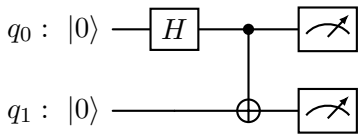
La entropía de von Neumann es $S(\rho) = -\,\mathrm{Tr}(\rho \log_2 \rho)$: vale 0 para un estado puro y 1 bit para un qubit totalmente mezclado ($\rho = I/2$).

7.3. Fundamento teórico

La puerta CNOT (control q_0, objetivo q_1) invierte q_1 si $q_0 = |1\rangle$. Escribiendo los estados como $|q_1 q_0\rangle$ (orden de Qiskit):

$$\mathrm{CNOT}\,|00\rangle = |00\rangle\,, \qquad \mathrm{CNOT}\,|01\rangle = |11\rangle\,. \tag{8}$$

El circuito es



y produce $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

No es un producto. Si $|\Phi^+\rangle = (a\,|0\rangle + b\,|1\rangle) \otimes (c\,|0\rangle + d\,|1\rangle) = ac\,|00\rangle + ad\,|01\rangle + bc\,|10\rangle + bd\,|11\rangle$, habría que tener $ad = 0$ y $bc = 0$ pero $ac = bd = \frac{1}{\sqrt{2}} \neq 0$. Si $a = 0$ o $d = 0$ se contradice $ac \neq 0$ o $bd \neq 0$ respectivamente; así que es imposible.

Cuantificarlo. Para $|\Phi^+\rangle$, $\rho_0 = \mathrm{Tr}_1\,\rho = I/2$, de modo que $S(\rho_0) = 1$ bit: el conjunto está en un estado puro bien definido, pero cada parte por separado es máximamente incierta.

Correlación sin comunicación. Al medir, solo salen 00 o 11 (cada uno con probabilidad $\frac{1}{2}$). Cada resultado individual es aleatorio, por lo que no se puede usar el entrelazamiento para enviar un mensaje (*no-signaling*): no hay transmisión de información más rápida que la luz.

Orden de bits

Qiskit escribe $|q_1 q_0\rangle$. Para este estado el orden no importa porque es simétrico.

7.4. Código (completamente comentado)

```python
# %% [markdown]
# # 05b - Entrelazamiento: estado de Bell (|00> + |11>)/sqrt(2)

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
# partial_trace "olvida" un qubit y deja el estado del otro; entropy mide
# cuanta incertidumbre (mezcla) tiene ese estado reducido.
from qiskit.quantum_info import Statevector, partial_trace, entropy
from qiskit.visualization import plot_histogram

# %% Celda 2: construir el estado de Bell
bell = QuantumCircuit(2)
bell.h(0)        # H sobre el qubit 0: (|0> + |1>)/sqrt(2) en ese qubit
bell.cx(0, 1)    # CNOT: invierte el qubit 1 solo si el qubit 0 vale 1
# Resultado: (|00> + |11>)/sqrt(2). Ojo: Qiskit escribe las cadenas con el
# qubit 0 a la DERECHA (orden little-endian), pero aqui es simetrico.
sv = Statevector(bell)
print("Estado:", sv.data.round(3))  # amplitudes de |00>, |01>, |10>, |11>

# %% Celda 3: prueba de que NO es un producto de dos qubits
# Si el estado fuera producto |a>|b>, cada qubit tendria un estado propio
# bien definido (entropia 0). Aqui, al descartar el qubit 1 el qubit 0
# queda totalmente mezclado: entropia = 1 bit, el maximo para un qubit.
# Eso es entrelazamiento: el todo esta bien definido, las partes no.
rho0 = partial_trace(sv, [1])   # matriz densidad del qubit 0
print("Entropía de entrelazamiento:", entropy(rho0, base=2))  # 1.0

# %% Celda 4: la correlacion al medir
m = bell.copy()
m.measure_all()   # agrega un registro clasico y mide todos los qubits
counts = AerSimulator().run(m, shots=1000).result().get_counts()
# Solo aparecen '00' y '11' (~50% cada uno). Nunca '01' ni '10':
# los resultados estan perfectamente correlacionados, aunque cada uno
# por separado es aleatorio. Esto NO permite enviar informacion
# (no-signaling): el resultado de uno no se puede elegir.
print(counts)
plot_histogram(counts)
```

7.5. Explicación celda por celda

Celda 2 Construye el circuito y obtiene el vector de estado: $(0.707, 0, 0, 0.707)$ para $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.

Celda 3 `partial_trace(sv, [1])` descarta el qubit 1 y `entropy` calcula S: sale 1.0.

Celda 4 Copia el circuito, añade mediciones con `measure_all()` y ejecuta 1000 shots.

7.6. Salida obtenida

```
Estado: [0.707+0.j 0.   +0.j 0.   +0.j 0.707+0.j]
Entropía de entrelazamiento: 1.0
{'11': 503, '00': 497}
```

Los conteos exactos de 00 y 11 varían entre ejecuciones, pero ambos rondan 500 y nunca aparecen 01 ni 10.

7.7. Interpretación

Los resultados de ambos qubits están perfectamente correlacionados aunque cada uno por separado sea aleatorio. Es un recurso que no tiene análogo en probabilidad clásica con variables

locales ocultas (desigualdades de Bell).

7.8. Ejercicios

1. Construya los otros tres estados de Bell ($|\Phi^-\rangle, |\Psi^\pm\rangle$) agregando puertas X y Z al inicio.
2. Calcule la entropía de un estado producto (p. ej. solo H en q_0).
3. Investigue la desigualdad CHSH e impleméntela.

8. Algoritmo de Grover: búsqueda cuadrática

8.1. Objetivo

Implementar Grover para $N = 8$ elementos y comparar el número de consultas con la búsqueda clásica, dejando claro que la ventaja es *cuadrática*, no exponencial.

8.2. Conceptos previos

Concepto previo: oráculo, reflexión y complejidad de consultas

- **Oráculo:** una caja negra O que reconoce la solución. Aquí $O|x\rangle = -|x\rangle$ si x es el elemento marcado y $O|x\rangle = |x\rangle$ si no.
- **Complejidad de consultas:** se cuenta cuántas veces se usa el oráculo, no el tiempo total. Clásico: $O(N)$; Grover: $O(\sqrt{N})$.
- **Interferencia** (capítulo 05a): Grover amplifica la amplitud de la respuesta correcta.

8.3. Fundamento teórico

Se parte de la superposición uniforme $|s\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}}\sum_x |x\rangle$ con $N = 2^n$. Cada iteración aplica $G = D\,O$, donde $D = 2|s\rangle\langle s| - I$ ("difusión": reflexión respecto al promedio de las amplitudes). Geométricamente G gira el estado, en el plano generado por $|s\rangle$ y la solución $|w\rangle$, un ángulo 2θ con

$$\sin\theta = \frac{1}{\sqrt{N}}. \tag{9}$$

Tras k iteraciones, la probabilidad de medir la solución es

$$P_k = \sin^2\big((2k+1)\theta\big), \qquad k_{\text{opt}} \approx \frac{\pi}{4}\sqrt{N}. \tag{10}$$

Para $N = 8$: $\theta = \arcsin(1/\sqrt{8}) \approx 0.3614$ y $k = \lfloor \frac{\pi}{4}\sqrt{8} \rfloor = 2$, de donde $P_2 = \sin^2(5\theta) \approx 0.945$.

Contraste. Clásicamente, buscar un elemento entre N sin estructura requiere $\sim N/2$ consultas en promedio; Grover usa $\sim \frac{\pi}{4}\sqrt{N}$ (mejora cuadrática, que se sabe óptima para este problema). Esto es muy distinto de Shor (factorización), cuya ventaja sobre los mejores algoritmos clásicos conocidos es *exponencial*.

Oráculo en el código. Para marcar $|101\rangle$ se aplica X a los qubits donde esa cadena tiene un 0 (convirtiéndola en $|111\rangle$), luego un CCZ (hecho como H–Toffoli–H en el último qubit), y se deshacen las X. El CCZ solo cambia el signo cuando los tres qubits valen 1.

8.4. Código (completamente comentado)

```
# %% [markdown]
# # 06 - Grover: busqueda cuadratica O(sqrt(N)) frente a O(N)

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
# grover_operator arma el operador de Grover G = (difusion)(oraculo).
from qiskit.circuit.library import grover_operator
from qiskit.quantum_info import Statevector

```

```python
# %% Celda 2: el oraculo
# Problema: entre N = 2^3 = 8 cadenas, encontrar la marcada "101".
# El oraculo es una puerta que cambia el signo (fase -1) SOLO de la
# cadena marcada, sin revelar cual es: U|x> = -|x> si x == "101", y |x> si no.
n, marcado = 3, "101"

# Qiskit numera el qubit 0 a la derecha de la cadena, por eso se invierte
# con reversed(). Buscamos los qubits donde la cadena marcada tiene un 0.
ceros = [i for i, b in enumerate(reversed(marcado)) if b == "0"]

oraculo = QuantumCircuit(n)
oraculo.x(ceros)                    # convierte "101" en "111" temporalmente
oraculo.h(n - 1)                    # H - mcx - H sobre el ultimo qubit
oraculo.mcx(list(range(n - 1)), n - 1)  # equivale a un CCZ: fase -1 si todo es 1
oraculo.h(n - 1)
oraculo.x(ceros)                    # deshace el primer paso

# %% Celda 3: aplicar Grover
G = grover_operator(oraculo)        # un paso = oraculo + difusion
# El numero optimo de iteraciones es ~ (pi/4)*sqrt(N). Aqui es 2.
iteraciones = int(np.floor(np.pi / 4 * np.sqrt(2 ** n)))

qc = QuantumCircuit(n)
qc.h(range(n))                      # superposicion uniforme de las 8 cadenas
for _ in range(iteraciones):
    qc.compose(G, inplace=True)     # agrega G al final del circuito
print("Iteraciones de Grover:", iteraciones)

# Cada iteracion amplifica la amplitud de la cadena marcada por interferencia.
probs = Statevector(qc).probabilities_dict()
print("P(", marcado, ") =", round(float(probs[marcado]), 3))  # ~0.945

# %% Celda 4: comparacion de consultas
# Clasico: en promedio hay que revisar N/2 elementos. Grover: ~(pi/4)*sqrt(N).
# La ventaja es cuadratica (no exponencial como en Shor).
for N in [8, 1_000, 1_000_000, 10**12]:
    print(f"N={N:>16,}  clásico ~ {N // 2:>16,}  Grover ~ {int(np.pi / 4 * np.sqrt(N)):>10,}")
```

8.5. Explicación celda por celda

Celda 2 Construye el oráculo. `reversed(marcado)` se usa porque Qiskit numera el qubit 0 a la derecha. `mcx` es una puerta X con varios controles.

Celda 3 `grover_operator` arma G a partir del oráculo. Se calcula k_{opt}, se aplica H a todos y se repite G k veces. `Statevector` da la probabilidad exacta de `"101"`.

Celda 4 Tabla comparativa de consultas: la ventaja crece con N pero solo como raíz.

8.6. Salida obtenida

```
Iteraciones de Grover: 2
P( 101 ) = 0.945
N=               8  clásico ~               4  Grover ~          2
N=           1,000  clásico ~             500  Grover ~         24
N=       1,000,000  clásico ~         500,000  Grover ~        785
N=1,000,000,000,000  clásico ~ 500,000,000,000  Grover ~    785,398
```

Coincide con la teoría: $P_2 \approx 0.945$.

8.7. Interpretación

Con $N = 10^{12}$ Grover necesita unas 7.9×10^5 consultas frente a 5×10^{11}: una mejora enorme, pero cuadrática. Además cada consulta cuántica puede ser mucho más cara que una clásica, y el oráculo debe construirse como un circuito; por eso la ventaja práctica depende del problema y del hardware.

8.8. Ejercicios

1. Cambie `marcado` a `"010"` y verifique el resultado.
2. Use 1 y 3 iteraciones en lugar de 2: ¿qué pasa con P_k? (Pista: es una función periódica de k; "pasarse" empeora el resultado.)
3. Extienda a $n = 4$ y calcule el k_{opt}.

9. Práctica guiada: Bell, variantes y GHZ

9.1. Objetivo

Ejecutar el circuito de Bell, predecir resultados antes de verlos, modificarlo para distinguir superposición de determinismo, y generalizarlo a tres qubits (estado GHZ).

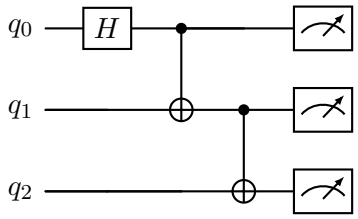
9.2. Conceptos previos

Concepto previo: entrelazamiento y shots

Haber leído el capítulo 05b (estado de Bell) y el 04 (shots y frecuencia).

9.3. Fundamento teórico

- **Bell:** H en q_0 y CNOT $q_0 \to q_1$ dan $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Se esperan $\approx 50\,\%$ de 00 y $\approx 50\,\%$ de 11.
- **Variante con X:** $X|0\rangle = |1\rangle$ no crea superposición; el CNOT copia ese 1, así que el estado es $|11\rangle$ y el resultado es determinista. Esto aísla el papel de H: es la fuente de la aleatoriedad.
- **GHZ:** con H y dos CNOT en cadena se obtiene $\frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$.



9.4. Código (completamente comentado)

```python
# %% [markdown]
# # 07 - Practica guiada: Bell, variantes y reto GHZ

# %% Celda 1: importaciones y simulador
import sys

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# La consola de Windows (cp1252) no imprime el dibujo del circuito sin esto.
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

sim = AerSimulator()  # un solo simulador reutilizado en todas las celdas

# %% Celda 2: circuito de Bell
qc = QuantumCircuit(2, 2)   # 2 qubits y 2 bits clasicos
qc.h(0)                     # superposicion en el qubit 0
qc.cx(0, 1)                 # CNOT: entrelaza el qubit 1 con el 0
qc.measure([0, 1], [0, 1])  # qubit i -> bit clasico i
print(qc.draw())
# Esperado: ~50% '00' y ~50% '11', y casi nunca '01' ni '10'.
print(sim.run(qc, shots=1000).result().get_counts())
```

```
25
26  # %% Celda 3: cambiar h(0) por x(0)
27  # X es el NOT: lleva |0> a |1> sin crear superposicion. El CNOT copia
28  # ese 1 al qubit 1, asi que el resultado es determinista: siempre '11'.
29  # Moraleja: el aleatorio de la celda 2 venia de la superposicion de H.
30  qx = QuantumCircuit(2, 2)
31  qx.x(0)
32  qx.cx(0, 1)
33  qx.measure([0, 1], [0, 1])
34  print(sim.run(qx, shots=1000).result().get_counts())
35
36  # %% Celda 4: reto - estado GHZ de 3 qubits
37  # Generaliza Bell: (|000> + |111>)/sqrt(2). Cada CNOT "propaga" el valor
38  # del qubit anterior al siguiente.
39  ghz = QuantumCircuit(3, 3)
40  ghz.h(0)
41  ghz.cx(0, 1)
42  ghz.cx(1, 2)
43  ghz.measure([0, 1, 2], [0, 1, 2])
44  print(sim.run(ghz, shots=1000).result().get_counts())  # solo '000' y '111'
45
46  # %% Celda 5: reto - efecto del numero de shots
47  # Con pocos shots el histograma es muy irregular (ej. 8 contra 2);
48  # con muchos se acerca al 50/50 teorico. Es ruido estadistico, no fisico.
49  for shots in [10, 100, 10000]:
50      print(shots, sim.run(qc, shots=shots).result().get_counts())
51  plot_histogram(sim.run(qc, shots=10000).result().get_counts())
```

9.5. Explicación celda por celda

Celda 2 Bell con mediciones; `measure([0,1],[0,1])` conecta cada qubit con un bit clásico.

Celda 3 Sustituye `h(0)` por `x(0)`.

Celda 4 Reto: GHZ de 3 qubits.

Celda 5 Reto: distintos números de shots.

9.6. Salida obtenida

```
{'11': 506, '00': 494}
{'11': 1000}
{'000': 503, '111': 497}
10 {'11': 8, '00': 2}
100 {'00': 51, '11': 49}
10000 {'00': 5003, '11': 4997}
```

(El dibujo de texto del circuito, que imprime el script, no se reproduce aquí.)

9.7. Interpretación

- Con x en lugar de h solo sale 11: ya no hay aleatoriedad.
- GHZ solo da 000 y 111.
- Con 10 shots, las proporciones pueden alejarse mucho de 50/50 (aquí, 8 contra 2); con 10 000 se acercan. Es ruido *estadístico*, no físico (el siguiente capítulo introduce el ruido físico).

Pregunta clave: ¿por qué nunca sale 01 ni 10? Porque el CNOT fuerza que q_1 copie el valor de q_0 en la base computacional.

9.8. Ejercicios

1. Prediga y verifique qué ocurre con $h(0)$ seguido de $x(1)$ (sin CNOT).
2. Extienda GHZ a 5 qubits.
3. Mida solo uno de los tres qubits de GHZ y observe cuál es la distribución del resto.

10. Ruido y la era NISQ

10.1. Objetivo

Ver cómo los errores de puertas hacen aparecer resultados que en un circuito ideal serían imposibles (01 y 10 en un estado de Bell).

10.2. Conceptos previos

Concepto previo: canal de ruido y NISQ

- **Canal de ruido:** una operación que describe cómo un error transforma un estado. El canal *depolarizante* sobre k qubits es $\rho \mapsto (1-\lambda)\rho + \lambda \frac{I}{2^k}$: con probabilidad λ el estado se reemplaza por el estado máximamente mezclado.
- **NISQ** (*Noisy Intermediate-Scale Quantum*, Preskill 2018): dispositivos con decenas a pocos cientos de qubits, con errores, sin corrección de errores a escala.
- **Corrección de errores:** codifica un qubit *lógico* en muchos qubits *físicos*; el costo (cientos o más por qubit lógico, según el código y la tasa de error) es una barrera actual.

10.3. Fundamento teórico

El modelo del código aplica error depolarizante de 1 % tras cada H y de 5 % tras cada CNOT. Las puertas de dos qubits tienen más error que las de uno, lo cual es cualitativamente cierto en hardware real. **Estos valores son ilustrativos y no corresponden a ningún dispositivo.**

Con ruido, el estado de salida deja de ser el estado puro de Bell y las mediciones producen también 01 y 10 con probabilidad pequeña. En la corrida mostrada aparecieron 42 de 2000 shots (2.1 %) con paridad incorrecta.

10.4. Código (completamente comentado)

```python
# %% [markdown]
# # 08 - Era NISQ: el ruido produce los resultados 01 y 10

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
# NoiseModel agrupa los errores; depolarizing_error(p, k) es un error
# que, con probabilidad p, reemplaza el estado de k qubits por uno aleatorio.
from qiskit_aer.noise import NoiseModel, depolarizing_error
from qiskit.visualization import plot_histogram

# %% Celda 2: circuito y modelo de ruido
# El mismo circuito de Bell de los ejemplos anteriores.
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

ruido = NoiseModel()
# Error de 1% en cada puerta H (puertas de 1 qubit: errores chicos).
ruido.add_all_qubit_quantum_error(depolarizing_error(0.01, 1), ["h"])
# Error de 5% en cada CNOT. En hardware real las puertas de 2 qubits
# son bastante peores que las de 1, por eso se les da mas error.
ruido.add_all_qubit_quantum_error(depolarizing_error(0.05, 2), ["cx"])
# Los valores son ilustrativos, NO miden ningun equipo concreto.
```

```
26
27 # %% Celda 3: comparar ideal y ruidoso
28 ideal = AerSimulator().run(qc, shots=2000).result().get_counts()
29 # Pasar noise_model hace que el simulador inyecte los errores al ejecutar.
30 noisy = AerSimulator(noise_model=ruido).run(qc, shots=2000).result().get_counts()
31 print("Ideal  :", ideal)    # solo 00 y 11
32 print("Ruidoso:", noisy)    # aparecen 01 y 10: errores que rompen la correlacion
33 # Este es el panorama NISQ (Noisy Intermediate-Scale Quantum): pocos qubits
34 # y con errores, lo que limita la profundidad de los circuitos utiles.
35 plot_histogram([ideal, noisy], legend=["ideal", "ruidoso"])
```

10.5. Explicación celda por celda

Celda 2 Mismo circuito de Bell; se crea un `NoiseModel` y se asocian errores a las puertas `h` y `cx`.

Celda 3 Se ejecuta el circuito en un simulador ideal y en uno con ruido (`noise_model=...`) y se comparan los conteos.

10.6. Salida obtenida

```
Ideal  : {'00': 996, '11': 1004}
Ruidoso: {'11': 993, '00': 960, '01': 21, '10': 26}
```

Los números exactos cambian en cada ejecución.

10.7. Interpretación

El ruido rompe la correlación perfecta. En un equipo real también hay errores de lectura, decoherencia, etc., que este modelo simple no incluye. Comparar este histograma con uno obtenido en hardware real (p. ej. IBM Quantum o las SpinQ) es la mejor forma de ilustrar la era NISQ.

10.8. Ejercicios

1. Aumente el error del CNOT a 20 % y observe el cambio.
2. Mida la fidelidad aproximada como la fracción de shots en 00 o 11.
3. Investigue el error de lectura (`ReadoutError`) y agréguelo.

11. Generador de números aleatorios cuántico (QRNG)

11.1. Objetivo

Generar bits aleatorios midiendo qubits en superposición y usarlos para elegir la jugada de un bot de Tic-Tac-Toe, señalando las limitaciones de la simulación.

11.2. Conceptos previos

Concepto previo: aleatoriedad, uniformidad y módulo

- **Aleatoriedad pseudoaleatoria vs. física:** un PRNG es un algoritmo determinista a partir de una semilla; una fuente física (p. ej. la medición de un qubit real) no lo es.
- **Uniformidad:** todos los resultados posibles tienen igual probabilidad.
- Medición y regla de Born (capítulo 04).

11.3. Fundamento teórico

$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$: medir da 0 o 1 con probabilidad $\frac{1}{2}$. Con n qubits en paralelo se obtienen n bits independientes $\rightarrow$ un entero uniforme en $\{0, \ldots, 2^n - 1\}$.

Sesgo por módulo. Para elegir entre m opciones con x mód m la distribución solo es uniforme si m divide a 2^n. En el código se eligen 4 bits (16 valores) para $m = 5$ casillas; $16 = 3 \cdot 5 + 1$, así que la casilla 0 sale con probabilidad $4/16$ y las demás con $3/16$. Una solución es el *muestreo por rechazo*: descartar los valores ≥ 15 y repetir.

La advertencia importante. `AerSimulator` calcula las probabilidades y luego toma muestras con un generador pseudoaleatorio. Por lo tanto, **en simulación los bits no son aleatoriedad física**. Solo ejecutando en hardware real la fuente es física (aun así, el ruido del equipo puede introducir sesgo y se recomienda post-procesamiento/extractores).

11.4. Código (completamente comentado)

```
# %% [markdown]
# # 09 - Aleatoriedad cuantica (QRNG), conecta con el Tic-Tac-Toe

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

# %% Celda 2: generador de bits
def bits_cuanticos(n_bits: int) -> str:
    """Devuelve una cadena de n_bits bits generada midiendo n_bits qubits en
    superposicion. Con H, cada qubit da 0 o 1 con probabilidad 1/2."""
    qc = QuantumCircuit(n_bits, n_bits)
    qc.h(range(n_bits))                        # cada qubit en (|0> + |1>)/sqrt(2)
    qc.measure(range(n_bits), range(n_bits))
    # memory=True guarda el resultado de CADA shot, no solo el conteo.
    # Con shots=1 obtenemos una unica cadena, p.ej. '11011110'.
    return AerSimulator().run(qc, shots=1, memory=True).result().get_memory()[0]

# %% Celda 3: usar los bits
bits = bits_cuanticos(8)
```

```
print("Bits:", bits, "-> entero:", int(bits, 2))  # int(..., 2) lee base 2

# Elegir una casilla libre del tablero 3x3 (indices 0-8).
# El modulo (%) mapea el entero a un indice valido de la lista.
# Nota: 16 valores % 5 casillas no es perfectamente uniforme (leve sesgo);
# para un uso serio se descartan los valores sobrantes y se repite.
libres = [0, 2, 3, 5, 8]
print("Jugada del bot:", libres[int(bits_cuanticos(4), 2) % len(libres)])

# IMPORTANTE: AerSimulator usa un generador pseudoaleatorio. Solo en
# hardware real la aleatoriedad es de origen fisico.
```

11.5. Explicación celda por celda

Celda 2 La función arma un circuito con n qubits, aplica H a todos y mide. `shots=1, memory=True` devuelve la cadena de esa única ejecución.

Celda 3 Convierte los bits en un entero (`int(bits, 2)`) y elige una casilla libre del tablero con el operador módulo.

11.6. Salida obtenida

```
Bits: 10100010 -> entero: 162
Jugada del bot: 3
```

Distinta en cada ejecución (es el punto del generador).

11.7. Interpretación

Se ilustra el vínculo entre el concepto (medición) y una aplicación concreta (un bot que elige jugadas). El ejemplo también expone una buena práctica de ingeniería: validar que el mapeo de bits a opciones no introduzca sesgo.

11.8. Ejercicios

1. Implemente el muestreo por rechazo y verifique con 10 000 jugadas que las 5 casillas tienen frecuencia $\approx 20\,\%$.
2. Mida con una prueba de χ^2 la uniformidad de 10 000 bits.
3. Cambie el simulador por un backend real (sin ejecutar en la charla) y compare.

12. La esfera de Bloch: un qubit como punto de una esfera

12.1. Objetivo

Entender que todo estado de un qubit se describe con solo dos ángulos, θ y φ, y que por eso se puede dibujar como un punto sobre una esfera.

12.2. Conceptos previos

Concepto previo: amplitudes, fase y regla de Born

Ver capítulo 01 y la sección 1. Hace falta saber que un qubit es $\alpha\,|0\rangle+\beta\,|1\rangle$ con $|\alpha|^2+|\beta|^2=1$, y que $|\alpha|^2$ y $|\beta|^2$ son las probabilidades de medir 0 y 1.

12.3. Fundamento teórico

Los coeficientes de un qubit son números complejos, así que cada uno tiene un módulo y una fase:

$$|\psi\rangle = r_1 e^{i\gamma_1}\,|0\rangle + r_2 e^{i\gamma_2}\,|1\rangle\,. \tag{11}$$

Para simplificar esta expresión se usan dos ideas.

1. **Solo importa la fase relativa.** Multiplicar todo el estado por $e^{i\gamma}$ no cambia nada observable (la *fase global* no se puede medir). Se puede entonces sacar $e^{i\gamma_1}$ como factor común y quedarse solo con la fase relativa $\varphi=\gamma_2-\gamma_1$.
2. **Normalización.** Como $r_1^2+r_2^2=1$ se pueden escribir $r_1=\cos\frac{\theta}{2}$ y $r_2=\sin\frac{\theta}{2}$.

El resultado es

$$|\psi\rangle = \cos\tfrac{\theta}{2}\;|0\rangle + e^{i\varphi}\sin\tfrac{\theta}{2}\;|1\rangle\,. \tag{12}$$

Cada qubit queda determinado por dos ángulos, y por tanto es un punto de una esfera de radio 1: la **esfera de Bloch**, propuesta por Felix Bloch en 1946. El polo norte es $|0\rangle$, el polo sur es $|1\rangle$, el ángulo θ se mide desde el eje z y el ángulo φ gira alrededor de él. En un qubit físico de espín, θ es el ángulo entre el momento angular de espín y el eje z, y φ es la fase del espín.

Las coordenadas del punto son los promedios de las matrices de Pauli:

$$(x,y,z) = \big(\langle X\rangle,\langle Y\rangle,\langle Z\rangle\big) = (\sin\theta\cos\varphi,\ \sin\theta\sin\varphi,\ \cos\theta). \tag{13}$$

Este mismo resultado se usa en el capítulo 13 (tomografía).

Estado	Punto (x,y,z)	Dónde está
$\|0\rangle$	$(0,0,+1)$	polo norte
$\|1\rangle$	$(0,0,-1)$	polo sur
$\|+\rangle=(\|0\rangle+\|1\rangle)/\sqrt{2}$	$(+1,0,0)$	ecuador
$\|-\rangle=(\|0\rangle-\|1\rangle)/\sqrt{2}$	$(-1,0,0)$	ecuador
$(\|0\rangle+i\,\|1\rangle)/\sqrt{2}$	$(0,+1,0)$	ecuador

Ángulo y probabilidades. Las probabilidades dependen solo de θ: $P(0)=\cos^2\frac{\theta}{2}$ y $P(1)=\sin^2\frac{\theta}{2}$. Cambiar φ mueve el punto alrededor del eje z sin tocar z, así que no cambia las probabilidades de esta medición; la fase relativa solo se nota después, por interferencia (capítulo 05a).

12.4. Código (completamente comentado)

```python
# %% [markdown]
# # 10 - La esfera de Bloch: un qubit como un punto en una esfera

# %% Celda 1: importaciones
import numpy as np
# Statevector guarda el estado; Pauli permite calcular los promedios <X>, <Y>, <Z>.
from qiskit.quantum_info import Pauli, Statevector
from qiskit.visualization import plot_bloch_multivector


def vector_bloch(sv):
    """Devuelve (x, y, z) = (<X>, <Y>, <Z>): la posicion del qubit en la esfera."""
    return np.array([sv.expectation_value(Pauli(p)).real for p in "XYZ"])


# %% Celda 2: de dos amplitudes complejas a dos angulos
# Un qubit es  |psi> = r1 e^{i g1}|0> + r2 e^{i g2}|1>.  Dos ideas permiten
# simplificarlo:
#  1) Solo importa la fase RELATIVA (g2 - g1); la fase global no se puede observar.
#  2) Como r1^2 + r2^2 = 1, se pueden escribir r1 = cos(theta/2), r2 = sin(theta/2).
# Resultado:  |psi> = cos(theta/2)|0> + e^{i phi} sin(theta/2)|1>,  con phi = g2 - g1.
# Cada qubit queda determinado por dos angulos (theta, phi): un punto de la esfera.
theta, phi = np.pi / 3, np.pi / 4
sv = Statevector([np.cos(theta / 2), np.exp(1j * phi) * np.sin(theta / 2)])
print("Amplitudes:", sv.data.round(3))

# %% Celda 3: la fase global no se puede observar
# Multiplicar todo el estado por e^{i gamma} no cambia nada medible:
# el punto de la esfera es el mismo.
con_fase_global = Statevector(np.exp(1j * 1.234) * sv.data)
print("Sin fase global:", vector_bloch(sv).round(4))
print("Con fase global:", vector_bloch(con_fase_global).round(4))

# %% Celda 4: los estados especiales
# |0> es el polo norte (z = +1) y |1> el polo sur (z = -1).
# Las superposiciones con igual probabilidad de 0 y 1 caen en el ecuador.
estados = {
    "|0>": Statevector.from_label("0"),
    "|1>": Statevector.from_label("1"),
    "|+>": Statevector.from_label("+"),    # (|0> + |1>)/sqrt2
    "|->": Statevector.from_label("-"),    # (|0> - |1>)/sqrt2
    "|+i>": Statevector.from_label("r"),   # (|0> + i|1>)/sqrt2
}
for nombre, s in estados.items():
    x, y, z = vector_bloch(s)
    print(f"{nombre:5s} -> (x, y, z) = ({x:+.2f}, {y:+.2f}, {z:+.2f})")

# %% Celda 5: theta y phi son los angulos de la esfera
# theta es el angulo desde el eje z (decide las probabilidades de medir 0 o 1).
# phi es el giro alrededor del eje z (la fase relativa).
# El punto es (sin(theta) cos(phi), sin(theta) sin(phi), cos(theta)).
esperado = [np.sin(theta) * np.cos(phi), np.sin(theta) * np.sin(phi), np.cos(theta)]
print("Calculado:", vector_bloch(sv).round(4))
print("Formula  :", np.round(esperado, 4))
print("P(0), P(1) =", sv.probabilities().round(4), " (dependen solo de theta)")

# %% Celda 6: cambiar phi no cambia las probabilidades
# Al girar alrededor del eje z cambian x e y, pero z (y por tanto P(0) y P(1))
# se queda igual. La fase relativa solo se nota despues, por interferencia.
for phi_nuevo in (0, np.pi / 2, np.pi):
    s = Statevector([np.cos(theta / 2), np.exp(1j * phi_nuevo) * np.sin(theta / 2)])
```

```
    x, y, z = vector_bloch(s)
    print(f"phi = {phi_nuevo:4.2f} -> (x, y, z) = ({x:+.3f}, {y:+.3f}, {z:+.3f}) | P(1) = {s.
    ↪ probabilities()[1]:.3f}")

# %% Celda 7: dibujar la esfera
# En Colab o Jupyter la figura se muestra sola.
plot_bloch_multivector(sv)
```

12.5. Explicación celda por celda

Celda 1 `Pauli` y `expectation_value` calculan $\langle X\rangle, \langle Y\rangle, \langle Z\rangle$; con ellos `vector_bloch` da el punto de la esfera.

Celda 2 Crea un qubit con $\theta = \pi/3$ y $\varphi = \pi/4$ a partir de la fórmula del capítulo.

Celda 3 Multiplica el estado por $e^{i\gamma}$ y comprueba que el punto es idéntico.

Celda 4 Calcula el punto de cinco estados especiales.

Celda 5 Compara el punto calculado con la fórmula con θ y φ.

Celda 6 Cambia φ y muestra que z y $P(1)$ no cambian.

Celda 7 Dibuja la esfera con `plot_bloch_multivector`.

12.6. Salida obtenida

```
Amplitudes: [0.866+0.j    0.354+0.354j]
Sin fase global: [0.6124 0.6124 0.5   ]
Con fase global: [0.6124 0.6124 0.5   ]
|0>   -> (x, y, z) = (+0.00, +0.00, +1.00)
|1>   -> (x, y, z) = (+0.00, +0.00, -1.00)
|+>   -> (x, y, z) = (+1.00, +0.00, +0.00)
|->   -> (x, y, z) = (-1.00, +0.00, +0.00)
|+i>  -> (x, y, z) = (+0.00, +1.00, +0.00)
Calculado: [0.6124 0.6124 0.5   ]
Formula  : [0.6124 0.6124 0.5   ]
P(0), P(1) = [0.75 0.25]  (dependen solo de theta)
phi = 0.00 -> (x, y, z) = (+0.866, +0.000, +0.500) | P(1) = 0.250
phi = 1.57 -> (x, y, z) = (+0.000, +0.866, +0.500) | P(1) = 0.250
phi = 3.14 -> (x, y, z) = (-0.866, +0.000, +0.500) | P(1) = 0.250
```

La salida es determinista (no hay medición).

12.7. Interpretación

El punto calculado coincide con la fórmula, con o sin fase global. Al pasar φ de 0 a $\pi/2$ y a π, el punto gira de $(0.866, 0, 0.5)$ a $(0, 0.866, 0.5)$ y a $(-0.866, 0, 0.5)$, pero $P(1) = 0.25$ no cambia.

12.8. Ejercicios

1. ¿Qué θ da $P(1) = 0.9$? Compruébalo con el código.
2. Calcula a mano el punto de $|-i\rangle = (|0\rangle - i\,|1\rangle)/\sqrt{2}$ y verifícalo con `Statevector.from_label("l")`.
3. ¿En qué lugar de la esfera quedan dos estados que solo difieren en una fase global?

12.9. Errores comunes

- Confundir θ con $\theta/2$: en la fórmula del estado aparece $\cos\frac{\theta}{2}$, pero el ángulo que se mide sobre la esfera es θ. Por eso $|0\rangle$ y $|1\rangle$, que son ortogonales, quedan en polos opuestos (a 180°).
- La esfera solo sirve para *un* qubit. Un estado entrelazado de varios qubits no se puede separar en puntos de esferas (capítulos 12 y 13).

13. Oscilación de Rabi: cómo un pulso cambia el estado de un qubit

13.1. Objetivo

Ver cómo un pulso de radiofrecuencia resonante hace girar el estado de un qubit de espín, por qué $P(1)$ oscila con la duración del pulso y cómo se mide la frecuencia de Rabi ω_1 en el laboratorio.

13.2. Conceptos previos

Concepto previo: espín, esfera de Bloch y evolución unitaria

Ver capítulo 10 (esfera de Bloch) y la sección 1. Idea clave: un qubit de espín 1/2 en un campo magnético fijo B_0 (eje z) tiene dos niveles de energía, $|0\rangle$ (espín arriba) y $|1\rangle$ (espín abajo).

13.3. Fundamento teórico

Cuando la frecuencia de un campo de radiofrecuencia (RF) coincide con la del espín (*resonancia*), el espín absorbe energía del campo y cambia de nivel. Esa es la oscilación de Rabi, y es la base del control de qubits en estos sistemas.

Campo de RF y sistema rotante. La bobina genera un campo oscilante $B_{\mathrm{rf}}(t) \propto \cos(2\pi\omega_{\mathrm{rf}}t + \varphi_{\mathrm{rf}})$ a lo largo de x. Un campo linealmente polarizado se puede descomponer en dos campos circulares, de intensidad B_1 cada uno, que giran en sentidos opuestos; con la *aproximación de onda rotante* solo se conserva el que gira en el mismo sentido que el espín. Si el observador gira junto con ese campo (*sistema rotante*), el campo se vuelve estático: tiene intensidad B_1 y dirección $\cos\varphi_{\mathrm{rf}}\,\hat{x} + \sin\varphi_{\mathrm{rf}}\,\hat{y}$, que depende solo de la fase del pulso.

Resonancia. En el sistema rotante, el espín gira alrededor del eje z con frecuencia $\omega_0 - \omega_{\mathrm{rf}}$ (la *desintonia* $\Delta\omega$). Si $\omega_{\mathrm{rf}} = \omega_0$ ese giro desaparece y solo queda el giro alrededor del campo B_1, a la **frecuencia de Rabi** $\omega_1 \propto B_1$. El ángulo de giro es

$$\theta = 2\pi\,\omega_1\,t_{\mathrm{Rabi}}. \tag{14}$$

Qué se mide. Partiendo de $|0\rangle$, el espín gira alrededor de un eje del plano xy, así que oscila entre $|0\rangle$ y $|1\rangle$. Como θ es también el ángulo de la esfera de Bloch,

$$P(1) = \sin^2\frac{\theta}{2} = \sin^2(\pi\,\omega_1\,t), \tag{15}$$

y esa población oscila con frecuencia ω_1. Un pulso con $\theta = \pi$ voltea el qubit (como la puerta X) y uno con $\theta = \pi/2$ lo deja en una superposición 50/50. Antes de usar el sistema para calcular, hay que medir ω_1: es un parámetro del equipo.

Cómo se mide ω_1. Medir colapsa el estado, así que:

1. Se prepara siempre el estado inicial $|0\rangle$.
2. Se aplica un pulso de duración t y se mide muchas veces; el promedio da $P(1)$.
3. Se repite aumentando poco a poco la duración t del pulso, siempre reiniciando.
4. Se ajusta una función seno a los datos; de ahí sale ω_1.

Desintonia. Si el pulso no es resonante, el espín gira a la vez alrededor de z (a frecuencia $\Delta\omega$) y alrededor del eje del pulso. Con una desintonia muy grande el giro alrededor de z cancela el efecto del pulso, que ya no excita al qubit.

Modelo usado en el código. Con $\hbar = 1$, en el sistema rotante,

$$H = \frac{2\pi}{2}\Big[\Delta\omega\,\sigma_z + \omega_1\big(\cos\varphi\,\sigma_x + \sin\varphi\,\sigma_y\big)\Big], \qquad U(t) = e^{-iHt}. \tag{16}$$

Las frecuencias están en Hz y por eso aparece 2π. El capítulo 12 explica de dónde sale $U(t) = e^{-iHt}$.

13.4. Código (completamente comentado)

```
# %% [markdown]
# # 11 - Oscilacion de Rabi: como un pulso de radiofrecuencia cambia al qubit

# %% Celda 1: importaciones
import numpy as np
# expm calcula la exponencial de una matriz: lo necesitamos para evolucionar el estado.
from scipy.linalg import expm
# curve_fit ajusta una curva a datos (como se hace en el laboratorio para hallar w1).
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# Matrices de Pauli. Trabajamos con hbar = 1 y frecuencias en Hz (por eso el 2*pi).
sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)
ket0 = np.array([1, 0], dtype=complex)

# %% Celda 2: el modelo (en el sistema que gira junto con el pulso)
# Un qubit de espin esta en un campo magnetico fijo B0 (eje z) y recibe un
# campo de radiofrecuencia (RF). En el "sistema rotante" el efecto del pulso es
# un campo FIJO de intensidad w1 (frecuencia de Rabi) cuya direccion en el
# plano xy la decide la fase phi del pulso:  eje = cos(phi) x + sin(phi) y.
# Si ademas el pulso esta desafinado (desintonia dw = w0 - w_rf), el qubit gira
# tambien alrededor de z. El Hamiltoniano completo es:
#     H = 2*pi*dw * sz/2  +  2*pi*w1 * (cos(phi) sx + sin(phi) sy)/2
def hamiltoniano(w1, dw=0.0, phi=0.0):
    return 2 * np.pi * (dw * sz + w1 * (np.cos(phi) * sx + np.sin(phi) * sy)) / 2


def estado_tras_pulso(w1, t, dw=0.0, phi=0.0):
    """Estado despues de un pulso de duracion t, partiendo de |0>: U(t)|0>, con U = exp(-iHt)."""
    return expm(-1j * hamiltoniano(w1, dw, phi) * t) @ ket0


# %% Celda 3: con pulso resonante (dw = 0), el angulo de giro es 2*pi*w1*t
# El estado gira alrededor del eje del pulso un angulo  theta = 2*pi*w1*t.
# Partiendo de |0>, el angulo theta de la esfera de Bloch es justo ese,
# asi que  P(1) = sin^2(theta/2) = sin^2(pi*w1*t): oscila entre 0 y 1.
w1 = 1000.0   # frecuencia de Rabi: 1000 Hz (1 oscilacion completa por milisegundo)
for t_ms in (0.25, 0.5, 0.75, 1.0):
    t = t_ms * 1e-3
    p1 = abs(estado_tras_pulso(w1, t)[1]) ** 2
    print(f"t = {t_ms:4.2f} ms -> theta = {2 * np.pi * w1 * t:5.2f} rad | P(1) simulada = {p1:.3f} | sin^2(pi w1 t) = {np.sin(np.pi * w1 * t) ** 2:.3f}")
# t = 0.5 ms es un pulso "pi" (invierte el qubit, como la puerta X);
# t = 0.25 ms es un pulso "pi/2" (superposicion 50/50).
```

```
46
47  # %% Celda 4: la fase del pulso decide el eje de giro
48  # phi = 0 gira alrededor de x; phi = pi/2, alrededor de y. Un pulso pi/2
49  # deja al qubit en puntos distintos del ecuador segun el eje elegido.
50  t = 0.25e-3    # pulso pi/2
51  for phi, nombre in ((0.0, "eje x"), (np.pi / 2, "eje y")):
52      est = estado_tras_pulso(w1, t, phi=phi)
53      x, y, z = (np.real(est.conj() @ (s @ est)) for s in (sx, sy, sz))
54      print(f"phi = {phi:4.2f} ({nombre}) -> (x, y, z) = ({x:+.2f}, {y:+.2f}, {z:+.2f})")
55
56  # %% Celda 5: simular el experimento del laboratorio
57  # En el laboratorio no se mide el estado de un solo golpe. Se hace asi:
58  #   1. Se prepara siempre |0>.
59  #   2. Se aplica un pulso de duracion t y se mide muchas veces (shots).
60  #   3. Se repite aumentando poco a poco la duracion del pulso (cada medicion
61  #      colapsa el estado, asi que se reinicia el experimento).
62  #   4. Se ajusta una funcion seno a los datos y de ahi sale w1.
63  rng = np.random.default_rng(7)          # semilla: el resultado es reproducible
64  tiempos = np.linspace(0, 2e-3, 41)       # anchos de pulso de 0 a 2 ms
65  shots = 500
66  # (min/max evita que un redondeo deje la probabilidad en 1.0000000000000002)
67  p1_medida = np.array([rng.binomial(shots, min(1.0, abs(estado_tras_pulso(w1, t)[1]) ** 2)) /
        ↪ shots for t in tiempos])
68
69
70  def modelo(t, f):
71      return np.sin(np.pi * f * t) ** 2
72
73
74  (f_ajustada,), _ = curve_fit(modelo, tiempos, p1_medida, p0=[900.0])
75  print(f"w1 real = {w1:.0f} Hz | w1 ajustada con {shots} shots = {f_ajustada:.1f} Hz")
76
77  # %% Celda 6: si el pulso esta desafinado, casi no excita
78  # Con desintonia, el qubit gira alrededor de z mientras el pulso intenta
79  # girarlo: P(1) ya no llega a 1. Si la desintonia es muy grande, el pulso
80  # practicamente no tiene efecto.
81  t_fino = np.linspace(0, 2e-3, 401)
82  for dw in (0.0, 500.0, 5000.0):
83      maximo = max(abs(estado_tras_pulso(w1, t, dw=dw)[1]) ** 2 for t in t_fino)
84      print(f"desintonia = {dw:6.0f} Hz -> maximo de P(1) = {maximo:.3f}")
85
86  # %% Celda 7: grafica
87  plt.plot(tiempos * 1e3, p1_medida, "o", label=f"datos ({shots} shots)")
88  plt.plot(t_fino * 1e3, modelo(t_fino, w1), label="teoria sin^2(pi w1 t)")
89  plt.plot(t_fino * 1e3, [abs(estado_tras_pulso(w1, t, dw=500.0)[1]) ** 2 for t in t_fino],
90           "--", label="desintonia 500 Hz")
91  plt.xlabel("ancho del pulso (ms)")
92  plt.ylabel("P(medir 1)")
93  plt.title("Oscilaciones de Rabi")
94  plt.legend()
95  plt.grid(alpha=0.3)
96  plt.show()
```

13.5. Explicación celda por celda

Celda 1 Importa `expm` (exponencial de una matriz), `curve_fit` (ajuste de curvas) y define las matrices de Pauli.

Celda 2 Define H y `estado_tras_pulso`, que calcula $U(t)\,|0\rangle$.

Celda 3 Con pulso resonante compara $P(1)$ simulada con $\sin^2(\pi\omega_1 t)$ para cuatro duraciones.

Celda 4 Muestra que la fase del pulso elige el eje: con $\varphi = 0$ y $\varphi = \pi/2$ un pulso $\pi/2$ deja al qubit en puntos distintos del ecuador.

Celda 5 Simula el experimento del laboratorio: 41 anchos de pulso, 500 mediciones cada uno (con semilla fija), y ajusta ω_1.

Celda 6 Calcula el máximo de $P(1)$ con distintas desintonías.

Celda 7 Grafica datos, teoría y el caso desafinado.

13.6. Salida obtenida

```
t = 0.25 ms -> theta =  1.57 rad | P(1) simulada = 0.500 | sin^2(pi w1 t) = 0.500
t = 0.50 ms -> theta =  3.14 rad | P(1) simulada = 1.000 | sin^2(pi w1 t) = 1.000
t = 0.75 ms -> theta =  4.71 rad | P(1) simulada = 0.500 | sin^2(pi w1 t) = 0.500
t = 1.00 ms -> theta =  6.28 rad | P(1) simulada = 0.000 | sin^2(pi w1 t) = 0.000
phi = 0.00 (eje x) -> (x, y, z) = (+0.00, -1.00, +0.00)
phi = 1.57 (eje y) -> (x, y, z) = (+1.00, -0.00, +0.00)
w1 real = 1000 Hz | w1 ajustada con 500 shots = 1002.4 Hz
desintonia =      0 Hz -> maximo de P(1) = 1.000
desintonia =    500 Hz -> maximo de P(1) = 0.800
desintonia =   5000 Hz -> maximo de P(1) = 0.038
```

La simulación usa semilla fija, así que el ajuste de ω_1 se repite igual; con otra semilla cambiaría un poco.

13.7. Interpretación

Con $\omega_1 = 1000$ Hz, un pulso de 0.5 ms da $P(1) = 1$ (pulso π) y uno de 0.25 ms da 0.5 (pulso $\pi/2$). El ajuste con 500 mediciones por punto recupera $\omega_1 \approx 1002$ Hz. Con desintonía de 500 Hz el máximo de $P(1)$ baja a 0.8, y con 5000 Hz baja a 0.038: el pulso casi no excita.

13.8. Ejercicios

1. Cambia ω_1 a 2000 Hz: ¿cuánto dura ahora un pulso π?
2. Repite el ajuste con 50 mediciones por punto y compara el error.
3. Con $\varphi = \pi/2$, ¿qué estado queda tras un pulso $\pi/2$? Compruébalo y ubícalo en la esfera de Bloch.

13.9. Errores comunes

- Olvidar el 2π al pasar de Hz a radianes por segundo.
- No reiniciar el estado entre mediciones: cada medición colapsa el qubit, así que el experimento se rehace desde $|0\rangle$.

14. Control cuántico: Hamiltoniano, rotaciones y acoplamiento

14.1. Objetivo

Entender de dónde salen las puertas cuánticas: cada una es una evolución unitaria $U(t) = e^{-iHt}$ que se consigue eligiendo el Hamiltoniano H. Se ve cómo se hacen rotaciones de un qubit, qué pasa con un pulso desafinado y cómo el acoplamiento entre dos qubits los entrelaza.

14.2. Conceptos previos

Concepto previo: unitariedad, matrices de Pauli y producto tensorial

Ver sección 1 (matrices unitarias y producto tensorial) y los capítulos 10 y 11. Las matrices de Pauli son $\sigma_x = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, $\sigma_y = \left(\begin{smallmatrix} 0 & -i \\ i & 0 \end{smallmatrix}\right)$ y $\sigma_z = \left(\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right)$.

14.3. Fundamento teórico

Unitariedad. Una operación U es unitaria si $U^\dagger U = I$. Los postulados de la mecánica cuántica dicen que toda evolución física, incluidas las operaciones lógicas de un computador cuántico, es unitaria. Su significado físico es que la suma de probabilidades se conserva:

$$\langle U\psi | U\psi \rangle = \langle \psi | U^\dagger U | \psi \rangle = \langle \psi | \psi \rangle = |\alpha|^2 + |\beta|^2. \tag{17}$$

No se pierde ni se crea nada: la operación no destruye información.

Hamiltoniano y evolución. El Hamiltoniano H es el operador de la energía total del sistema y manda su evolución en el tiempo (ecuación de Schrödinger):

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = H\, |\psi(t)\rangle\,. \tag{18}$$

Con $\hbar = 1$ y un H constante, la solución es $|\psi(t)\rangle = U(t)\,|\psi(0)\rangle$ con

$$U(t) = e^{-iHt}. \tag{19}$$

Controlar un computador cuántico es, básicamente, modificar H para obtener la evolución $U(t)$ deseada. En general $H = H_0 + H_1$: la parte *interna* H_0 (energía Zeeman de cada qubit y acoplamiento entre ellos) y la parte *externa* H_1 (el campo de radiofrecuencia aplicado).

Un qubit: rotaciones. Un pulso resonante de duración t, frecuencia de Rabi ω_1 y fase φ produce

$$U = \exp\Big[-i\,\pi\omega_1 t \,\big(\cos\varphi\,\sigma_x + \sin\varphi\,\sigma_y\big)\Big], \tag{20}$$

una **rotación** alrededor del eje $\cos\varphi\,\hat{x} + \sin\varphi\,\hat{y}$ de la esfera de Bloch, por un ángulo $\theta = 2\pi\omega_1 t$. La fase del pulso elige el eje y la duración elige el ángulo. Se escribe $R_x(\theta)$, $R_y(\theta)$, etc.

La rotación alrededor de z. Un solo pulso no puede girar alrededor de z, pero se puede armar con tres:

1. girar $\pi/2$ alrededor de y (el eje x ocupa el lugar de z);
2. girar θ alrededor de x (equivale a girar alrededor de z);
3. girar $\pi/2$ alrededor de $-y$ para volver al sistema original.

Así, con pulsos de radiofrecuencia se puede girar un qubit alrededor de cualquier eje. El **orden de los pulsos importa**, porque las rotaciones, en general, no conmutan.

Pulso desafinado. Si la frecuencia del pulso difiere de la del qubit en $\Delta\omega$, la evolución combina dos giros a la vez: alrededor de z (a frecuencia $\Delta\omega$) y alrededor del eje del pulso. Con una desintonía grande el pulso no logra excitar al qubit.

Dos qubits acoplados. Con acoplamiento J_{12}, la parte interna del Hamiltoniano (en el sistema doblemente rotante) es $H_0 = 2\pi J_{12}\,\sigma_z \otimes \sigma_z/4$. Es una rotación alrededor del eje $\sigma_z \otimes \sigma_z$, que equivale a decir que *el eje de giro de un qubit depende del estado del otro*: si el otro está en $|0\rangle$ el eje es $+z$ y si está en $|1\rangle$ es $-z$. Si el otro qubit está en una superposición, la frecuencia de precesión también queda en superposición y los dos qubits se **entrelazan**.

Criterio de entrelazamiento. Si los qubits no están entrelazados, el estado total es un producto $\rho = \rho_A \otimes \rho_B$. Si ρ no se puede escribir así, el sistema está entrelazado.

14.4. Código (completamente comentado)

```
# %% [markdown]
# # 12 - Control cuantico: Hamiltoniano, rotaciones y acoplamiento entre qubits

# %% Celda 1: importaciones
import numpy as np
from scipy.linalg import expm
# Statevector es el estado; partial_trace y entropy sirven para ver si hay entrelazamiento.
from qiskit.quantum_info import DensityMatrix, Statevector, entropy, partial_trace

# Matrices de Pauli. Trabajamos con hbar = 1 y frecuencias en Hz (por eso aparece 2*pi).
sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)

# %% Celda 2: la ecuacion de Schrodinger da una operacion unitaria
# El Hamiltoniano H es la energia total del sistema y manda su evolucion:
#     i d|psi>/dt = H |psi>.
# Si H no cambia en el tiempo, la solucion es  |psi(t)> = U(t)|psi(0)>  con
#     U(t) = exp(-i H t).
# "Controlar" un qubit es cambiar H para obtener la U que queremos.
H = 2 * np.pi * 1000 * (0.8 * sx + 0.6 * sz) / 2   # un H cualquiera
U = expm(-1j * H * 0.3e-3)
print("U^dagger U = I (es unitaria):", np.allclose(U.conj().T @ U, np.eye(2)))

# Unitaria significa que la suma de probabilidades sigue valiendo 1: no se
# pierde ni se crea nada, y por eso la operacion se puede invertir.
psi = np.array([0.6, 0.8j])
print("Suma de probabilidades antes  :", round(float(np.sum(abs(psi) ** 2)), 6))
print("Suma de probabilidades despues:", round(float(np.sum(abs(U @ psi) ** 2)), 6))


# %% Celda 3: rotaciones de un qubit con pulsos
# Un pulso de duracion t, con frecuencia de Rabi w1 y fase phi, aplica
#     U = exp(-i * 2*pi*w1 * (cos(phi) sx + sin(phi) sy)/2 * t)
# o sea, una ROTACION de angulo  theta = 2*pi*w1*t  alrededor del eje
# cos(phi) x + sin(phi) y.  phi = 0 -> eje x;  phi = pi/2 -> eje y.
def rotacion(theta, phi):
    """Rotacion de angulo theta alrededor del eje cos(phi) x + sin(phi) y."""
    return expm(-1j * theta / 2 * (np.cos(phi) * sx + np.sin(phi) * sy))


def Rx(theta):
    return rotacion(theta, 0.0)


```

```python
def Ry(theta):
    return rotacion(theta, np.pi / 2)


def Rz(theta):
    return expm(-1j * theta / 2 * sz)   # un pulso solo no la da directamente (ver Celda 4)


w1, t = 1000.0, 0.25e-3                      # pulso de 0.25 ms a 1000 Hz
theta = 2 * np.pi * w1 * t
print(f"theta = 2*pi*w1*t = {theta:.4f} rad (pi/2 = {np.pi / 2:.4f})")
print("El pulso coincide con Rx(pi/2):", np.allclose(expm(-1j * 2 * np.pi * w1 * sx / 2 * t), Rx
    ↪ (np.pi / 2)))

# %% Celda 4: la rotacion alrededor de z se arma combinando tres pulsos
# Un pulso solo puede girar alrededor de ejes del plano xy, no de z. Truco:
#   1. Girar pi/2 alrededor de y   (el eje x pasa a ocupar el lugar de z).
#   2. Girar theta alrededor de x  (equivale a girar alrededor de z).
#   3. Girar pi/2 alrededor de -y  (volver al sistema original).
# El orden importa, porque las rotaciones no conmutan.
theta_z = 0.9
# En un producto de matrices, el primer pulso aplicado queda a la DERECHA.
secuencia = Ry(-np.pi / 2) @ Rx(theta_z) @ Ry(np.pi / 2)
cambiada = Ry(np.pi / 2) @ Rx(theta_z) @ Ry(-np.pi / 2)
print("Ry(pi/2), Rx(th), Ry(-pi/2) == Rz(+th):", np.allclose(secuencia, Rz(theta_z)))
print("Ry(pi/2), Rx(th), Ry(-pi/2) == Rz(-th):", np.allclose(secuencia, Rz(-theta_z)))
print("Con el orden cambiado        == Rz(+th):", np.allclose(cambiada, Rz(theta_z)),
      "| == Rz(-th):", np.allclose(cambiada, Rz(-theta_z)))

# %% Celda 5: un pulso desafinado (fuera de resonancia)
# Si la frecuencia del pulso difiere de la del qubit en dw, aparece un giro
# extra alrededor de z:   H = 2*pi*(dw sz + w1 sx)/2.
# Con desintonia grande el pulso casi no logra voltear al qubit.
ket0 = np.array([1, 0], dtype=complex)
for dw in (0.0, 1000.0, 10000.0):
    Hd = 2 * np.pi * (dw * sz + w1 * sx) / 2
    p1 = abs((expm(-1j * Hd * 0.5e-3) @ ket0)[1]) ** 2   # pulso "pi" de 0.5 ms
    print(f"desintonia = {dw:7.0f} Hz -> P(1) tras un pulso pi = {p1:.3f}")

# %% Celda 6: dos qubits acoplados
# Cuando dos qubits interactuan con acoplamiento J, el Hamiltoniano es
#     H = 2*pi*J * (sz (x) sz)/4.
# Es una rotacion alrededor del eje sz(x)sz: el eje de giro de un qubit
# DEPENDE del estado del otro (+z si el otro es |0>, -z si es |1>).
J = 696.0                        # acoplamiento en Hz
zz = np.kron(sz, sz)


def U_acople(t):
    return expm(-1j * 2 * np.pi * J * zz / 4 * t)


# El qubit 0 empieza en |+> (sobre el ecuador). Segun el estado del otro qubit,
# acumula una fase relativa de signo contrario.
t = 1 / (2 * J)                  # con este tiempo el angulo de rotacion es pi/2
mas = np.array([1, 1], dtype=complex) / np.sqrt(2)
for otro, nombre in ((np.array([1, 0], dtype=complex), "|0>"), (np.array([0, 1], dtype=complex),
    ↪  "|1>")):
    # np.kron(otro, mas): el factor de la izquierda es el qubit 1 y el de la derecha el qubit 0.
    estado = Statevector(U_acople(t) @ np.kron(otro, mas))
    rho = partial_trace(DensityMatrix(estado), [1]).data   # qubit 0 solo
    x, y = np.real(np.trace(rho @ sx)), np.real(np.trace(rho @ sy))
    print(f"otro qubit en {nombre}: qubit 0 queda en (x, y) = ({x:+.2f}, {y:+.2f})")
```

```
107
108 # %% Celda 7: el acoplamiento entrelaza
109 # Desde |++>, tras t = 1/(2J) los dos qubits quedan entrelazados. Criterio:
110 # un estado NO entrelazado se puede escribir como producto de los estados de
111 # cada qubit (rho = rho_A (x) rho_B). Si no se puede, hay entrelazamiento.
112 inicial = np.kron(mas, mas)
113 for t in (0.0, 1 / (2 * J)):
114     rho = DensityMatrix(Statevector(U_acople(t) @ inicial))
115     rho_A, rho_B = partial_trace(rho, [1]), partial_trace(rho, [0])
116     es_producto = np.allclose(rho.data, np.kron(rho_B.data, rho_A.data))
117     print(f"t = {t * 1e3:.3f} ms: rho == rho_A (x) rho_B? {es_producto} | "
118           f"entropia del qubit 0 = {float(entropy(rho_A, base=2)):.3f} bit")
```

14.5. Explicación celda por celda

Celda 2 Calcula $U = e^{-iHt}$ para un H cualquiera y comprueba $U^\dagger U = I$ y que la suma de probabilidades sigue valiendo 1.

Celda 3 Define R_x, R_y, R_z y verifica que un pulso de 0.25 ms a 1000 Hz es $R_x(\pi/2)$.

Celda 4 Arma $R_z(\theta)$ con la secuencia de tres pulsos y comprueba que el orden contrario da $R_z(-\theta)$.

Celda 5 Aplica un pulso π con distintas desintonías.

Celda 6 Con $J = 696$ Hz y $t = 1/(2J)$ deja al qubit 0 en $|+\rangle$ y muestra que su fase cambia de signo según el estado del qubit 1.

Celda 7 Comprueba, desde $|{+}{+}\rangle$, que tras $t = 1/(2J)$ el estado ya no es un producto.

14.6. Salida obtenida

```
U^dagger U = I (es unitaria): True
Suma de probabilidades antes  : 1.0
Suma de probabilidades despues: 1.0
theta = 2*pi*w1*t = 1.5708 rad (pi/2 = 1.5708)
El pulso coincide con Rx(pi/2): True
Ry(pi/2), Rx(th), Ry(-pi/2) == Rz(+th): True
Ry(pi/2), Rx(th), Ry(-pi/2) == Rz(-th): False
Con el orden cambiado        == Rz(+th): False | == Rz(-th): True
desintonia =       0 Hz -> P(1) tras un pulso pi = 1.000
desintonia =    1000 Hz -> P(1) tras un pulso pi = 0.317
desintonia =   10000 Hz -> P(1) tras un pulso pi = 0.000
otro qubit en |0>: qubit 0 queda en (x, y) = (+0.00, +1.00)
otro qubit en |1>: qubit 0 queda en (x, y) = (+0.00, -1.00)
t = 0.000 ms: rho == rho_A (x) rho_B? True | entropia del qubit 0 = 0.000 bit
t = 0.718 ms: rho == rho_A (x) rho_B? False | entropia del qubit 0 = 1.000 bit
```

Es determinista (no hay medición).

14.7. Interpretación

La suma de probabilidades queda en 1 antes y después de U. Los tres pulsos $R_y(\pi/2)$, $R_x(\theta)$, $R_y(-\pi/2)$ dan $R_z(+\theta)$, y con el orden cambiado se obtiene $R_z(-\theta)$. Con desintonía de 1000 Hz un pulso π solo logra $P(1) = 0.317$, y con 10 000 Hz, 0.000. Con el qubit 1 en $|0\rangle$ o en $|1\rangle$, el qubit 0 acaba en $(x, y) = (0, +1)$ o $(0, -1)$: fases opuestas. Tras $t = 0.718$ ms el estado ya no es un producto y la entropía de un qubit es 1 bit (máximo).

14.8. Ejercicios

1. Construye $R_z(\theta)$ con el orden contrario de pulsos ($R_y(-\pi/2)$ primero) y comprueba qué signo obtienes.

2. ¿Qué tiempo t hace falta con $J = 696$ Hz para una rotación de π en el eje $\sigma_z \otimes \sigma_z$?

3. En la Celda 7, calcula la entropía para varios tiempos entre 0 y $1/J$ y observa cuándo es máxima.

14.9. Errores comunes

- Multiplicar las matrices en el orden en que se aplican los pulsos: en un producto de matrices el primer pulso aplicado queda a la *derecha*.
- Con `np.kron(A, B)`, el factor de la izquierda es el qubit 1 y el de la derecha el qubit 0 (orden de Qiskit).

15. Tomografía de estados: reconstruir un estado a partir de mediciones

15.1. Objetivo

Entender cómo se lee por completo el estado de uno o dos qubits, midiendo en varios ejes y reconstruyendo la matriz densidad ρ.

15.2. Conceptos previos

Concepto previo: esfera de Bloch, matrices de Pauli y entrelazamiento

Ver capítulo 10 (esfera de Bloch), capítulo 04 (medición y shots) y el capítulo 12 (producto tensorial y entrelazamiento). Una *matriz densidad* ρ es otra forma de describir un estado; para un estado puro es $|\psi\rangle\langle\psi|$.

15.3. Fundamento teórico

Al terminar un cálculo hay que leer el estado de los qubits. La **tomografía cuántica** es la técnica para leerlo completo.

Un qubit. Las coordenadas del qubit en la esfera de Bloch son $(a, b, c) = (\langle\sigma_x\rangle, \langle\sigma_y\rangle, \langle\sigma_z\rangle)$, los promedios de los momentos angulares de espín en cada eje. Medir colapsa el estado y solo da el resultado de un autoestado, pero la información queda en las probabilidades:

$$P(0) = \frac{1+c}{2}, \qquad P(1) = \frac{1-c}{2} \implies c = P(0) - P(1). \tag{21}$$

Repitiendo muchas veces se estima c. Con (a, b, c) se reconstruye

$$\rho = \frac{1}{2}\big(I + a\,\sigma_x + b\,\sigma_y + c\,\sigma_z\big). \tag{22}$$

Medir x e y. Algunos sistemas solo pueden medir en el eje z. Para medir $\langle\sigma_x\rangle$ y $\langle\sigma_y\rangle$ se aplican antes unos pulsos de observación $\pi/2$ que giran el eje que se quiere medir hasta z. El cuadro completo se obtiene solo al reunir los tres ejes perpendiculares (una "vista en tres ángulos").

Varios qubits. Para n qubits, ρ se expande en los productos de matrices de Pauli ($\sigma_0 = I$, $\sigma_1 = \sigma_x$, $\sigma_2 = \sigma_y$, $\sigma_3 = \sigma_z$). Para dos qubits:

$$\rho = \frac{1}{4}\sum_{i,j=0}^{3} c_{ij}\,\sigma_i \otimes \sigma_j, \qquad c_{00} = 1, \tag{23}$$

una suma de 16 términos, de los cuales $c_{00} = 1$ es la normalización y hay que medir los otros 15. Los términos como $\sigma_x \otimes I$ son el momento angular de un solo qubit; los términos como $\sigma_z \otimes \sigma_z$ son la *correlación* entre los dos. Para más qubits el principio es el mismo y solo aumenta el número de mediciones.

Por qué no basta medir cada qubit por separado

Si hubiera entrelazamiento, el estado completo no se puede obtener midiendo cada qubit por separado y juntando los resultados, porque parte de la información está en las correlaciones.

Diferencia con el sistema de resonancia magnética

En un sistema de resonancia magnética nuclear (RMN) las mediciones son sobre un *conjunto* de moléculas y la señal medida (el *FID*) trae a la vez información de dos ejes, por lo que bastan 6 observaciones para dos qubits. En este capítulo se simula un computador que mide solo en z: el principio (reconstruir ρ con promedios de Pauli) es el mismo, pero se hacen más mediciones (el código mide cada uno de los 15 coeficientes por separado).

15.4. Código (completamente comentado)

```python
# %% [markdown]
# # 13 - Tomografia de estados: reconstruir un estado a partir de mediciones

# %% Celda 1: importaciones
import itertools
import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector

I2 = np.eye(2, dtype=complex)
sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)
PAULI = {"I": I2, "X": sx, "Y": sy, "Z": sz}
sim = AerSimulator(seed_simulator=123)   # semilla fija: las salidas se pueden reproducir


def fidelidad(rho, estado_puro):
    """Fidelidad de rho con un estado puro: <psi| rho |psi> (1 = identico)."""
    return float(np.real(estado_puro.data.conj() @ rho @ estado_puro.data))


# %% Celda 2: la idea
# Al terminar un calculo hay que "leer" el estado de los qubits. Medir lo
# destruye (colapso), pero la informacion queda escrita en las PROBABILIDADES.
# Las coordenadas del qubit en la esfera de Bloch son (a, b, c) = (<X>, <Y>, <Z>), y
#     P(0) = (1 + c)/2,   P(1) = (1 - c)/2.
# Repitiendo muchas veces la medicion se estima c. Con (a, b, c) se arma la
# matriz densidad:
#     rho = (I + a sx + b sy + c sz) / 2.
# Eso es la tomografia: medir los tres ejes y reconstruir rho.


# %% Celda 3: medir el promedio de un producto de Pauli
# Solo se puede medir en el eje z. Para medir x o y se gira el qubit antes de
# medir (en el laboratorio son pulsos pi/2) y asi ese eje queda en z.
def promedio(preparacion, paulis, shots):
    """Estima <paulis> (por ejemplo "X", "ZZ" o "XI") de un estado.

    `paulis` tiene una letra por qubit, escrita de izquierda a derecha como
    qubit 0, qubit 1, ... La letra "I" significa que ese qubit no se usa.
    """
    qc = preparacion.copy()
    for q, p in enumerate(paulis):
        if p == "X":
            qc.h(q)          # lleva x a z
        elif p == "Y":
            qc.sdg(q)        # sdg y luego h llevan y a z
            qc.h(q)
    qc.measure_all()
```

```
    cuentas = sim.run(qc, shots=shots).result().get_counts()
    total = 0
    for bits, n in cuentas.items():
        bits = bits[::-1]    # Qiskit escribe el qubit 0 a la derecha; lo ponemos a la izquierda
        # El resultado de la medicion vale +1 si hay un numero par de 1s entre
        # los qubits medidos, y -1 si es impar.
        unos = sum(1 for q, p in enumerate(paulis) if p != "I" and bits[q] == "1")
        total += (-1) ** unos * n
    return total / shots


# %% Celda 4: reconstruir un qubit
# Preparamos |psi> = cos(theta/2)|0> + e^{i phi} sin(theta/2)|1> y lo medimos
# como si no lo conociéramos. Luego comparamos con el estado real.
theta, phi = np.pi / 3, np.pi / 4
prep = QuantumCircuit(1)
prep.ry(theta, 0)
prep.rz(phi, 0)
real = Statevector(prep)
print("Bloch real:", np.round([np.sin(theta) * np.cos(phi), np.sin(theta) * np.sin(phi), np.cos(
    ↪ theta)], 3))
for shots in (100, 1000, 10000):
    a, b, c = (promedio(prep, eje, shots) for eje in "XYZ")
    rho = (I2 + a * sx + b * sy + c * sz) / 2
    fid = fidelidad(rho, real)
    largo = np.sqrt(a**2 + b**2 + c**2)
    print(f"{shots:6d} shots -> (a, b, c) = ({a:+.3f}, {b:+.3f}, {c:+.3f}) | |(a,b,c)| = {largo
    ↪ :.3f} | fidelidad = {fid:.4f}")
# Un estado fisico tiene |(a,b,c)| <= 1. Con pocos shots el ruido estadistico
# puede hacerlo un poco mayor: entonces rho reconstruida no es un estado valido
# y la fidelidad puede salir un poco por encima de 1 (como con 100 shots).
# La fidelidad (1 = identico) mejora al aumentar los shots, como en el capitulo 04.

# %% Celda 5: varios qubits - hay que medir TODAS las correlaciones
# Para dos qubits, rho se expande en los 16 productos de Pauli:
#     rho = (1/4) * suma de  c_ij * (sigma_i (x) sigma_j),   con c_II = 1.
# Los c_ij se llaman coeficientes. No basta con hacer tomografia de cada qubit
# por separado: si estan entrelazados, parte de la informacion esta en las
# correlaciones entre ellos (terminos como XX, YY o ZZ).
def tomografia_2_qubits(preparacion, shots):
    rho = np.zeros((4, 4), dtype=complex)
    coefs = {}
    for p0, p1 in itertools.product("IXYZ", repeat=2):
        c = 1.0 if (p0, p1) == ("I", "I") else promedio(preparacion, p0 + p1, shots)
        coefs[p0 + p1] = c
        # np.kron(A, B): A actua en el qubit 1 (izquierda) y B en el qubit 0 (derecha).
        rho += c * np.kron(PAULI[p1], PAULI[p0]) / 4
    return rho, coefs


# %% Celda 6: tomografia de un estado de Bell
bell = QuantumCircuit(2)
bell.h(0)
bell.cx(0, 1)
rho_bell, coefs = tomografia_2_qubits(bell, 5000)
print("Coeficientes distintos de 0 (el resto ~ 0):", {k: round(v, 2) for k, v in coefs.items()
    ↪ if abs(v) > 0.2})
print("Fidelidad con el Bell ideal:", round(fidelidad(rho_bell, Statevector(bell)), 4))

# %% Celda 7: por que no basta con medir cada qubit por separado
# Cada qubit del estado de Bell, visto solo, esta totalmente mezclado
# (coeficientes XI, YI, ZI, IX, IY, IZ ~ 0, es decir rho = I/2). Si juntamos
# esas dos tomografias en un producto obtenemos I/4, que NO es el estado real:
```

```python
# lo que le falta son las correlaciones XX, YY y ZZ.
producto = np.kron(I2 / 2, I2 / 2)
print("Fidelidad del producto I/4 con el Bell real:", round(fidelidad(producto, Statevector(bell
    ↪ )), 4))
```

15.5. Explicación celda por celda

Celda 1 Matrices de Pauli y un simulador con semilla fija.

Celda 3 `promedio` estima $\langle \sigma_{p_0} \otimes \sigma_{p_1} \otimes \cdots \rangle$: añade los giros que llevan cada eje a z (`h` para x; `sdg` y `h` para y), mide y calcula el promedio con signo $+1$ o -1 según haya un número par o impar de unos.

Celda 4 Reconstruye ρ de un qubit con 100, 1000 y 10 000 mediciones y calcula la fidelidad con el estado real.

Celda 5 `tomografia_2_qubits` mide los 15 coeficientes distintos de c_{00} y arma ρ.

Celda 6 Aplica la tomografía a un estado de Bell.

Celda 7 Compara con el producto $\frac{I}{2} \otimes \frac{I}{2}$ de las tomografías por separado.

15.6. Salida obtenida

```
Bloch real: [0.612 0.612 0.5  ]
   100 shots -> (a, b, c) = (+0.640, +0.640, +0.520) | |(a,b,c)| = 1.044 | fidelidad = 1.0219
  1000 shots -> (a, b, c) = (+0.588, +0.588, +0.498) | |(a,b,c)| = 0.969 | fidelidad = 0.9846
 10000 shots -> (a, b, c) = (+0.609, +0.609, +0.494) | |(a,b,c)| = 0.993 | fidelidad = 0.9966
Coeficientes distintos de 0 (el resto ~ 0): {'II': 1.0, 'XX': 1.0, 'YY': -1.0, 'ZZ': 1.0}
Fidelidad con el Bell ideal: 1.0
Fidelidad del producto I/4 con el Bell real: 0.25
```

Se usa semilla fija, así que los números se repiten; con otra semilla los valores de las mediciones cambian ligeramente.

15.7. Interpretación

Con 100 mediciones el módulo de (a, b, c) sale 1.044, mayor que 1: el ruido estadístico produce una ρ que no es un estado válido, y por eso la "fidelidad" pasa de 1. Con 10 000 mediciones el módulo es 0.993 y la fidelidad 0.9966. Para el estado de Bell los únicos coeficientes distintos de cero son $c_{xx} = 1$, $c_{yy} = -1$ y $c_{zz} = 1$ (más $c_{00} = 1$), y la fidelidad con el Bell ideal es 1. En cambio, el producto de las tomografías de cada qubit por separado da $I/4$, con fidelidad de solo 0.25: las correlaciones eran justo lo que faltaba.

15.8. Ejercicios

1. Haz la tomografía del estado $|+\rangle$ y verifica que $(a, b, c) \approx (1, 0, 0)$.
2. Haz la tomografía de dos qubits en el estado $|00\rangle$. ¿Qué coeficientes son distintos de cero?
3. Cuenta cuántos coeficientes habría que medir para 3 qubits.

15.9. Errores comunes

- Esperar un resultado exacto con pocas mediciones: el error baja como $1/\sqrt{N}$ (capítulo 04).
- Confundir el orden de las letras: en `promedio("XZ", ...)` la primera letra es el qubit 0 y la segunda el qubit 1.

16. Puertas universales y lógica clásica reversible

16.1. Objetivo

Ver que toda puerta de un qubit se arma con rotaciones alrededor de z y de y, cómo se construye su versión controlada y cómo la puerta Toffoli permite hacer cálculos de lógica clásica (AND, OR, XOR…) con un computador cuántico.

16.2. Conceptos previos

Concepto previo: rotaciones y puertas controladas

Ver capítulo 12 (rotaciones R_y y R_z) y la sección 1 (CNOT). Una puerta *controlada* aplica una operación al qubit objetivo solo si el qubit de control vale $|1\rangle$.

16.3. Fundamento teórico

Un algoritmo cuántico es un circuito: puertas conectadas en un orden determinado. Este capítulo muestra que con puertas de uno, dos y tres qubits se puede armar cualquier operación.

Puerta universal de un qubit. Toda puerta de un qubit es una matriz unitaria 2×2 y se puede escribir como una rotación alrededor de z, otra alrededor de y y otra alrededor de z, más una fase global:

$$U = e^{i\alpha} R_z(\beta)\, R_y(\gamma)\, R_z(\delta), \tag{24}$$

con $R_z(\theta) = e^{-i\theta Z/2}$ y $R_y(\theta) = e^{-i\theta Y/2}$. Los ángulos β, γ, δ son los de las rotaciones sobre la esfera de Bloch.

Forma $U = e^{i\alpha} AXBXC$. Se puede escribir U con tres matrices unitarias A, B, C que cumplen $ABC = I$:

$$A = R_z(\beta) R_y(\tfrac{\gamma}{2}), \quad B = R_y(-\tfrac{\gamma}{2}) R_z(-\tfrac{\delta+\beta}{2}), \quad C = R_z(\tfrac{\delta-\beta}{2}). \tag{25}$$

Como $X R_y(\theta) X = R_y(-\theta)$ y $X R_z(\theta) X = R_z(-\theta)$, se obtiene $e^{i\alpha} AXBXC = U$. Esto sirve para la puerta controlada: si el control vale $|0\rangle$ las X no actúan y se aplica $ABC = I$ (no pasa nada); si vale $|1\rangle$ se aplica $AXBXC$, que es U (salvo la fase $e^{i\alpha}$, que se agrega con una puerta de fase en el control). Así, una puerta controlada se arma con puertas de uno y dos qubits. Con n controles se obtiene la puerta $C^n(U)$, que aplica U solo si todos valen $|1\rangle$.

Toffoli. La puerta Toffoli es la $C^2(X)$: tiene dos qubits de control y uno objetivo, y lo invierte solo cuando ambos controles valen $|1\rangle$.

Lógica clásica con puertas cuánticas. Toda puerta lógica clásica se puede convertir en una puerta cuántica reversible hecha con X (NOT), CNOT y Toffoli. Con las entradas A, B y la salida C:

- **AND:** Toffoli con A, B de controles y C (en $|0\rangle$) como objetivo: C pasa a $|1\rangle$ solo si $A = B = 1$.
- **OR:** el objetivo C empieza en $|1\rangle$; se aplica X a A y B, luego la Toffoli, y de nuevo X a A y B. El objetivo pasa a $|0\rangle$ solo si $A = B = 0$.
- **NAND, NOR, XOR...:** se arman con las mismas piezas (en este ejemplo, NAND y NOR añaden una X a la salida; XOR son dos CNOT hacia C).

16.4. Código (completamente comentado)

```python
# %% [markdown]
# # 14 - Puertas universales y logica clasica reversible (Toffoli)

# %% Celda 1: importaciones
import itertools
import numpy as np
from scipy.linalg import expm
from qiskit import QuantumCircuit
# OneQubitEulerDecomposer encuentra los
# angulos de una puerta de un qubit.
from qiskit.quantum_info import Statevector, random_unitary
from qiskit.synthesis import OneQubitEulerDecomposer

sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)


def Rz(t):
    return expm(-1j * t / 2 * sz)


def Ry(t):
    return expm(-1j * t / 2 * sy)


# %% Celda 2: cualquier puerta de un qubit es  U = e^{i a} Rz(b) Ry(g) Rz(d)
# Toda puerta de un qubit es una matriz unitaria 2x2. Siempre se puede escribir como
# una rotacion alrededor de z, luego una alrededor de y y otra alrededor de z, mas una
# fase global e^{i a}. Probamos con una unitaria al azar.
U = random_unitary(2, seed=5).data
# Qiskit calcula los angulos (theta = g, phi, lam) de Rz(phi) Ry(theta) Rz(lam).
theta, phi, lam, fase = OneQubitEulerDecomposer("ZYZ").angles_and_phase(U)
reconstruida = np.exp(1j * fase) * Rz(phi) @ Ry(theta) @ Rz(lam)
print("U se reconstruye con Rz Ry Rz:", np.allclose(U, reconstruida))
print(f"angulos: beta = {phi:.3f}, gamma = {theta:.3f}, delta = {lam:.3f}")

# %% Celda 3: U = e^{i a} A X B X C  con  A B C = I
# Esta descomposicion permite construir la version "controlada" de U (aplicar U solo
# si otro qubit vale 1) con puertas de uno y dos qubits. Se toma:
#   A = Rz(b) Ry(g/2),   B = Ry(-g/2) Rz(-(d+b)/2),   C = Rz((d-b)/2).
beta, gamma, delta = phi, theta, lam
A = Rz(beta) @ Ry(gamma / 2)
B = Ry(-gamma / 2) @ Rz(-(delta + beta) / 2)
C = Rz((delta - beta) / 2)
print("A B C = I      :", np.allclose(A @ B @ C, np.eye(2)))
print("e^{ia} A X B X C = U:", np.allclose(np.exp(1j * fase) * A @ sx @ B @ sx @ C, U))
# Si el qubit de control vale 0 se aplica A B C = I (no pasa nada); si vale 1, las X
# se activan y se aplica A X B X C = U.

# %% Celda 4: la puerta Toffoli (CCX)
# Tiene tres qubits: dos de control y uno objetivo. El objetivo se invierte solo si
# los dos controles valen 1. En Qiskit el orden de la cadena es 'q2 q1 q0': aqui
# q0 y q1 son los controles y q2 el objetivo.
toffoli = QuantumCircuit(3)
toffoli.ccx(0, 1, 2)
print("Tabla de verdad (entrada q2q1q0 -> salida q2q1q0):")
for bits in itertools.product("01", repeat=3):
    entrada = "".join(bits)
    salida = Statevector.from_label(entrada).evolve(toffoli).probabilities_dict()
    print(f"  {entrada} -> {max(salida, key=salida.get)}")
```

```
# Solo cambian 011 y 111: el objetivo (q2) se invierte cuando q0 = q1 = 1.

# %% Celda 5: computacion clasica con puertas cuanticas
# Toda puerta logica clasica se puede reescribir con puertas reversibles: X (NOT),
# CNOT y Toffoli. Las entradas son A y B (q0 y q1); la salida queda en C (q2).
def circuito_logico(nombre):
    qc = QuantumCircuit(3)
    if nombre == "AND":                # C empieza en 0; Toffoli lo invierte si A = B = 1
        qc.ccx(0, 1, 2)
    elif nombre == "OR":               # C empieza en 1; sale 0 solo si A = B = 0
        qc.x(2)
        qc.x([0, 1])
        qc.ccx(0, 1, 2)
        qc.x([0, 1])
    elif nombre == "NAND":             # AND y luego NOT a la salida
        qc.ccx(0, 1, 2)
        qc.x(2)
    elif nombre == "NOR":              # OR y luego NOT a la salida
        qc.compose(circuito_logico("OR"), inplace=True)
        qc.x(2)
    elif nombre == "XOR":              # C = A xor B: dos CNOT hacia C
        qc.cx(0, 2)
        qc.cx(1, 2)
    elif nombre == "XNOR":             # XOR y luego NOT a la salida
        qc.compose(circuito_logico("XOR"), inplace=True)
        qc.x(2)
    return qc


clasicas = {
    "AND": lambda a, b: a & b, "OR": lambda a, b: a | b,
    "NAND": lambda a, b: 1 - (a & b), "NOR": lambda a, b: 1 - (a | b),
    "XOR": lambda a, b: a ^ b, "XNOR": lambda a, b: 1 - (a ^ b),
}

# %% Celda 6: comprobar las tablas de verdad
# Para cada puerta, se pone A y B, se corre el circuito y se lee C. Para OR (y NOR)
# el qubit C debe empezar en 1 (lo hace el circuito mismo con la X inicial).
for nombre, f in clasicas.items():
    ok = True
    for a, b in itertools.product((0, 1), repeat=2):
        # Cadena q2 q1 q0 = "0" + str(b) + str(a), con C = q2 en 0.
        inicial = Statevector.from_label(f"0{b}{a}")
        probs = inicial.evolve(circuito_logico(nombre)).probabilities_dict()
        final = max(probs, key=probs.get)
        c = int(final[0])
        ok &= (c == f(a, b))
    print(f"{nombre:5s} coincide con la tabla de verdad clasica: {ok}")
```

16.5. Explicación celda por celda

Celda 2 Toma una unitaria al azar, calcula sus ángulos β, γ, δ con `OneQubitEulerDecomposer` y comprueba que se reconstruye con $R_z R_y R_z$.

Celda 3 Construye A, B, C con esos ángulos y verifica $ABC = I$ y $e^{i\alpha} AXBXC = U$.

Celda 4 Imprime la tabla de verdad de la Toffoli.

Celda 5 Define el circuito de cada puerta lógica.

Celda 6 Compara cada circuito con la tabla de verdad clásica, para las cuatro combinaciones de A y B.

16.6. Salida obtenida

```
U se reconstruye con Rz Ry Rz: True
angulos: beta = -4.179, gamma = 0.822, delta = 0.990
A B C = I      : True
e^{ia} A X B X C = U: True
Tabla de verdad (entrada q2q1q0 -> salida q2q1q0):
  000 -> 000
  001 -> 001
  010 -> 010
  011 -> 111
  100 -> 100
  101 -> 101
  110 -> 110
  111 -> 011
AND   coincide con la tabla de verdad clasica: True
OR    coincide con la tabla de verdad clasica: True
NAND  coincide con la tabla de verdad clasica: True
NOR   coincide con la tabla de verdad clasica: True
XOR   coincide con la tabla de verdad clasica: True
XNOR  coincide con la tabla de verdad clasica: True
```

Es determinista. Los ángulos dependen de la unitaria aleatoria elegida, que usa semilla fija.

16.7. Interpretación

Las dos descomposiciones se verifican (`True`). En la Toffoli, la cadena se lee como $q_2q_1q_0$: solo cambian $011 \to 111$ y $111 \to 011$, es decir, el objetivo q_2 se invierte cuando $q_0 = q_1 = 1$. Las seis puertas lógicas coinciden con su tabla de verdad clásica.

16.8. Ejercicios

1. Cambia la unitaria aleatoria (otra `seed`) y verifica de nuevo las dos descomposiciones.
2. Construye una puerta lógica con tres entradas, por ejemplo $C = A \wedge B \wedge D$ (necesita una puerta con tres controles).
3. ¿Por qué el circuito OR necesita que el objetivo empiece en $|1\rangle$?

16.9. Errores comunes

- Olvidar el orden de bits de Qiskit: la cadena `'011'` es $q_2 = 0$, $q_1 = 1$, $q_0 = 1$.
- Pensar que la puerta lógica clásica se aplica con una sola entrada: en la versión reversible las entradas se conservan (en A y B) y la salida se escribe en un qubit aparte.

17. Algoritmo de Deutsch

17.1. Objetivo

Resolver con una sola consulta un problema que clásicamente necesita dos: decidir si una función de un bit es constante o balanceada.

17.2. Conceptos previos

Concepto previo: superposición, puerta H e interferencia

Ver capítulos 01, 02 y 05a. La puerta H convierte $|0\rangle$ en $(|0\rangle + |1\rangle)/\sqrt{2}$ y $|1\rangle$ en $(|0\rangle - |1\rangle)/\sqrt{2}$, y dos H seguidas se cancelan.

17.3. Fundamento teórico

El algoritmo de Deutsch, propuesto por David Deutsch en 1985, se considera el primer algoritmo que demuestra la ventaja de un computador cuántico sobre uno clásico.

El problema (las cajas de pintura). Hay dos bolas, una liviana ($x = 0$) y una pesada ($x = 1$), que se pintan de blanco ($f = 0$) o negro ($f = 1$). Hay cajas de un solo color (pintan las dos bolas igual) y cajas de dos colores (las pintan distinto), pero se mezclaron. ¿Cuántas veces hay que pintar para distinguirlas?

En términos matemáticos, la caja es una función $f : \{0, 1\} \to \{0, 1\}$:

- **constante** si $f(0) = f(1)$ (caja de un solo color);
- **balanceada** si $f(0) \neq f(1)$ (caja de dos colores).

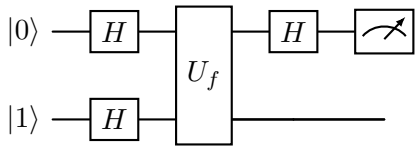
Clásicamente hay que evaluar $f(0)$ y $f(1)$ y compararlos: **dos consultas**.

El oráculo. La función se da como una puerta unitaria U_f que actúa como

$$U_f \,|x\rangle \,|y\rangle = |x\rangle \,|y \oplus f(x)\rangle \,, \tag{26}$$

donde $\oplus$ es la suma módulo 2 (XOR). El primer qubit es el *registro de datos* y el segundo el *registro objetivo*. Gracias a la superposición, una sola aplicación de U_f evalúa $f(0)$ y $f(1)$ a la vez (*paralelismo cuántico*).

El circuito.



1. Estado inicial $|\psi_0\rangle = |0\rangle \,|1\rangle$.
2. Con H en ambos qubits: $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \, \frac{|0\rangle-|1\rangle}{\sqrt{2}}$.
3. Una consulta a U_f. Como el objetivo está en $(|0\rangle - |1\rangle)/\sqrt{2}$, el oráculo solo cambia el signo de cada $|x\rangle$:

$$U_f \,|x\rangle \, \frac{|0\rangle - |1\rangle}{\sqrt{2}} = (-1)^{f(x)} \,|x\rangle \, \frac{|0\rangle - |1\rangle}{\sqrt{2}}. \tag{27}$$

   Por tanto $|\psi_2\rangle = \frac{(-1)^{f(0)}|0\rangle+(-1)^{f(1)}|1\rangle}{\sqrt{2}} \, \frac{|0\rangle-|1\rangle}{\sqrt{2}}$.

- Si f es constante, los datos quedan en $\pm\frac{|0\rangle+|1\rangle}{\sqrt{2}}$.
- Si f es balanceada, quedan en $\pm\frac{|0\rangle-|1\rangle}{\sqrt{2}}$.

4. H sobre los datos: $|\psi_3\rangle = \pm|0\rangle$ si f es constante y $\pm|1\rangle$ si es balanceada.
5. Se mide el registro de datos: 0 significa constante y 1 significa balanceada.

Una consulta contra dos.

17.4. Código (completamente comentado)

```python
# %% [markdown]
# # 15 - Algoritmo de Deutsch: una consulta en vez de dos

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector

sim = AerSimulator()

# %% Celda 2: el problema (las cajas de pintura)
# Una funcion f(x) con x en {0, 1} es una "caja negra". Hay cuatro posibles:
#   constante: f(0) = f(1)  (siempre 0, o siempre 1)    -> caja de un solo color
#   balanceada: f(0) != f(1)  (identidad o negacion)    -> caja de dos colores
# Clasicamente hay que preguntar dos veces (f(0) y f(1)) para saberlo.
funciones = {
    "constante 0": (0, 0),
    "constante 1": (1, 1),
    "balanceada f(x)=x": (0, 1),
    "balanceada f(x)=1-x": (1, 0),
}
for nombre, (f0, f1) in funciones.items():
    tipo = "constante" if f0 == f1 else "balanceada"
    print(f"{nombre:22s}: f(0) = {f0}, f(1) = {f1} -> {tipo}")


# %% Celda 3: el oraculo Uf
# El oraculo es una puerta unitaria que actua asi:  Uf |x>|y> = |x>|y xor f(x)>.
# q0 es el registro de datos (x) y q1 el registro objetivo (y).
# Se construye aplicando una X al objetivo cuando f(x) = 1:
def oraculo(f0, f1):
    qc = QuantumCircuit(2)
    if (f0, f1) == (1, 1):
        qc.x(1)                         # f(x) = 1 siempre: se invierte siempre
    elif (f0, f1) == (0, 1):
        qc.cx(0, 1)                     # f(x) = x: se invierte si x = 1
    elif (f0, f1) == (1, 0):
        qc.cx(0, 1)                     # f(x) = 1 - x: se invierte si x = 0
        qc.x(1)
    return qc                           # (0, 0): no se hace nada


# %% Celda 4: el circuito de Deutsch
#   1. Datos en |0> y objetivo en |1>.
#   2. H en los dos: datos en (|0>+|1>)/sqrt2 y objetivo en (|0>-|1>)/sqrt2.
#   3. Una sola consulta al oraculo Uf.
#   4. H en el registro de datos y se mide.
# Si sale 0, f es constante; si sale 1, f es balanceada.
def deutsch(f0, f1):
    qc = QuantumCircuit(2, 1)
```

```
    qc.x(1)                    # objetivo en |1>
    qc.h([0, 1])
    qc.compose(oraculo(f0, f1), inplace=True)
    qc.h(0)
    qc.measure(0, 0)
    return qc


# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
deutsch(0, 1).draw()

# %% Celda 5: ejecutar con las cuatro funciones
# Una sola consulta basta, y el resultado es siempre el mismo (no es aleatorio).
for nombre, (f0, f1) in funciones.items():
    cuentas = sim.run(deutsch(f0, f1), shots=1000).result().get_counts()
    dato = max(cuentas, key=cuentas.get)
    veredicto = "constante" if dato == "0" else "balanceada"
    print(f"{nombre:22s}: medicion = {cuentas} -> {veredicto}")

# %% Celda 6: por que funciona (cambio de signo por la fase)
# Con el objetivo en (|0>-|1>)/sqrt2, el oraculo solo cambia el signo de cada x:
#   Uf |x> (|0>-|1>)/sqrt2 = (-1)^f(x) |x> (|0>-|1>)/sqrt2.
# El registro de datos queda en ((-1)^f(0)|0> + (-1)^f(1)|1>)/sqrt2:
#   constante  -> (|0>+|1>)/sqrt2  (o su opuesto)  -> la H final lo lleva a |0>
#   balanceada -> (|0>-|1>)/sqrt2  (o su opuesto)  -> la H final lo lleva a |1>
for nombre, (f0, f1) in funciones.items():
    qc = QuantumCircuit(2)
    qc.x(1)
    qc.h([0, 1])
    qc.compose(oraculo(f0, f1), inplace=True)
    amplitudes = Statevector(qc).data
    # El objetivo (q1) vale (|0>-|1>)/sqrt2, asi que las amplitudes con q1 = 0 (indices
    # 0 y 1) son las de los datos multiplicadas por 1/sqrt2. Multiplicando por 2 queda
    # +-1, que son los signos (-1)^f(x) de la formula.
    a0, a1 = amplitudes[0] * 2, amplitudes[1] * 2
    print(f"{nombre:22s}: datos = ({a0.real:+.0f}|0> {a1.real:+.0f}|1>)/sqrt2")
```

17.5. Explicación celda por celda

Celda 2 Lista las cuatro funciones posibles y dice si cada una es constante o balanceada.

Celda 3 Construye el oráculo U_f: una X al objetivo si f vale 1 siempre, una CNOT si $f(x) = x$, y CNOT más X si $f(x) = 1 - x$.

Celda 4 Arma el circuito: objetivo en $|1\rangle$, H en ambos, oráculo, H en los datos y medición.

Celda 5 Lo ejecuta con las cuatro funciones y 1000 mediciones.

Celda 6 Muestra los signos $(-1)^{f(x)}$ del registro de datos después del oráculo.

17.6. Salida obtenida

```
constante 0            : f(0) = 0, f(1) = 0 -> constante
constante 1            : f(0) = 1, f(1) = 1 -> constante
balanceada f(x)=x      : f(0) = 0, f(1) = 1 -> balanceada
balanceada f(x)=1-x    : f(0) = 1, f(1) = 0 -> balanceada
constante 0            : medicion = {'0': 1000} -> constante
constante 1            : medicion = {'0': 1000} -> constante
balanceada f(x)=x      : medicion = {'1': 1000} -> balanceada
balanceada f(x)=1-x    : medicion = {'1': 1000} -> balanceada
constante 0            : datos = (+1|0> +1|1>)/sqrt2
```

```
constante 1           : datos = (-1|0> -1|1>)/sqrt2
balanceada f(x)=x     : datos = (+1|0> -1|1>)/sqrt2
balanceada f(x)=1-x   : datos = (-1|0> +1|1>)/sqrt2
```

(El dibujo del circuito no se reproduce aquí.)

17.7. Interpretación

El resultado no es aleatorio: las dos funciones constantes siempre dan 0 y las dos balanceadas siempre dan 1, con una sola consulta. Los signos de la última parte coinciden con $(-1)^{f(x)}$: $(+1, +1)$ y $(-1, -1)$ para las constantes, $(+1, -1)$ y $(-1, +1)$ para las balanceadas. Lo que distingue los casos es la *fase relativa* entre $|0\rangle$ y $|1\rangle$, que la H final convierte en una diferencia medible (interferencia, capítulo 05a).

17.8. Ejercicios

1. Elimina la X inicial del objetivo y mira qué cambia en la Celda 6.
2. ¿Cuántas consultas necesitaría un algoritmo clásico para $n = 1$? ¿Y el cuántico?
3. Cambia el oráculo de $f(x) = x$ por otro circuito equivalente y comprueba que el resultado no cambia.

17.9. Errores comunes

- Creer que el algoritmo conoce $f(0)$ y $f(1)$ por separado: solo averigua si son iguales o distintos.
- Olvidar poner el objetivo en $|1\rangle$ antes de la H: sin esa fase el oráculo no cambia el signo.

18. Algoritmo de Bernstein–Vazirani

18.1. Objetivo

Descubrir un número binario oculto de n bits con una sola consulta, en vez de las n consultas que necesita un algoritmo clásico, y ver que la ventaja viene de la superposición y no del entrelazamiento.

18.2. Conceptos previos

Concepto previo: puertas H sobre varios qubits y producto punto módulo 2

Ver capítulos 05a y 15. El *producto punto* de dos cadenas de bits es $a \cdot x = a_1x_1 + a_2x_2 + \cdots + a_nx_n$, y "módulo 2" significa quedarse con el resto de dividir entre 2 (la paridad).

18.3. Fundamento teórico

El algoritmo fue propuesto por Bernstein y Vazirani en 1993. Muestra la ventaja cuántica en términos de número de consultas.

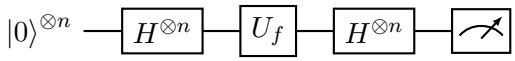
El problema. Una base de datos guarda un número oculto $a \in \{0,1\}^n$. Se puede hacer una consulta eligiendo un $x \in \{0,1\}^n$, y la respuesta es

$$f(x) = a \cdot x \pmod 2 = a_1x_1 \oplus a_2x_2 \oplus \cdots \oplus a_nx_n, \tag{28}$$

un solo bit (0 o 1). El objetivo es hallar a.

Solución clásica. Se consulta con $x = 100\ldots0,\ 010\ldots0,\ \ldots,\ 000\ldots1$: la consulta i da el bit a_i. Hacen falta n **consultas**. Por ejemplo, para $a = 1011$ se necesitan cuatro.

Solución cuántica. Se prepara una superposición uniforme de n qubits y se usa como entrada de la función, de modo que todas las consultas se hacen en paralelo. El circuito de este ejemplo es una variante de Phys. Rev. A **64**, 042306, y no usa qubits auxiliares:



con el oráculo $U_f\,|x\rangle = (-1)^{a\cdot x}\,|x\rangle$.

Por qué funciona. Una H sobre un qubit da $H\,|a\rangle = \frac{1}{\sqrt{2}}\sum_{x\in\{0,1\}}(-1)^{ax}\,|x\rangle$. Para n qubits:

$$H^{\otimes n}\,|a\rangle = \frac{1}{\sqrt{2^n}}\sum_{x\in\{0,1\}^n}(-1)^{a\cdot x}\,|x\rangle\,, \qquad H^{\otimes n}\,|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}}\sum_{x}|x\rangle\,. \tag{29}$$

Es decir, U_f convierte la superposición uniforme en $H^{\otimes n}\,|a\rangle$. Como $H^{\otimes n}H^{\otimes n} = I$, la H final lo lleva de vuelta a $|a\rangle$:

$$|a\rangle = H^{\otimes n}\,U_f\,H^{\otimes n}\,|0\rangle^{\otimes n}\,. \tag{30}$$

Al medir los n qubits se lee directamente a. **Una sola consulta**, es decir, complejidad $O(1)$ frente a $O(n)$ clásica.

Sin entrelazamiento. El oráculo se arma como un producto de n puertas de un solo qubit: $U_i = I$ si $a_i = 0$ y $U_i = Z$ si $a_i = 1$. Todas las puertas del circuito son de un qubit y el estado inicial no está entrelazado, así que el estado nunca lo está. La aceleración viene de la **superposición**, no del entrelazamiento.

18.4. Código (completamente comentado)

```python
# %% [markdown]
# # 16 - Algoritmo de Bernstein-Vazirani: adivinar un numero oculto con una consulta

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector, entropy, partial_trace

sim = AerSimulator()

# %% Celda 2: el problema
# Hay un numero oculto de n bits, a = a1 a2 ... an. Solo se puede hacer una pregunta:
# se elige un x de n bits y se recibe f(x) = a . x (mod 2), es decir, la suma (mod 2)
# de los bits donde a y x valen 1. La respuesta es un solo bit (0 o 1).
a_secreto = "1011"          # lo "desconoce" el algoritmo; a1 es el primer bit de la izquierda
n = len(a_secreto)
a = [int(b) for b in a_secreto]


def f(x):
    """La caja negra: devuelve a . x (mod 2)."""
    return sum(ai * xi for ai, xi in zip(a, x)) % 2


# %% Celda 3: solucion clasica, n consultas
# Se pregunta con x = 1000, 0100, 0010, 0001: la respuesta i-esima da el bit a_i.
encontrado = ""
for i in range(n):
    x = [1 if j == i else 0 for j in range(n)]
    encontrado += str(f(x))
    print(f"consulta {i + 1}: x = {''.join(map(str, x))} -> f(x) = {f(x)}")
print("Numero encontrado:", encontrado, f"({n} consultas)")

# %% Celda 4: el oraculo cuantico
# En la version de este ejemplo el oraculo actua asi:  Uf |x> = (-1)^(a . x) |x>.
# Se arma como un producto de puertas de UN solo qubit: una Z en cada qubit donde a_i = 1
# y nada donde a_i = 0. (Z cambia el signo de |1>.)
# Qiskit numera el qubit 0 a la DERECHA al escribir cadenas; aqui el qubit i-1 guarda a_i.
def oraculo(a_bits):
    qc = QuantumCircuit(len(a_bits))
    for i, ai in enumerate(a_bits):
        if ai == 1:
            qc.z(i)
    return qc


# %% Celda 5: el circuito de Bernstein-Vazirani
#   |0...0> -> H en todos -> Uf (una sola consulta) -> H en todos -> medir.
# La H final deshace la superposicion y deja en los qubits justo el numero a.
def bernstein_vazirani(a_bits):
    n = len(a_bits)
    qc = QuantumCircuit(n, n)
    qc.h(range(n))
    qc.compose(oraculo(a_bits), inplace=True)
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc


# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
```

```
bernstein_vazirani(a).draw()

# %% Celda 6: ejecutar, con una sola consulta
cuentas = sim.run(bernstein_vazirani(a), shots=1000).result().get_counts()
# Qiskit escribe el qubit 0 a la derecha, asi que se invierte la cadena para leer a1...an.
medido = max(cuentas, key=cuentas.get)[::-1]
print("Mediciones:", cuentas)
print("Numero oculto:", a_secreto, "| medido:", medido, "| 1 consulta en vez de", n)

# %% Celda 7: sin entrelazamiento
# Todas las puertas son de un solo qubit y el estado inicial no esta entrelazado, asi
# que el estado nunca lo esta: la ventaja viene de la superposicion, no del
# entrelazamiento. Se comprueba con la entropia de un qubit (0 = no entrelazado).
sin_medir = QuantumCircuit(n)
sin_medir.h(range(n))
sin_medir.compose(oraculo(a), inplace=True)
sin_medir.h(range(n))
sv = Statevector(sin_medir)
for q in range(n):
    resto = [i for i in range(n) if i != q]
    print(f"entropia del qubit {q}: {float(entropy(partial_trace(sv, resto), base=2)):.3f}")
```

18.5. Explicación celda por celda

Celda 2 Define el número oculto $a = 1011$ y la caja negra $f(x) = a \cdot x$.

Celda 3 Solución clásica: cuatro consultas con $x = 1000, 0100, 0010, 0001$.

Celda 4 El oráculo: una Z en cada qubit donde $a_i = 1$.

Celda 5 El circuito $H^{\otimes n}$, U_f, $H^{\otimes n}$ y medición.

Celda 6 Lo ejecuta (1000 mediciones) e invierte la cadena, porque Qiskit escribe el qubit 0 a la derecha.

Celda 7 Calcula la entropía de cada qubit antes de medir para comprobar que no hay entrelazamiento.

18.6. Salida obtenida

```
consulta 1: x = 1000 -> f(x) = 1
consulta 2: x = 0100 -> f(x) = 0
consulta 3: x = 0010 -> f(x) = 1
consulta 4: x = 0001 -> f(x) = 1
Numero encontrado: 1011 (4 consultas)
Mediciones: {'1101': 1000}
Numero oculto: 1011 | medido: 1011 | 1 consulta en vez de 4
entropia del qubit 0: 0.000
entropia del qubit 1: 0.000
entropia del qubit 2: 0.000
entropia del qubit 3: 0.000
```

(El dibujo del circuito no se reproduce aquí.)

18.7. Interpretación

Las cuatro consultas clásicas dan $1, 0, 1, 1$, es decir, $a = 1011$. El circuito cuántico, con una sola consulta, mide siempre `1101` en 1000 mediciones; al invertir la cadena se lee `1011`, el número oculto. La entropía de cada qubit es 0: el estado nunca está entrelazado.

18.8. Ejercicios

1. Prueba con otro número oculto, por ejemplo $a = 0110$ y $a = 11111$.
2. ¿Qué pasa si $a = 0000$? Predícelo y compruébalo.
3. Cuenta cuántas consultas necesita el algoritmo clásico para $n = 20$ y cuántas el cuántico.

18.9. Errores comunes

- Olvidar que Qiskit imprime el qubit 0 a la derecha: la cadena medida está al revés respecto de $a_1 a_2 \dots a_n$.
- Confundir el número de *consultas* con el tiempo total: el algoritmo cuenta cuántas veces se usa la caja negra.

19. Transformada de Fourier cuántica y estimación de fase

19.1. Objetivo

Construir la transformada de Fourier cuántica (QFT) con un circuito y usarla para estimar la fase de un autovalor (estimación de fase, QPE).

19.2. Conceptos previos

Concepto previo: fases, puertas controladas y números binarios

Ver capítulos 05a (fases e interferencia) y 14 (puertas controladas). Una fase se puede escribir como una fracción binaria: $0.\varphi_1\varphi_2\ldots\varphi_m = \varphi_1/2 + \varphi_2/4 + \cdots + \varphi_m/2^m$. Por ejemplo $0.75 = 0.11_2$ y $1/4 = 0.01_2$.

19.3. Fundamento teórico

QFT. La QFT es un componente central de muchos algoritmos (Shor y la estimación de fase, entre ellos). Es la versión cuántica de la transformada de Fourier discreta (DFT). Con $N = 2^n$ estados de la base, lleva

$$|j\rangle \longmapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i\, jk/N} |k\rangle\,, \tag{31}$$

una matriz unitaria con entradas $U[k,j] = e^{2\pi ijk/N}/\sqrt{N}$. Aunque la fórmula se parece a la de la DFT, el significado es distinto: la DFT transforma una función discreta en el dominio del tiempo y la QFT transforma un estado cuántico en otro.

El circuito. Escribiendo $j = j_1 j_2 \ldots j_n$ en binario, la QFT se puede reescribir como un producto de qubits:

$$|j\rangle \mapsto \frac{1}{2^{n/2}}\big(\,|0\rangle + e^{2\pi i\, 0.j_n}\,|1\rangle\,\big)\big(\,|0\rangle + e^{2\pi i\, 0.j_{n-1}j_n}\,|1\rangle\,\big)\cdots\big(\,|0\rangle + e^{2\pi i\, 0.j_1 j_2 \ldots j_n}\,|1\rangle\,\big). \tag{32}$$

Esto da un circuito con tres tipos de pieza:

1. una Hadamard en cada qubit;
2. fases controladas CR_k, que aplican la fase $e^{2\pi i/2^k}$ a $|1\rangle$ (CR_2 es la puerta S controlada) y trasladan la información de los demás bits a la fase;
3. al final, puertas SWAP que invierten el orden de los qubits.

El número de puertas crece como $O(n^2)$, es decir, $O((\log N)^2)$ para N estados.

QPE. Dada una unitaria U y un autovector $|\psi\rangle$ con $U|\psi\rangle = e^{2\pi i\varphi}|\psi\rangle$, la estimación de fase calcula φ. La QFT pasa información del estado a la fase; la QPE pasa información de la fase al estado. Usa dos registros: el primero con m qubits (da una precisión de m bits) y el segundo, inicializado en el autovector $|\psi\rangle$. Pasos:

1. Primer registro en $|0\rangle^{\otimes m}$ y segundo en $|\psi\rangle$.
2. Hadamard en cada qubit del primer registro.
3. Puertas U^{2^k} controladas por el qubit k del primer registro. Cada una deja una fase $e^{2\pi i\varphi 2^k}$ en $|1\rangle$.

4. QFT inversa sobre el primer registro.

5. Se mide el primer registro: sale el número binario $\varphi_1\varphi_2\ldots\varphi_m$.

Por ejemplo, la puerta S tiene autovector $|1\rangle$ con autovalor $i = e^{2\pi i\cdot 1/4}$, así que $\varphi = 1/4 = 0.01_2$.

19.4. Código (completamente comentado)

```python
# %% [markdown]
# # 17 - Transformada de Fourier cuantica (QFT) y estimacion de fase (QPE)

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Operator, Statevector

sim = AerSimulator()

# %% Celda 2: la QFT como matriz
# La QFT lleva cada estado de la base |j> al estado
#     |j>  ->  (1/sqrt(N)) * suma_k  e^{2 pi i j k / N} |k>,     N = 2^n.
# Es la version cuantica de la transformada de Fourier discreta (DFT) y es una
# matriz unitaria con entradas  U[k, j] = e^{2 pi i j k / N} / sqrt(N).
def matriz_qft(n):
    N = 2 ** n
    j, k = np.meshgrid(np.arange(N), np.arange(N))
    return np.exp(2j * np.pi * j * k / N) / np.sqrt(N)


U = matriz_qft(3)
print("La QFT es unitaria (U^dagger U = I):", np.allclose(U.conj().T @ U, np.eye(8)))


# %% Celda 3: el circuito de la QFT
# Para cada qubit, de arriba hacia abajo:
#   1. Una Hadamard.
#   2. Fases controladas CR_k por cada qubit siguiente, que pasan la informacion
#      de los demas bits a la fase de |1>. CR_k aplica la fase e^{2 pi i / 2^k}
#      (CR_2 es la puerta S controlada).
# Al final una serie de SWAP invierte el orden de los qubits.
# Qiskit numera el qubit 0 a la derecha, asi que el bit mas significativo es el
# qubit n-1 y se empieza por el.
def circuito_qft(n):
    qc = QuantumCircuit(n)
    for objetivo in reversed(range(n)):
        qc.h(objetivo)
        for control in range(objetivo - 1, -1, -1):
            k = objetivo - control + 1
            qc.cp(2 * np.pi / 2 ** k, control, objetivo)
    for i in range(n // 2):
        qc.swap(i, n - 1 - i)
    return qc


for n in (2, 3, 4):
    coincide = np.allclose(Operator(circuito_qft(n)).data, matriz_qft(n))
    puertas = sum(circuito_qft(n).count_ops().values())
    print(f"n = {n}: el circuito coincide con la matriz de la QFT: {coincide} ({puertas} puertas
    ↪ )")

# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
circuito_qft(3).draw()
```

```
55
56  # %% Celda 4: la QFT de dos qubits sobre cada estado de la base
57  # Cada entrada |j> da cuatro amplitudes iguales en modulo (1/2) y lo que cambia es la
58  # fase: e^{2 pi i j k / 4} para cada k, es decir, multiplos de 90 grados.
59  qft2 = circuito_qft(2)
60  print("Fases (en grados) de las amplitudes de |k> para k = 00, 01, 10, 11:")
61  for j in range(4):
62      entrada = format(j, "02b")
63      amplitudes = Statevector.from_label(entrada).evolve(qft2).data
64      # Los indices de Statevector crecen con el valor binario de la cadena (q1 q0).
65      fases = [round(float(np.degrees(np.angle(c))) % 360) for c in amplitudes]
66      print(f"  |{entrada}> -> {fases}")
67
68  # %% Celda 5: estimacion de fase (QPE)
69  # Sea U una puerta unitaria y |psi> un autovector con U|psi> = e^{2 pi i phi}|psi>.
70  # QPE estima phi con m bits: phi = 0.phi1 phi2 ... phim (en binario).
71  #   1. Primer registro (m qubits) en |0>, segundo registro en el autovector |psi>.
72  #   2. Hadamard en cada qubit del primer registro.
73  #   3. Puertas U^(2^k) controladas por el qubit k del primer registro.
74  #   4. QFT inversa sobre el primer registro.
75  #   5. Se mide el primer registro: sale el numero binario phi1 ... phim.
76  # La QFT pasa informacion de la base a la fase; la QPE hace lo contrario.
77  def qpe(phi, m):
78      qc = QuantumCircuit(m + 1, m)
79      qc.x(m)                       # el qubit m es el segundo registro: autovector |1> de la
    ↪ puerta fase
80      qc.h(range(m))
81      for k in range(m):
82          # La puerta de fase P(2 pi phi) tiene autovalor e^{2 pi i phi} en |1>.
83          # Controlada por el qubit k y elevada a 2^k: fase 2 pi phi 2^k.
84          qc.cp(2 * np.pi * phi * 2 ** k, k, m)
85      qc.compose(circuito_qft(m).inverse(), qubits=range(m), inplace=True)
86      qc.measure(range(m), range(m))
87      return qc
88
89
90  def estimar(phi, m, shots=1000):
91      cuentas = sim.run(qpe(phi, m), shots=shots).result().get_counts()
92      mejor = max(cuentas, key=cuentas.get)
93      return mejor, int(mejor, 2) / 2 ** m, cuentas[mejor] / shots
94
95
96  # %% Celda 6: fases que se pueden escribir exactamente con m bits
97  # La puerta S tiene autovector |1> con autovalor i = e^{2 pi i (1/4)}, o sea phi = 1/4.
98  # En binario 1/4 = 0.01, y con m = 2 la QPE deberia dar 01.
99  # La puerta T tiene phi = 1/8 = 0.001: hacen falta m = 3.
100 for nombre, phi, m in (("S", 1 / 4, 2), ("T", 1 / 8, 3), ("Z", 1 / 2, 2)):
101     bits, estimada, prob = estimar(phi, m)
102     print(f"puerta {nombre}: phi = {phi:.3f} | QPE con m = {m}: {bits} -> {estimada:.3f} (
    ↪ probabilidad {prob:.2f})")
103
104 # %% Celda 7: fases que NO caben exactamente
105 # phi = 1/3 no se puede escribir con pocos bits. La QPE da el valor mas cercano con
106 # alta probabilidad, y mas bits dan mas precision.
107 phi = 1 / 3
108 for m in (2, 4, 6):
109     bits, estimada, prob = estimar(phi, m, shots=2000)
110     print(f"m = {m}: resultado mas probable {bits} -> {estimada:.4f} | error = {abs(estimada -
    ↪ phi):.4f} | probabilidad {prob:.2f}")
```

19.5. Explicación celda por celda

Celda 2 Construye la matriz de la QFT y comprueba que es unitaria.

Celda 3 Construye el circuito (Hadamard, fases controladas y SWAP) y comprueba que su matriz es igual a la de la QFT para $n = 2, 3, 4$.

Celda 4 Aplica la QFT de dos qubits a cada estado de la base y muestra las fases de las cuatro amplitudes.

Celda 5 Define la QPE: Hadamard, fases controladas con $2\pi\varphi 2^k$, QFT inversa y medición.

Celda 6 Prueba fases exactas: puertas S (1/4), T (1/8) y Z (1/2).

Celda 7 Prueba $\varphi = 1/3$, que no se escribe exactamente con pocos bits, con $m = 2, 4, 6$.

19.6. Salida obtenida

```
La QFT es unitaria (U^dagger U = I): True
n = 2: el circuito coincide con la matriz de la QFT: True (4 puertas)
n = 3: el circuito coincide con la matriz de la QFT: True (7 puertas)
n = 4: el circuito coincide con la matriz de la QFT: True (12 puertas)
Fases (en grados) de las amplitudes de |k> para k = 00, 01, 10, 11:
  |00> -> [0, 0, 0, 0]
  |01> -> [0, 90, 180, 270]
  |10> -> [0, 180, 0, 180]
  |11> -> [0, 270, 180, 90]
puerta S: phi = 0.250 | QPE con m = 2: 01 -> 0.250 (probabilidad 1.00)
puerta T: phi = 0.125 | QPE con m = 3: 001 -> 0.125 (probabilidad 1.00)
puerta Z: phi = 0.500 | QPE con m = 2: 10 -> 0.500 (probabilidad 1.00)
m = 2: resultado mas probable 01 -> 0.2500 | error = 0.0833 | probabilidad 0.70
m = 4: resultado mas probable 0101 -> 0.3125 | error = 0.0208 | probabilidad 0.69
m = 6: resultado mas probable 010101 -> 0.3281 | error = 0.0052 | probabilidad 0.70
```

(El dibujo del circuito no se reproduce aquí.) Las cuentas de la QPE dependen del azar de las mediciones; el resultado más probable es estable.

19.7. Interpretación

El circuito coincide con la matriz de la QFT. Con 2 qubits, cada entrada da cuatro amplitudes del mismo módulo y fases que son múltiplos de 90°; por ejemplo $|01\rangle \to [0, 90, 180, 270]$. Con las fases exactas, la QPE da el resultado correcto con probabilidad 1: $S \to 01$ (0.25), $T \to 001$ (0.125) y $Z \to 10$ (0.5). Para $\varphi = 1/3$, el resultado más probable se acerca cada vez más: el error baja de 0.083 ($m = 2$) a 0.021 ($m = 4$) y a 0.005 ($m = 6$), con probabilidad cercana a 0.69.

19.8. Ejercicios

1. Comprueba la QPE con $\varphi = 3/8$ y $m = 3$. ¿Qué cadena esperas?
2. ¿Qué cadena da la QPE para la puerta Z con $m = 3$ bits? Predícelo y compruébalo.
3. Cuenta las puertas del circuito de la QFT para $n = 5$ y compara con n^2.

19.9. Errores comunes

- Olvidar los SWAP finales: sin ellos el circuito da la QFT con los qubits en orden inverso.
- Esperar un resultado exacto cuando la fase no cabe en m bits: la QPE da el valor más cercano, con alta probabilidad pero no exacta.

20. Algoritmo HHL: resolver un sistema lineal con dos qubits

20.1. Objetivo

Entender cómo el algoritmo HHL resuelve $A\mathbf{x} = \mathbf{b}$ dividiendo los coeficientes de $\mathbf{b}$ entre los autovalores de A, y construir su versión de dos qubits para una matriz 2×2.

20.2. Conceptos previos

Concepto previo: autovalores, autovectores y estimación de fase

Ver capítulo 17 (estimación de fase) y la sección 1. Una matriz hermítica A se escribe como $A = \sum_i \lambda_i |u_i\rangle \langle u_i|$, con autovalores λ_i y autovectores $|u_i\rangle$.

20.3. Fundamento teórico

HHL (Harrow, Hassidim y Lloyd, 2009) resuelve sistemas de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$, con A una matriz hermítica $N \times N$. Clásicamente, con métodos directos o iterativos, el tiempo es $O(N^3)$; en condiciones ideales, HHL lo reduce a $O(\log N)$. Se usa como subrutina en modelos de aprendizaje automático cuántico.

Un ejemplo clásico. Dos grupos hacen vasijas. En 2.5 horas, el grupo A hace una vasija más que el grupo B en 0.5 horas; y en 0.5 horas, A hace tantas como B en 2.5 horas. Si x e y son las vasijas por hora:

$$\begin{pmatrix} 2.5 & -0.5 \\ -0.5 & 2.5 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad x = \tfrac{5}{12},\ y = \tfrac{1}{12}. \tag{33}$$

La idea de HHL. Los vectores se codifican como estados cuánticos normalizados, $|b\rangle$ y $|x\rangle$, y resolver $\mathbf{x} = A^{-1}\mathbf{b}$ pasa a ser $|x\rangle \propto A^{-1}|b\rangle$. Se escribe $|b\rangle$ en la base de autovectores, $|b\rangle = \sum_i \beta_i |u_i\rangle$ con $\beta_i = \langle u_i|b\rangle$, y entonces

$$|x\rangle \ \propto \ \sum_i \frac{\beta_i}{\lambda_i} |u_i\rangle \,. \tag{34}$$

Basta dividir cada coeficiente entre su autovalor. Para la matriz del ejemplo, los autovalores son $\lambda_1 = 2$ y $\lambda_2 = 3$, con autovectores $|u_1\rangle = \frac{1}{\sqrt{2}}\binom{1}{1}$ y $|u_2\rangle = \frac{1}{\sqrt{2}}\binom{1}{-1}$.

Los tres componentes. HHL usa tres grupos de qubits: de codificación (guardan $|b\rangle$, $n_b = \log_2 N$ qubits), de registro (guardan los autovalores estimados) y un qubit auxiliar. Los pasos son:

1. **Estimación de fase:** estima los autovalores de A y los guarda en el registro: $\sum_i \beta_i \left|\tilde{\lambda}_i\right\rangle |u_i\rangle$.
2. **Rotación controlada:** usando el registro como control, gira el qubit auxiliar un ángulo $\theta_i = 2\sin^{-1}(c/\lambda_i)$. Así el recíproco del autovalor pasa a la amplitud del auxiliar: $\sum_i \beta_i |\lambda_i\rangle |u_i\rangle \left(\sqrt{1 - c^2/\lambda_i^2}\,|0\rangle + \frac{c}{\lambda_i}|1\rangle\right)$. El parámetro c cumple $c \le \min|\lambda_i|$.
3. **Estimación de fase inversa:** desentrelaza el registro.
4. **Medición:** si el auxiliar vale $|1\rangle$ (con cierta probabilidad), el qubit de codificación queda en $|x\rangle \propto \sum_i \frac{\beta_i}{\lambda_i}|u_i\rangle$.

Versión de dos qubits. Con una matriz 2×2 de autovalores 2 y 3, el registro necesita un solo qubit y el circuito se simplifica hasta quedar en dos qubits: un qubit de codificación y uno auxiliar. Si $A = U\,\mathrm{diag}(\lambda_1, \lambda_2)\,U^\dagger$, el circuito es: $U^\dagger$ sobre el qubit de codificación (los autovectores pasan a $|0\rangle$ y $|1\rangle$), una rotación $R_y(\theta_i)$ del auxiliar controlada por el qubit de codificación y, al final, U. Con $\lambda_1 = 2$ y $\lambda_2 = 3$, el mayor valor posible de c es 2.

20.4. Código (completamente comentado)

```
# %% [markdown]
# # 18 - Algoritmo HHL: resolver un sistema lineal Ax = b con dos qubits

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

# %% Celda 2: el problema clasico
# Dos grupos hacen vasijas: 2.5 x - 0.5 y = 1  y  -0.5 x + 2.5 y = 0.
# En forma matricial es  A x = b  con
A = np.array([[2.5, -0.5], [-0.5, 2.5]])
b = np.array([1.0, 0.0])
x_clasico = np.linalg.solve(A, b)
print("Solucion clasica x = A^-1 b =", x_clasico.round(4), "(5/12, 1/12)")

# %% Celda 3: la idea de HHL, con autovalores y autovectores
# A se escribe con sus autovalores l_i y autovectores |u_i>:  A = suma l_i |u_i><u_i|.
# El vector b se escribe en esa base:  |b> = suma beta_i |u_i>,  beta_i = <u_i|b>.
# Entonces la solucion es  |x> proporcional a  suma (beta_i / l_i) |u_i>:
# basta dividir cada coeficiente entre su autovalor.
autovalores, U = np.linalg.eigh(A)      # las columnas de U son los autovectores
b_norm = b / np.linalg.norm(b)
beta = U.T @ b_norm
x_por_autovalores = U @ (beta / autovalores)
print("Autovalores:", autovalores)
print("beta_i = <u_i|b>:", beta.round(4))
print("Solucion con autovalores (normalizada):", (x_por_autovalores / np.linalg.norm(
    ↪ x_por_autovalores)).round(4))
print("Solucion clasica       (normalizada):", (x_clasico / np.linalg.norm(x_clasico)).round(4))


# %% Celda 4: el circuito de HHL de dos qubits
# HHL tiene tres partes: estimacion de fase (para hallar los autovalores), una
# rotacion controlada (para dividir entre ellos) y la estimacion de fase inversa.
# Para una matriz 2x2 con autovalores conocidos, el circuito se simplifica a dos
# qubits (como en el PDF): q0 = qubit de codificacion, q1 = qubit auxiliar.
#   1. Codificar |b> en q0.
#   2. U^dagger: lleva los autovectores |u_i> a |0> y |1>.
#   3. Rotacion controlada: si q0 vale |i>, Ry(theta_i) sobre q1, con
#      theta_i = 2 arcsin(c / l_i). Asi la amplitud de |1> en q1 es c / l_i.
#   4. U: regresa a la base original.
# c es un numero ajustable con c <= l_min. Aqui se toma el mayor posible.
def hhl_2x2(A, b):
    autovalores, U = np.linalg.eigh(A)
    c = autovalores.min()
    qc = QuantumCircuit(2)
    qc.prepare_state(b / np.linalg.norm(b), [0])        # 1. |b> en q0
    qc.unitary(U.conj().T, [0], label="U^dagger")        # 2. cambio de base
    for k, l in enumerate(autovalores):                  # 3. rotaciones controladas
        theta = 2 * np.arcsin(c / l)
        if k == 0:
            qc.x(0)                                      # controla cuando q0 = |0>
        qc.cry(theta, 0, 1)
```

```
        if k == 0:
            qc.x(0)
    qc.unitary(U, [0], label="U")                          # 4. vuelta a la base original
    return qc, c


# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
hhl_2x2(A, b)[0].draw()


# %% Celda 5: ejecutar y quedarse con el caso "auxiliar = 1"
# La solucion aparece en q0 solo cuando q1 se mide en |1>. Se obtiene con
# probabilidad P1 (depende de c). Las amplitudes de q0 dadas q1 = 1 son proporcionales a x.
def resolver(A, b):
    qc, c = hhl_2x2(A, b)
    amplitudes = Statevector(qc).data           # indices: q1 q0 = 00, 01, 10, 11
    cond = amplitudes[2:]                        # q1 = 1: q0 = 0, 1
    prob_exito = float(np.sum(abs(cond) ** 2))
    return np.real(cond / np.sqrt(prob_exito)), prob_exito


x_cuantico, p_exito = resolver(A, b)
# Una fase global (signo) no importa: se compara el valor absoluto.
x_ref = x_clasico / np.linalg.norm(x_clasico)
print("x cuantico (normalizado):", np.abs(x_cuantico).round(4), "| clasico:", np.abs(x_ref).
    ↪ round(4))
print("Probabilidad de que el auxiliar valga 1:", round(p_exito, 4))

# %% Celda 6: la matriz del experimento
# En el laboratorio se resuelve  A x = b  con
#     A = [[2.14645, -0.35355], [-0.35355, 2.85355]],  b = (0.70711, 0.70711).
# Tiene autovalores 2 y 3. Sus autovectores son (cos(pi/8), sin(pi/8)) y
# (-sin(pi/8), cos(pi/8)): la matriz U es una rotacion Ry(pi/4).
A2 = np.array([[2.14645, -0.35355], [-0.35355, 2.85355]])
b2 = np.array([0.70711, 0.70711])
autovalores2, U2 = np.linalg.eigh(A2)
Ry_pi_4 = np.array([[np.cos(np.pi / 8), -np.sin(np.pi / 8)], [np.sin(np.pi / 8), np.cos(np.pi /
    ↪ 8)]])
print("Autovalores de A:", autovalores2.round(4))
print("U coincide con Ry(pi/4) (salvo signos):", np.allclose(abs(U2), abs(Ry_pi_4), atol=1e-4))

x2, p2 = resolver(A2, b2)
x2_clasico = np.linalg.solve(A2, b2)
x2_clasico = x2_clasico / np.linalg.norm(x2_clasico)
print("x cuantico (normalizado):", np.abs(x2).round(4), "| clasico:", np.abs(x2_clasico).round
    ↪ (4))
print("Probabilidad de que el auxiliar valga 1:", round(p2, 4))

# %% Celda 7: probabilidades que se medirian
# En un equipo real se mide en la base computacional. Las probabilidades de
# |q1 q0> = |00>, |01>, |10>, |11> salen de las amplitudes del estado final.
qc2, _ = hhl_2x2(A2, b2)
probs = np.abs(Statevector(qc2).data) ** 2
for etiqueta, p in zip(("00", "01", "10", "11"), probs):
    print(f"P(|{etiqueta}>) = {p:.4f}")
```

20.5. Explicación celda por celda

Celda 2 Resuelve el sistema de las vasijas de forma clásica.

Celda 3 Calcula autovalores y autovectores, los coeficientes β_i, y comprueba que $\sum_i (\beta_i/\lambda_i)\,|u_i\rangle$ da la misma solución que la clásica (normalizada).

Celda 4 Define el circuito HHL de dos qubits: codificar $|b\rangle$, $U^\dagger$, rotaciones controladas y U.

Celda 5 Calcula las amplitudes del qubit de codificación cuando el auxiliar vale $|1\rangle$ y las compara con la solución clásica.

Celda 6 Repite con la matriz del experimento y comprueba que su U es $R_y(\pi/4)$.

Celda 7 Imprime las probabilidades de las cuatro cadenas $|q_1 q_0\rangle$ que se medirían.

20.6. Salida obtenida

```
Solucion clasica x = A^-1 b = [0.4167 0.0833] (5/12, 1/12)
Autovalores: [2. 3.]
beta_i = <u_i|b>: [-0.7071 -0.7071]
Solucion con autovalores (normalizada): [0.9806 0.1961]
Solucion clasica       (normalizada): [0.9806 0.1961]
x cuantico (normalizado): [0.9806 0.1961] | clasico: [0.9806 0.1961]
Probabilidad de que el auxiliar valga 1: 0.7222
Autovalores de A: [2. 3.]
U coincide con Ry(pi/4) (salvo signos): True
x cuantico (normalizado): [0.7887 0.6148] | clasico: [0.7887 0.6148]
Probabilidad de que el auxiliar valga 1: 0.9186
P(|00>) = 0.0119
P(|01>) = 0.0694
P(|10>) = 0.5714
P(|11>) = 0.3472
```

(El dibujo del circuito no se reproduce aquí.) Es determinista.

20.7. Interpretación

Para el sistema de las vasijas, la solución cuántica normalizada es $(0.9806, 0.1961)$, igual que la clásica $(5/12, 1/12)$ normalizada. Para la matriz del experimento, $A = \left(\begin{smallmatrix} 2.14645 & -0.35355 \\ -0.35355 & 2.85355 \end{smallmatrix}\right)$ y $\mathbf{b} = (0.70711, 0.70711)$, también coinciden: $(0.7887, 0.6148)$. Sus autovalores son 2 y 3 y su matriz U coincide con $R_y(\pi/4)$ (salvo signos). Al medir, la solución solo se obtiene cuando el auxiliar vale 1, con probabilidad 0.72 en el primer caso y 0.92 en el segundo.

Qué se simplifica en este ejemplo

Igual que en el PDF de la práctica, aquí los autovalores de A se *conocen* y se usan directamente para armar las rotaciones; la estimación de fase que los obtiene en un HHL completo se sustituye por esa información previa. Además, un estado cuántico no se puede leer entero: el código calcula las amplitudes con el simulador, lo que un equipo real no permite.

20.8. Ejercicios

1. Cambia $\mathbf{b}$ por $(\cos\varphi, \sin\varphi)$ con otro ángulo y compara con la solución clásica.
2. Cambia c a un valor menor que 2: ¿qué pasa con la probabilidad de que el auxiliar valga 1?
3. Prueba con una matriz 2×2 simétrica distinta (con autovalores positivos) y comprueba el resultado.

20.9. Errores comunes

- Confundir $|x\rangle$ con $\mathbf{x}$: HHL entrega el vector *normalizado*, no su tamaño.

- Elegir c mayor que el menor autovalor: entonces $c/\lambda_i > 1$ y el ángulo $2\sin^{-1}(c/\lambda_i)$ no existe.

21. VQE: eigensolver cuántico variacional

21.1. Objetivo

Calcular la energía del estado fundamental de un Hamiltoniano con un algoritmo híbrido: un circuito cuántico parametrizado prepara estados de prueba y un optimizador clásico ajusta los parámetros.

21.2. Conceptos previos

Concepto previo: valor esperado, Hamiltoniano y era NISQ

Ver capítulos 08 (era NISQ), 12 (Hamiltoniano) y 13 (valores esperados de Pauli). El *valor esperado* de H en el estado $|\psi\rangle$ es $\langle H\rangle = \langle\psi| H |\psi\rangle = \mathrm{Tr}(H\rho)$ con $\rho = |\psi\rangle\langle\psi|$.

21.3. Fundamento teórico

Los computadores cuánticos actuales tienen ruido y pocos qubits (era NISQ, término de John Preskill de 2018). En ese contexto son muy usados los algoritmos **híbridos cuántico-clásicos**: parte del cálculo corre en un computador clásico y parte en uno cuántico. El VQE es uno de ellos; su tarea es calcular la energía aproximada del estado fundamental de un Hamiltoniano. Se aplica sobre todo en simulación de química cuántica (moléculas como el agua o cadenas de hidrógeno) y también en optimización y aprendizaje automático cuántico.

Principio variacional. Para cualquier estado de prueba $|\psi\rangle$,

$$E_0 \;\le\; \frac{\langle\psi| H |\psi\rangle}{\langle\psi|\psi\rangle}, \tag{35}$$

donde E_0 es la energía del estado fundamental. Por eso, minimizar el valor esperado acerca el resultado a E_0.

Los pasos del VQE.

1. **Hamiltoniano:** se escribe el problema como un Hamiltoniano de operadores de Pauli.
2. **Estado de prueba:** un circuito parametrizado (rotaciones con parámetro y puertas de dos qubits como CNOT o CZ) se aplica a $|0\ldots0\rangle$.
3. **Valor esperado:** se mide $\langle H\rangle$ en el computador cuántico y se envía al clásico.
4. **Optimización:** un método clásico (con o sin gradiente) actualiza los parámetros.
5. Se repiten los pasos 3 y 4 hasta converger o llegar al máximo de iteraciones.
6. La última iteración da el autoestado y el autovalor óptimos.

Regla del desplazamiento de parámetros. Para obtener el gradiente se evalúa el circuito con el parámetro movido:

$$\frac{dE}{d\theta} = \frac{E(\theta+s) - E(\theta-s)}{2} \quad (s=\pi/2), \qquad \theta \leftarrow \theta - \eta\,\frac{dE}{d\theta}, \tag{36}$$

con η la tasa de aprendizaje. Esta forma (con $s=\pi/2$) es exacta para puertas de rotación $e^{-i\theta P/2}$; con un desplazamiento δ cualquiera se obtiene solo una aproximación.

Cálculo de $\langle H \rangle$. Se puede hacer con el vector de estado, $\langle\psi| H |\psi\rangle = \sum_{i,j} \bar{\psi}_i H_{ij} \psi_j$, o con la traza, $\mathrm{Tr}(H\rho)$. Dan el mismo resultado.

El caso del ejemplo. El modelo de Heisenberg de dos qubits, $H = XX + YY + ZZ$, tiene energías -3 (estado fundamental, el singlete $(|01\rangle - |10\rangle)/\sqrt{2}$) y $+1$ (tres veces). El circuito de prueba usa dos parámetros: $R_y(\theta_1)$ sobre q_0, una CNOT $q_0 \to q_1$ y $R_x(\theta_2)$ sobre q_0.

21.4. Código (completamente comentado)

```python
# %% [markdown]
# # 19 - VQE: encontrar la energia del estado fundamental con un circuito parametrizado

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, SparsePauliOp, Statevector
import matplotlib.pyplot as plt

# %% Celda 2: el problema
# Se busca la energia mas baja de un Hamiltoniano (la energia del estado fundamental).
# Aqui: el modelo de Heisenberg de dos qubits,  H = XX + YY + ZZ.
H = SparsePauliOp(["XX", "YY", "ZZ"])
H_matriz = H.to_matrix()
# Como el sistema es pequeno, se puede diagonalizar para saber la respuesta exacta.
print("Energias exactas de H:", np.linalg.eigvalsh(H_matriz).round(4))
print("Energia del estado fundamental:", float(np.linalg.eigvalsh(H_matriz).min()))
# VQE busca ese minimo SIN diagonalizar, usando solo valores esperados.

# %% Celda 3: dos formas de calcular el valor esperado <H>
# (1) Con el vector de estado:   <H> = <psi| H |psi>.
# (2) Con la matriz densidad:    <H> = tr(H rho),   rho = |psi><psi|.
# Dan el mismo numero. Se prueba con un estado cualquiera.
psi = Statevector(np.array([1, 1j, -1, 0.5]) / np.linalg.norm([1, 1j, -1, 0.5]))
via_vector = np.real(psi.data.conj() @ H_matriz @ psi.data)
via_traza = np.real(np.trace(H_matriz @ DensityMatrix(psi).data))
print(f"<psi|H|psi> = {via_vector:.4f} | tr(H rho) = {via_traza:.4f}")


# %% Celda 4: el circuito parametrizado ("ansatz")
# El estado de prueba |psi(theta)> sale de aplicar un circuito con parametros
# theta = (theta1, theta2) al estado inicial |00>:
#     Ry(theta1) en q0, CNOT(q0 -> q1), Rx(theta2) en q0.
def ansatz(theta):
    qc = QuantumCircuit(2)
    qc.ry(theta[0], 0)
    qc.cx(0, 1)
    qc.rx(theta[1], 0)
    return qc


def energia(theta):
    """Valor esperado <H> del estado de prueba (via tr(H rho))."""
    rho = DensityMatrix(Statevector(ansatz(theta)))
    return float(np.real(np.trace(H_matriz @ rho.data)))


# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
ansatz([0.3, 0.5]).draw()

# %% Celda 5: regla del desplazamiento de parametros (parameter-shift)
# Para saber hacia donde mover cada parametro hace falta el gradiente dE/dtheta.
```

```python
# Con una puerta de rotacion exp(-i theta P / 2) se puede obtener exacto con
# solo dos evaluaciones del circuito:
#     dE/dtheta = ( E(theta + s) - E(theta - s) ) / 2,    con s = pi/2.
def gradiente(theta, s=np.pi / 2):
    g = np.zeros_like(theta)
    for i in range(len(theta)):
        mas, menos = theta.copy(), theta.copy()
        mas[i] += s
        menos[i] -= s
        g[i] = (energia(mas) - energia(menos)) / 2
    return g


# Se comprueba contra una derivada numerica con un paso muy pequeno.
theta0 = np.array([1.1, 2.3])
eps = 1e-6
numerico = np.array([(energia(theta0 + eps * e) - energia(theta0 - eps * e)) / (2 * eps) for e
    ↪ in np.eye(2)])
print("Gradiente (parameter-shift):", gradiente(theta0).round(5))
print("Gradiente (numerico)       :", numerico.round(5))

# %% Celda 6: el ciclo de VQE
# 1. Elegir parametros iniciales.
# 2. Calcular <H> (computador cuantico).
# 3. Calcular el gradiente con parameter-shift.
# 4. Actualizar:  theta <- theta - lr * gradiente  (computador clasico).
# 5. Repetir hasta que <H> converja.
theta = np.array([1.3, 2.0])      # parametros iniciales (radianes)
lr = 0.3                          # tasa de aprendizaje
historia = [energia(theta)]
print("iteracion | theta1  theta2 |  <H>")
for it in range(1, 41):
    theta = theta - lr * gradiente(theta)
    historia.append(energia(theta))
    if it <= 5 or it % 10 == 0:
        print(f"{it:9d} | {theta[0]:6.3f} {theta[1]:6.3f} | {historia[-1]:+.4f}")
print(f"Energia final = {historia[-1]:.4f} (exacta: -3)")

# %% Celda 7: grafica de convergencia
plt.plot(historia, "o-")
plt.axhline(-3, color="gray", linestyle="--", label="energía exacta (-3)")
plt.xlabel("iteración")
plt.ylabel("<H>")
plt.title("Convergencia de VQE")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

21.5. Explicación celda por celda

Celda 2 Define $H = XX + YY + ZZ$ y calcula sus energías exactas por diagonalización, solo para comparar.

Celda 3 Calcula $\langle H \rangle$ de un estado de prueba de las dos formas y muestra que coinciden.

Celda 4 Define el circuito de prueba y la función de energía $\mathrm{Tr}(H\rho)$.

Celda 5 Implementa la regla del desplazamiento y la compara con una derivada numérica.

Celda 6 El ciclo de VQE: 40 iteraciones con tasa de aprendizaje 0.3.

Celda 7 Gráfica de convergencia.

21.6. Salida obtenida

```
Energias exactas de H: [-3.  1.  1.  1.]
Energia del estado fundamental: -3.0
<psi|H|psi> = -0.2308 | tr(H rho) = -0.2308
Gradiente (parameter-shift): [ 0.75582 -0.08113]
Gradiente (numerico)       : [ 0.75582 -0.08113]
iteracion | theta1  theta2 |  <H>
        1 |  1.186  2.010 | +0.8960
        2 |  1.026  2.030 | +0.7911
        3 |  0.802  2.069 | +0.5841
        4 |  0.493  2.143 | +0.1886
        5 |  0.086  2.276 | -0.5063
       10 | -1.519  3.109 | -2.9962
       20 | -1.571  3.142 | -3.0000
       30 | -1.571  3.142 | -3.0000
       40 | -1.571  3.142 | -3.0000
Energia final = -3.0000 (exacta: -3)
```

(El dibujo del circuito no se reproduce aquí.) Es determinista.

21.7. Interpretación

Las dos formas de calcular $\langle H \rangle$ dan -0.2308, y el gradiente por desplazamiento coincide con el numérico. Partiendo de $\theta = (1.3, 2.0)$ con $\langle H \rangle = +0.9484$, la energía baja a -0.51 en 5 iteraciones, a -2.996 en 10 y a -3.0000 en 20: el valor exacto del estado fundamental. Los parámetros terminan en $(-\pi/2,\ \pi)$.

21.8. Ejercicios

1. Cambia la tasa de aprendizaje a 0.1 y a 1.0. ¿Cuántas iteraciones hacen falta en cada caso?
2. Prueba otros parámetros iniciales: ¿converge siempre a -3?
3. Cambia el Hamiltoniano a $H = ZZ$ y observa qué energía mínima se obtiene.

21.9. Errores comunes

- Creer que VQE garantiza el mínimo global: el resultado depende del circuito de prueba y de los parámetros iniciales, y puede quedarse en un mínimo local.
- Olvidar que el valor esperado se estima con muchas mediciones en un equipo real; aquí se calcula exacto con el simulador.

22. QAOA: algoritmo de optimización cuántica aproximada

22.1. Objetivo

Resolver el problema Max-Cut con un circuito variacional de capas alternadas de costo y de mezcla, y comprobar cómo influye el número de capas p.

22.2. Conceptos previos

Concepto previo: VQE y rotaciones ZZ

Ver capítulo 19 (VQE, regla del desplazamiento) y capítulo 12 (rotaciones y acoplamiento $\sigma_z \otimes \sigma_z$). QAOA usa la misma idea híbrida: un circuito parametrizado y un optimizador clásico.

22.3. Fundamento teórico

QAOA fue propuesto por Farhi, Goldstone y Gutmann en 2014. Es un algoritmo híbrido cuántico-clásico para problemas de optimización combinatoria (como el viajante TSP, la coloración de grafos o Max-Cut), aplicables a logística, planificación de producción o diseño de circuitos.

Algoritmo adiabático cuántico (QAA). Si un sistema empieza en el estado fundamental de un Hamiltoniano H_I y $H(t)$ cambia lo bastante despacio hasta H_F, el sistema permanece en el estado fundamental. Se elige H_I con un estado fundamental fácil de preparar y H_F de modo que su estado fundamental codifique la solución. Con $H(t)$ pasando de H_I en $t = 0$ a H_F en $t = T$, la evolución se simula con la fórmula de Trotter–Suzuki, alternando pasos pequeños de H_I y de H_F. En la práctica, el ruido y los límites físicos impiden las evoluciones largas que pide la adiabaticidad.

QAOA. Adopta la misma idea con evoluciones más cortas y rápidas, y busca una solución *aproximada* optimizando parámetros. Los pasos son:

1. **Hamiltonianos:** se define un Hamiltoniano de costo H_C (el estado de mayor energía codifica la solución) y uno de mezcla H_M que no conmuta con H_C (aquí, $H_M = \sum_i X_i$).
2. **Estado inicial:** $|s\rangle = |+\rangle^{\otimes n}$, el estado de mayor energía de H_M.
3. **Circuito:** la capa de costo $U_C(\gamma) = e^{-i\gamma H_C}$ y la de mezcla $U_M(\beta) = e^{-i\beta H_M}$ (que es $R_x(2\beta)$ en cada qubit).
4. **Evolución:** con p capas, $\gamma = (\gamma_1, \ldots, \gamma_p)$ y $\beta = (\beta_1, \ldots, \beta_p)$:
$$|\psi_p(\gamma, \beta)\rangle = e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \cdots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n}. \tag{37}$$
5. **Valor esperado:** $E(\gamma, \beta) = \langle\psi_p | H_C | \psi_p\rangle$.
6. **Optimización:** un método clásico actualiza γ y β para *maximizar* E.

Max-Cut. Dado un grafo $G = (V, E)$ con n vértices, se reparten los vértices en dos grupos para *maximizar* el número de aristas entre grupos distintos. Si $x_i \in \{0, 1\}$ dice a qué grupo (por ejemplo, a qué día) pertenece el vértice i, la función a maximizar es

$$L(x) = \sum_{(u,v)\in E} \big(x_u + x_v - 2x_u x_v\big). \tag{38}$$

Para dos lugares turísticos unidos por una arista hay $2^2 = 4$ formas de repartirlos: 00 y 11 cortan 0 aristas y 01 y 10 cortan 1. Para pasar a un Hamiltoniano se sustituye $x_i \to \frac{I - Z_i}{2}$ (vale 0 en $|0\rangle$ y 1 en $|1\rangle$) y queda

$$H_C = \sum_{(u,v)\in E} \frac{1 - Z_u Z_v}{2}. \tag{39}$$

Cómo se hace $e^{-i\gamma Z_u Z_v}$. Con una CNOT, una rotación R_z sobre el segundo qubit y otra CNOT. En el ejemplo, la capa de costo con ángulo de puerta $a = -\gamma$ coincide con $e^{-i\gamma H_C}$ salvo una fase global.

22.4. Código (completamente comentado)

```python
# %% [markdown]
# # 20 - QAOA: resolver el problema Max-Cut con un circuito variacional

# %% Celda 1: importaciones
import itertools
import numpy as np
from scipy.linalg import expm
from qiskit import QuantumCircuit
from qiskit.quantum_info import Operator, Statevector, SparsePauliOp

# %% Celda 2: el problema Max-Cut
# Dado un grafo, se reparten los vertices en dos grupos (0 o 1) para que haya
# el MAYOR numero posible de aristas entre grupos distintos ("aristas cortadas").
# Ejemplo del PDF: dos lugares turisticos unidos por una arista; x_i dice en
# que dia se visita el lugar i. Funcion a maximizar:
#     L(x) = suma sobre aristas (u,v) de  x_u + x_v - 2 x_u x_v.
aristas = [(0, 1)]
n = 2


def corte(x):
    return sum(x[u] + x[v] - 2 * x[u] * x[v] for u, v in aristas)


for x in itertools.product((0, 1), repeat=n):
    print(f"x = {x} -> aristas cortadas = {corte(x)}")

# %% Celda 3: pasar el problema a un Hamiltoniano
# Se cambia x_i por (I - Z_i)/2 (vale 0 si el qubit esta en |0> y 1 si esta en |1>):
#     Hc = suma sobre aristas de (1 - Z_u Z_v)/2.
# Su valor esperado es el numero promedio de aristas cortadas. Maximizarlo
# resuelve el problema.
def hamiltoniano_costo():
    terminos, coefs = [], []
    for u, v in aristas:
        terminos += ["I" * n, "".join("Z" if i in (u, v) else "I" for i in range(n))[::-1]]
        coefs += [0.5, -0.5]
    return SparsePauliOp(terminos, coefs).simplify()


Hc = hamiltoniano_costo()
Hc_matriz = Hc.to_matrix()
print("Autovalores de Hc:", np.linalg.eigvalsh(Hc_matriz).round(3), "-> el maximo es", Hc_matriz
    ↪ .diagonal().real.max())


# %% Celda 4: el circuito QAOA
# Estado inicial |+>...|+> (todas las cadenas con la misma amplitud). Cada capa tiene:
```

```
#   - capa de costo   e^{-i gamma Hc}: para cada arista, una rotacion ZZ,
#   - capa de mezcla  e^{-i beta sum X_i}: una Rx(2 beta) en cada qubit.
# Aqui los parametros se usan como angulos de las puertas: a = -gamma en la rotacion
# ZZ (puerta Rzz(a) = CNOT, Rz(a), CNOT) y b = 2 beta en la Rx.
def circuito_qaoa(params, desplazar=None):
    """Circuito QAOA. `desplazar` = (parametro, numero_de_uso, delta) mueve el angulo
    de UNA sola puerta (se necesita para el gradiente, ver Celda 5)."""
    usos = {}

    def angulo(i):
        k = usos.get(i, 0)
        usos[i] = k + 1
        if desplazar is not None and desplazar[0] == i and desplazar[1] == k:
            return params[i] + desplazar[2]
        return params[i]

    p = len(params) // 2
    qc = QuantumCircuit(n)
    qc.h(range(n))
    for capa in range(p):
        for u, v in aristas:
            qc.cx(u, v)
            qc.rz(angulo(2 * capa), v)
            qc.cx(u, v)
        for q in range(n):
            qc.rx(angulo(2 * capa + 1), q)
    return qc


# Comprobacion: la capa de costo con angulo a = -gamma es e^{-i gamma Hc}
# (salvo una fase global).
gamma = 0.7
capa_costo = QuantumCircuit(n)
capa_costo.cx(0, 1)
capa_costo.rz(-gamma, 1)
capa_costo.cx(0, 1)
exacta = expm(-1j * gamma * Hc_matriz)
M = Operator(capa_costo).data
fase = (exacta[0, 0] / M[0, 0])
print("La capa de costo coincide con e^{-i gamma Hc} (salvo fase):", np.allclose(exacta, fase *
    ↪ M))

# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
circuito_qaoa([0.5, 0.3]).draw()


# %% Celda 5: el valor esperado <Hc> y su gradiente
def valor_esperado(params, desplazar=None):
    return float(np.real(Statevector(circuito_qaoa(params, desplazar)).expectation_value(Hc)))


def gradiente(params, s=np.pi / 2):
    """Regla del desplazamiento de parametros: dE/d(angulo) = (E(+s) - E(-s)) / 2.

    Un mismo parametro aparece en varias puertas (la mezcla usa el mismo beta en
    todos los qubits). La regla vale para UNA puerta a la vez, asi que se suma la
    contribucion de cada puerta donde aparece el parametro."""
    g = np.zeros_like(params)
    for i in range(len(params)):
        usos = len(aristas) if i % 2 == 0 else n
        for k in range(usos):
            g[i] += (valor_esperado(params, (i, k, s)) - valor_esperado(params, (i, k, -s))) / 2
    return g
```

```
110
111
112 # Se comprueba contra una derivada numerica.
113 p0 = np.array([0.4, 0.9])
114 eps = 1e-6
115 numerico = np.array([(valor_esperado(p0 + eps * e) - valor_esperado(p0 - eps * e)) / (2 * eps)
        ↪ for e in np.eye(2)])
116 print("Gradiente (parameter-shift):", gradiente(p0).round(5), "| numerico:", numerico.round(5))
117
118
119 # %% Celda 6: optimizar (maximizar <Hc>) con p = 1 capa
120 # Ascenso por gradiente:  params <- params + lr * gradiente.
121 def optimizar(p, lr=0.3, iteraciones=60, semilla=1):
122     rng = np.random.default_rng(semilla)
123     params = rng.uniform(0, 1, 2 * p)
124     for it in range(iteraciones):
125         params = params + lr * gradiente(params)
126     return params, valor_esperado(params)
127
128
129 params, valor = optimizar(1)
130 print(f"p = 1: parametros = {params.round(3)} | <Hc> = {valor:.4f} (maximo posible = 1)")
131
132 # %% Celda 7: que cadena se mide con mas probabilidad
133 # Con los parametros optimos se miden casi siempre cadenas que cortan la arista.
134 probs = Statevector(circuito_qaoa(params)).probabilities_dict()
135 for cadena, prob in sorted(probs.items()):
136     print(f"cadena {cadena}: probabilidad = {prob:.4f} | aristas cortadas = {corte([int(c) for c
        ↪  in cadena[::-1]])}")
137
138 # %% Celda 8: cambiar el numero de capas p
139 # El PDF pide repetir con otros p para ver si el resultado depende de p.
140 for p in (1, 2, 3):
141     _, valor = optimizar(p)
142     print(f"p = {p}: <Hc> = {valor:.4f}")
```

22.5. Explicación celda por celda

Celda 2 Lista las cuatro formas de repartir los dos lugares y cuántas aristas corta cada una.

Celda 3 Construye H_C y comprueba que su valor máximo es 1.

Celda 4 Arma el circuito QAOA y verifica que la capa de costo es $e^{-i\gamma H_C}$.

Celda 5 Calcula $\langle H_C \rangle$ y su gradiente. Como un mismo parámetro aparece en varias puertas (la mezcla usa el mismo β en todos los qubits), la regla del desplazamiento se aplica a cada puerta por separado y se suman las contribuciones. Se compara con la derivada numérica.

Celda 6 Ascenso por gradiente con $p = 1$.

Celda 7 Muestra las probabilidades de cada cadena con los parámetros óptimos.

Celda 8 Repite con $p = 1, 2, 3$, como pide la práctica.

22.6. Salida obtenida

```
x = (0, 0) -> aristas cortadas = 0
x = (0, 1) -> aristas cortadas = 1
x = (1, 0) -> aristas cortadas = 1
x = (1, 1) -> aristas cortadas = 0
Autovalores de Hc: [0. 0. 1. 1.] -> el maximo es 1.0
```

```
La capa de costo coincide con e^{-i gamma Hc} (salvo fase): True
Gradiente (parameter-shift): [-0.44849  0.08848] | numerico: [-0.44849  0.08848]
p = 1: parametros = [-1.571  0.785] | <Hc> = 1.0000 (maximo posible = 1)
cadena 00: probabilidad = 0.0000 | aristas cortadas = 0
cadena 01: probabilidad = 0.5000 | aristas cortadas = 1
cadena 10: probabilidad = 0.5000 | aristas cortadas = 1
cadena 11: probabilidad = 0.0000 | aristas cortadas = 0
p = 1: <Hc> = 1.0000
p = 2: <Hc> = 1.0000
p = 3: <Hc> = 1.0000
```

(El dibujo del circuito no se reproduce aquí.) Es determinista (la optimización usa una semilla fija).

22.7. Interpretación

El gradiente por desplazamiento coincide con el numérico (-0.44849 y 0.08848). Con $p = 1$ el valor esperado llega a 1.0000 (el máximo): la probabilidad se reparte entre 01 y 10 (0.5 cada una), que son las dos formas de cortar la arista, y 00 y 11 quedan en 0. Con $p = 2$ y $p = 3$ también se obtiene 1.0000: en este grafo tan pequeño, una sola capa alcanza.

22.8. Ejercicios

1. Cambia el grafo a un triángulo (aristas $(0, 1), (1, 2), (0, 2)$, $n = 3$). ¿Cuál es el valor máximo de H_C? ¿Llega QAOA con $p = 1$?

2. Cambia la tasa de aprendizaje y el número de iteraciones y mira cómo cambia el resultado.

3. Prueba otras semillas para los parámetros iniciales con $p = 1$: ¿llega siempre a 1?

22.9. Errores comunes

- Confundir el signo: QAOA aquí *maximiza* $\langle H_C \rangle$ (el número de aristas cortadas), mientras que VQE minimiza una energía.

- Olvidar que los parámetros compartidos necesitan sumar las contribuciones de cada puerta al calcular el gradiente.

23. Decoherencia y spin echo: perder y recuperar la fase de un qubit

23.1. Objetivo

Entender por qué un qubit pierde información con el tiempo (relajación T_1 y T_2, estados mezclados), cómo el *spin echo* recupera la fase perdida por la inhomogeneidad del campo y para qué sirve la secuencia CPMG.

23.2. Conceptos previos

Concepto previo: esfera de Bloch, precesión y estados mezclados

Ver capítulos 10 (esfera de Bloch), 11 (precesión y pulsos) y 12 (rotaciones). Un *estado puro* es un punto en la superficie de la esfera de Bloch; un *estado mezclado* es un punto en su interior.

23.3. Fundamento teórico

Relajación y pérdida de información. Un qubit de espín 1/2 en un campo magnético tiene $|\uparrow\rangle \leftrightarrow |0\rangle$ y $|\downarrow\rangle \leftrightarrow |1\rangle$. Idealmente, el módulo del momento angular de espín promedio, $|\langle S\rangle| = \sqrt{\langle S_x\rangle^2 + \langle S_y\rangle^2 + \langle S_z\rangle^2}$, se conserva y el estado es puro. En la práctica, la relajación hace que el vector se acorte y entre en la esfera: el estado se vuelve **mezclado**, $\rho_{\text{mix}} = p_1 |1\rangle\langle 1| + p_2 |2\rangle\langle 2| + \cdots$, una combinación de estados puros con probabilidades clásicas. Eso representa *pérdida de información*, porque aumenta la incertidumbre. Si el estado se mezcló poco, se puede estimar el estado puro correspondiente; si la componente transversal desaparece por completo, la información de fase se perdió por completo, y en el caso extremo (estado máximamente mezclado) ya no se puede obtener ninguna información midiendo. Hay dos procesos:

- **Relajación longitudinal** (T_1): cambia la población de los niveles (la componente z). Es la pérdida de energía hacia el entorno.
- **Relajación transversal** (T_2, *decoherencia*): se pierde la coherencia de fase (las componentes x e y).

En los sistemas de resonancia magnética nuclear (RMN), la relajación transversal pesa más que la longitudinal en la pérdida de información.

Inhomogeneidad del campo. Una muestra tiene muchos espines (un *conjunto*). Si el campo no es uniforme, cada espín gira con una frecuencia de Larmor ligeramente distinta, así que sus fases se separan con el tiempo y la magnetización neta (el promedio) se acorta. Tras un pulso $R_y(\pi/2)$ que lleva el espín a x, la señal oscila y decae con envolvente exponencial e^{-t/T_2^*}. En un sistema sin conjunto (un solo espín medido muchas veces) pasa lo mismo si el campo fluctúa en el *tiempo*.

El eco de espín. Erwin Hahn descubrió en 1950 que un pulso π (de 180 grados) aplicado en medio de la relajación revierte las fases y produce un "eco". Para un conjunto de espines, con tiempo total t:

1. Al inicio todos apuntan a x: polarización máxima.
2. Durante $t/2$ cada espín acumula la fase $\theta_i = \omega_i t/2$ y la magnetización decae.
3. Un pulso $R_x(\pi)$ en $t/2$ invierte las fases: $\theta_i \to -\omega_i t/2$.

4. Durante la segunda mitad cada espín vuelve a acumular $\omega_i t/2$.

5. En t la fase es $-\omega_i t/2 + \omega_i t/2 = 0$ para todos: la polarización se recupera.

El eco es *menor* que la señal inicial: el spin echo retrasa la relajación transversal pero no la elimina, porque los espines también interactúan con la red (relajación natural). Con eco, la señal decae con T_2; sin eco, con T_2^*, y en general $T_2 \geq T_2^*$. Después de $t = T_2$ la componente transversal baja a $1/e \approx 0.368$ de su máximo. Para informática cuántica se usa T_2 como tiempo de coherencia.

Secuencias. Un solo pulso π da un eco, pero su fase puede no coincidir con la inicial (depende del eje del pulso y de la dirección inicial). Para conservar el estado se añade un segundo pulso π en el eco: así la fase coincide siempre con la inicial. La secuencia **CPMG** (Carr, Purcell, Meiboom y Gill) aplica pulsos π a intervalos regulares para intervalos largos; *CPMG-n* son n pulsos en un tiempo t, con separación t/n (el primero y el último a $t/2n$ de los extremos). CPMG funciona en muchos experimentos, pero no elimina todas las fuentes de decaimiento (como la difusión o el intercambio químico).

Fuentes y modelo del ejemplo

Este capítulo se basa en las notas de spin echo del curso. Los valores numéricos ($T_1 = 100$ ms, $T_2 = 40$ ms, dispersión de frecuencias de 100 Hz, fluctuación lenta de 30 Hz con tiempo de correlación de 10 ms) **son elegidos para ilustrar**, no medidos. El archivo de teoría de decoherencia del curso es un escaneo sin texto legible, así que el modelo de Bloch para T_1 y T_2 es el estándar de la definición de las notas.

23.4. Código (completamente comentado)

```
# %% [markdown]
# # 21 - Decoherencia y spin echo: perder y recuperar la fase de un qubit

# %% Celda 1: importaciones
import numpy as np

rng = np.random.default_rng(11)   # semilla fija: las salidas se pueden reproducir

# %% Celda 2: relajacion y estados mezclados
# Un qubit puro es un punto en la SUPERFICIE de la esfera de Bloch (vector de longitud 1).
# Con el tiempo el entorno lo "acorta": el vector entra en la esfera y el estado se
# vuelve MEZCLADO (se pierde informacion). Hay dos procesos (en este modelo, ademas, la
# poblacion termina volviendo al estado de equilibrio |0>, por eso el vector
# vuelve a alargarse al final):
#   - relajacion longitudinal (T1): cambia la poblacion (la componente z),
#   - relajacion transversal (T2): se pierde la fase (las componentes x e y).
# Modelo de Bloch con estos dos tiempos (T2 <= 2 T1). Los valores de T1 y T2 de
# este ejemplo son elegidos para ilustrar, no medidos.
T1, T2 = 100e-3, 40e-3        # segundos


def vector_bloch(t, r0=(1.0, 0.0, 0.0)):
    x0, y0, z0 = r0
    z_eq = 1.0                  # estado de equilibrio: |0>, z = +1
    return np.array([x0 * np.exp(-t / T2), y0 * np.exp(-t / T2), z_eq - (z_eq - z0) * np.exp(-t
    ↪ / T1)])


# Se prepara el qubit sobre el ecuador con un pulso Ry(pi/2): r0 = (1, 0, 0).
print("t (ms) |  x      y      z    | |r|   | pureza = (1 + |r|^2)/2")
```

```
for t_ms in (0, 10, 40, 100, 300):
    r = vector_bloch(t_ms * 1e-3)
    largo = np.linalg.norm(r)
    print(f"{t_ms:6d} | {r[0]:+.3f} {r[1]:+.3f} {r[2]:+.3f} | {largo:.3f} | {(1 + largo ** 2) / 2:.3f}")
# La componente transversal cae con exp(-t/T2) (mas rapido aqui); la longitudinal
# se recupera con 1 - exp(-t/T1).

# %% Celda 3: desfase en un conjunto de espines (inhomogeneidad)
# Una muestra tiene muchos espines. Si el campo magnetico no es uniforme, cada
# espin gira con una frecuencia ligeramente distinta f_i. Todos empiezan en la
# direccion x (fase 0), pero tras un tiempo t tienen fases 2*pi*f_i*t distintas, y la
# magnetizacion neta (el promedio) se acorta:  M(t) = promedio de e^{i 2 pi f_i t}.
N = 5000
sigma = 100.0                       # dispersion de frecuencias: 100 Hz
f = rng.normal(0.0, sigma, N)       # una frecuencia por espin
T2_estrella = 1 / (2 * np.pi * sigma)
print(f"T2* (escala del desfase) ~ {T2_estrella * 1e3:.2f} ms")
for t_ms in (0, 1, 2, 4, 8):
    t = t_ms * 1e-3
    M = abs(np.mean(np.exp(2j * np.pi * f * t)))
    teoria = np.exp(-((2 * np.pi * sigma * t) ** 2) / 2)
    print(f"t = {t_ms} ms: |M| simulado = {M:.3f} | teoria exp(-(2 pi sigma t)^2/2) = {teoria:.3f}")

# %% Celda 4: el spin echo recupera la senal
# Idea de Hahn (1950): a mitad del tiempo se aplica un pulso pi (giro de 180 grados
# alrededor de x). Eso invierte las fases acumuladas, theta -> -theta, y en la
# segunda mitad cada espin "desacumula" lo que acumulo:
#   fase final = -(w t/2) + w t/2 = 0   para todos los espines.
# Todos vuelven a apuntar igual y aparece el "eco". Pero el eco es menor que la
# senal inicial, porque la relajacion natural (T2) no se corrige.
print("\nt (ms) | sin eco |M| | con eco |M| | e^{-t/T2}")
for t_ms in (2, 4, 8, 16, 32):
    t = t_ms * 1e-3
    sin_eco = abs(np.mean(np.exp(2j * np.pi * f * t)))
    fases = -2 * np.pi * f * (t / 2) + 2 * np.pi * f * (t / 2)      # = 0 para todos
    con_eco = abs(np.mean(np.exp(1j * fases))) * np.exp(-t / T2)    # solo queda T2
    print(f"{t_ms:6d} | {sin_eco:11.3f} | {con_eco:11.3f} | {np.exp(-t / T2):.3f}")
# Con eco, la unica perdida es la relajacion natural: el eco decae con T2, y sin
# eco la senal decae mucho mas rapido (T2*). Por eso T2 >= T2*.

# %% Celda 5: la fase del eco depende del pulso pi
# Con un solo pulso pi, la fase del eco puede no coincidir con la fase inicial: depende
# del eje del pulso pi y de la direccion inicial. Se sigue la fase de UN espin
# con frecuencia w en el plano xy (vector de Bloch con rotaciones 3x3).
def giro_z(angulo):
    c, s = np.cos(angulo), np.sin(angulo)
    return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])


PI_X = np.diag([1.0, -1.0, -1.0])    # giro de 180 grados alrededor de x
PI_Y = np.diag([-1.0, 1.0, -1.0])    # giro de 180 grados alrededor de y


def fase_eco(fase_inicial, pulso, dos_pulsos, w=2 * np.pi * 123.0, t=4e-3):
    r = np.array([np.cos(fase_inicial), np.sin(fase_inicial), 0.0])
    r = giro_z(w * t / 2) @ r       # primera mitad: precesion
    r = pulso @ r                   # pulso pi
    r = giro_z(w * t / 2) @ r       # segunda mitad
    if dos_pulsos:
        r = pulso @ r               # segundo pulso pi para "corregir" la fase
    return round(float(np.degrees(np.arctan2(r[1], r[0])))) % 360
```

```python


print("\nfase inicial | pulso | eco con 1 pulso | eco con 2 pulsos")
for fase0 in (0, 90):
    for nombre, pulso in (("pi_x", PI_X), ("pi_y", PI_Y)):
        print(f"{fase0:10d}° | {nombre} | {fase_eco(np.radians(fase0), pulso, False):13.0f}° | {fase_eco(np.radians(fase0), pulso, True):14.0f}°")
# Con 1 pulso la fase del eco cambia segun el caso; con el segundo pulso del mismo
# eje coincide siempre con la fase inicial: la secuencia "mantiene la fase".

# %% Celda 6: spin echo frente a CPMG
# Si el campo ademas fluctua LENTAMENTE en el tiempo (aqui con un tiempo de
# correlacion de 10 ms), un solo pulso pi en un intervalo largo ya no refocaliza todo:
# el ruido de la primera mitad ya es distinto al de la segunda. La secuencia
# CPMG-n (Carr, Purcell, Meiboom, Gill) pone n pulsos pi repartidos en el tiempo
# t, con la separacion t/n y el primero y el ultimo a t/(2n) de los extremos.
# Con pulsos mas frecuentes que los cambios del ruido, cada fluctuacion se cancela.
paso = 0.5e-3                  # paso de tiempo
t_total = 40e-3
pasos = int(round(t_total / paso))
tau_c = 10e-3                                  # tiempo de correlacion del ruido
sigma_ruido = 30.0                             # fluctuacion de 30 Hz
estatico = rng.normal(0.0, sigma, N)
# Ruido que cambia suavemente: cada paso se parece al anterior (proceso AR(1)).
rho = np.exp(-paso / tau_c)
ruido = np.zeros((N, pasos))
ruido[:, 0] = rng.normal(0.0, sigma_ruido, N)
for i in range(1, pasos):
    ruido[:, i] = rho * ruido[:, i - 1] + np.sqrt(1 - rho ** 2) * rng.normal(0.0, sigma_ruido, N)


def magnetizacion(n_pulsos):
    """Magnitud de la magnetizacion neta al final, con n_pulsos pulsos pi repartidos."""
    posiciones = {int(round(pasos * (2 * k - 1) / (2 * n_pulsos))) for k in range(1, n_pulsos + 1)} if n_pulsos else set()
    signo, fase = 1.0, np.zeros(N)
    for i in range(pasos):
        if i in posiciones:
            signo = -signo             # cada pulso pi invierte la fase acumulada
        fase += signo * 2 * np.pi * (estatico + ruido[:, i]) * paso
    return abs(np.mean(np.exp(1j * fase)))


print("\nSecuencia                | |M| al final")
print(f"sin pulsos               | {magnetizacion(0):.3f}")
print(f"spin echo (1 pulso)      | {magnetizacion(1):.3f}")
print(f"CPMG-8 (8 pulsos)        | {magnetizacion(8):.3f}")
```

23.5. Explicación celda por celda

Celda 2 Evoluciona el vector de Bloch con T_1 y T_2 y calcula su longitud y la pureza $(1+|\mathbf{r}|^2)/2$.

Celda 3 Simula 5000 espines con frecuencias distintas y compara la magnetización con la fórmula gaussiana $e^{-(2\pi\sigma t)^2/2}$.

Celda 4 Compara la señal sin eco y con eco para varios tiempos.

Celda 5 Sigue la fase de un espín con rotaciones 3×3 para cuatro combinaciones de fase inicial y eje del pulso π, con uno y con dos pulsos.

Celda 6 Compara sin pulsos, spin echo y CPMG-8 con un campo que fluctúa lentamente.

23.6. Salida obtenida

```
t (ms) |  x      y      z   | |r|   | pureza = (1 + |r|^2)/2
     0 | +1.000 +0.000 +0.000 | 1.000 | 1.000
    10 | +0.779 +0.000 +0.095 | 0.785 | 0.808
    40 | +0.368 +0.000 +0.330 | 0.494 | 0.622
   100 | +0.082 +0.000 +0.632 | 0.637 | 0.703
   300 | +0.001 +0.000 +0.950 | 0.950 | 0.951
T2* (escala del desfase) ~ 1.59 ms
t = 0 ms: |M| simulado = 1.000 | teoria exp(-(2 pi sigma t)^2/2) = 1.000
t = 1 ms: |M| simulado = 0.821 | teoria exp(-(2 pi sigma t)^2/2) = 0.821
t = 2 ms: |M| simulado = 0.459 | teoria exp(-(2 pi sigma t)^2/2) = 0.454
t = 4 ms: |M| simulado = 0.054 | teoria exp(-(2 pi sigma t)^2/2) = 0.042
t = 8 ms: |M| simulado = 0.017 | teoria exp(-(2 pi sigma t)^2/2) = 0.000

t (ms) | sin eco |M| | con eco |M| | e^{-t/T2}
     2 |       0.459 |       0.951 | 0.951
     4 |       0.054 |       0.905 | 0.905
     8 |       0.017 |       0.819 | 0.819
    16 |       0.011 |       0.670 | 0.670
    32 |       0.009 |       0.449 | 0.449

fase inicial | pulso | eco con 1 pulso | eco con 2 pulsos
         0° | pi_x |             0° |              0°
         0° | pi_y |           180° |              0°
        90° | pi_x |           270° |             90°
        90° | pi_y |            90° |             90°

Secuencia               | |M| al final
sin pulsos              | 0.005
spin echo (1 pulso)     | 0.009
CPMG-8 (8 pulsos)       | 0.749
```

Usa semilla fija. Con otra semilla cambian ligeramente los números del conjunto de espines.

23.7. Interpretación

La componente transversal baja a 0.368 en $t = T_2 = 40$ ms, mientras que la poblacional (z) se recupera más despacio. El desfase de un conjunto coincide con la fórmula (por ejemplo 0.459 frente a 0.454 a los 2 ms). Con eco, la señal a 32 ms es 0.449 (solo la relajación natural e^{-t/T_2}), mientras que sin eco es 0.009. Con un solo pulso, la fase del eco coincide con la inicial solo en 2 de los 4 casos (0°, 180°, 270° y 90° para fases iniciales 0° y 90° con π_x y π_y); con el segundo pulso del mismo eje es siempre la inicial. Con un campo que fluctúa lentamente, un solo spin echo no basta (0.009), pero CPMG-8 conserva 0.749 de la señal.

23.8. Ejercicios

1. Cambia T_2 a 20 ms y observa cómo cambia la columna “con eco”.
2. Prueba CPMG con $n = 2, 4, 16$ y compara la señal final.
3. ¿Qué pasa con CPMG si el ruido fluctúa muy rápido (tiempo de correlación de 1 ms)?

23.9. Errores comunes

- Creer que el eco recupera *toda* la señal: solo corrige el desfase por campos estáticos (o lentos); la relajación natural T_2 sigue.
- Confundir T_2 con T_2^*: T_2^* incluye la inhomogeneidad y es más corto.

24. Desacople dinámico: apagar el acoplamiento con pulsos π

24.1. Objetivo

Ver cómo el acoplamiento entre dos espines hace que la fase de uno dependa del estado del otro, y cómo un pulso π a mitad de la evolución lo cancela (*desacople*).

24.2. Conceptos previos

Concepto previo: acoplamiento $\sigma_z \otimes \sigma_z$ y spin echo

Ver capítulo 12 (el acoplamiento $H = 2\pi J\, \sigma_z \otimes \sigma_z/4$ y cómo el eje de giro de un qubit depende del otro) y capítulo 21 (un pulso π revierte las fases acumuladas).

24.3. Fundamento teórico

El desacople dinámico es una técnica de control que usa pulsos π para suprimir interacciones no deseadas, y la idea es la misma del spin echo.

Modelo. Se usan dos espines: H (un protón) y P (un núcleo de fósforo), con acoplamiento $J = 696$ Hz (el valor del capítulo 12). Durante un tiempo libre t,

$$U(t) = \exp\big(-i\, 2\pi J\, \sigma_z \otimes \sigma_z\, t/4\big). \tag{40}$$

Si H está en $|0\rangle$ (arriba), P gira alrededor de z con una frecuencia, y si está en $|1\rangle$ (abajo) con la contraria. La fase que acumula P es $\pm\pi J t$.

Superposición. Si H está en una superposición, P gira en los dos sentidos a la vez, y los dos espines se entrelazan: el vector de Bloch de P se acorta ($|\cos \pi J t|$ para la superposición 50/50) y su fase se pierde.

Desacople con un pulso π. Se deja evolucionar $t/2$, se aplica un pulso π y se deja evolucionar otros $t/2$:

- π **sobre H:** invierte el estado de H y, con él, el signo del giro de P en la segunda mitad; las fases se cancelan.
- π **sobre P:** invierte la fase de P en la mitad; lo que acumula después deshace lo anterior (como en el spin echo).
- π **sobre ambos:** ambos efectos se suman en vez de cancelarse. El producto $\sigma_z \otimes \sigma_z$ no cambia al invertir los dos espines, así que el acoplamiento sigue activo.

Se considera que hubo desacople si P termina igual que lo haría sin acoplamiento ($J = 0$) con la misma secuencia de pulsos.

Fuentes y alcance

Este capítulo se basa en la guía de la práctica de desacople dinámico y en el modelo de acoplamiento del capítulo 12. El archivo de teoría de esta práctica es un escaneo sin texto legible, así que las conclusiones salen **de calcular** el modelo, no de citar el documento. No se cubre el último ejercicio de la guía (diseñar un conjunto de pulsos para 5 espines con una matriz de desacople).

24.4. Código (completamente comentado)

```python
# %% [markdown]
# # 22 - Desacople dinamico: apagar el acoplamiento entre dos espines con pulsos pi

# %% Celda 1: importaciones
import numpy as np
from scipy.linalg import expm

# Matrices de Pauli e identidad. Trabajamos con hbar = 1 y frecuencias en Hz.
I2 = np.eye(2, dtype=complex)
sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)

# Dos espines: H (primer factor del producto) y P (segundo factor).
# Acoplamiento entre ellos (el mismo valor del capitulo 12).
J = 696.0
H_acople = 2 * np.pi * J * np.kron(sz, sz) / 4

# %% Celda 2: herramientas
def evolucion(t, J_activo=True):
    """Evolucion libre durante un tiempo t bajo el acoplamiento."""
    return expm(-1j * (H_acople if J_activo else 0 * H_acople) * t)


def pi_en(espin):
    """Pulso pi (giro de 180 grados alrededor de x) sobre el espin 'H' o 'P' (o 'HP' = ambos)."""
    xH = np.kron(sx, I2) if "H" in espin else np.eye(4)   # 'H' aparece en 'H' y en 'HP'
    xP = np.kron(I2, sx) if "P" in espin else np.eye(4)
    return -1j * (xH @ xP)        # el factor -i es la fase global de un giro de 180 grados


def bloch_de_P(estado):
    """Vector de Bloch (x, y, z) del espin P (se descarta el espin H: traza parcial)."""
    rho = np.outer(estado, estado.conj()).reshape(2, 2, 2, 2)
    rho_P = np.einsum("ijik->jk", rho)
    return np.array([np.real(np.trace(rho_P @ s)) for s in (sx, sy, sz)])


def fase_P(estado):
    x, y, _ = bloch_de_P(estado)
    return float(np.degrees(np.arctan2(y, x)))


mas = np.array([1, 1], dtype=complex) / np.sqrt(2)        # |+>: espin apuntando a +x
arriba, abajo = np.array([1, 0], dtype=complex), np.array([0, 1], dtype=complex)

# %% Celda 3: el efecto del acoplamiento (el espin H en |0> o en |1>)
# P empieza apuntando a +x. Segun H este en |0> o |1>, P gira en sentido contrario.
# La fase acumulada vale  +-pi*J*t  (con signo segun el estado de H).
print("t (ms) | H en |0> | H en |1> | sin acoplamiento")
for t_ms in (0.5, 1.0, 2.0):
    t = t_ms * 1e-3
    fases = []
    for h in (arriba, abajo):
        fases.append(fase_P(evolucion(t) @ np.kron(h, mas)))
    sin = fase_P(evolucion(t, J_activo=False) @ np.kron(arriba, mas))
    print(f"{t_ms:6.1f} | {fases[0]:+8.1f}° | {fases[1]:+8.1f}° | {sin:+.1f}°")

# %% Celda 4: H en superposicion: P pierde su fase (queda entrelazado)
# Con H en cos(a)|0> + sin(a)|1>, P gira en los dos sentidos "a la vez". Su vector
```

```python
# de Bloch se acorta (queda mezclado): la fase de P se pierde.
print("\nProporcion |0> de H | t (ms) | longitud transversal de P")
for a in (0.0, np.pi / 8, np.pi / 4):          # a = pi/4 es la superposicion 50/50
    h = np.array([np.cos(a), np.sin(a)], dtype=complex)
    for t_ms in (0.5, 1.0):
        x, y, _ = bloch_de_P(evolucion(t_ms * 1e-3) @ np.kron(h, mas))
        print(f"{np.cos(a) ** 2:20.2f} | {t_ms:6.1f} | {np.hypot(x, y):.3f}")

# %% Celda 5: pulso pi en la mitad del tiempo (desacople)
# Un pulso pi sobre uno de los espines a mitad de la evolucion invierte el signo del
# acoplamiento en la segunda mitad, y la fase acumulada en la primera se cancela.
# Se compara con la misma secuencia SIN acoplamiento: si P termina igual, hubo desacople.
def secuencia(t, pulsos, J_activo=True):
    """Evolucion t/2, pulso(s) pi, evolucion t/2."""
    U = evolucion(t / 2, J_activo)
    for esp in pulsos:
        U = pi_en(esp) @ U
    return evolucion(t / 2, J_activo) @ U


print("\nt (ms) | pulso pi sobre | fase de P (con J) | fase de P (J = 0) | desacople")
for t_ms in (1.0, 2.0):
    t = t_ms * 1e-3
    estado0 = np.kron(arriba, mas)                  # H en |0>, P en |+>
    for nombre, pulsos in (("ninguno", []), ("H", ["H"]), ("P", ["P"]), ("H y P", ["HP"])):
        con = fase_P(secuencia(t, pulsos) @ estado0)
        sin = fase_P(secuencia(t, pulsos, J_activo=False) @ estado0)
        desacoplado = np.isclose((con - sin + 180) % 360 - 180, 0, atol=1e-6)
        print(f"{t_ms:6.1f} | {nombre:14s} | {con:+16.1f}° | {sin:+16.1f}° | {'si' if
    ↪ desacoplado else 'no'}")
```

24.5. Explicación celda por celda

Celda 2 Herramientas: evolución libre, pulso π sobre H, P o ambos, vector de Bloch de P (traza parcial) y su fase.

Celda 3 Fase de P con H en $|0\rangle$ y en $|1\rangle$ para $t = 0.5$, 1 y 2 ms.

Celda 4 Longitud transversal de P con H en superposición.

Celda 5 Aplica un pulso π a mitad de la evolución y compara la fase de P con y sin acoplamiento.

24.6. Salida obtenida

```
t (ms) | H en |0> | H en |1> | sin acoplamiento
   0.5 |    +62.6° |    -62.6° | +0.0°
   1.0 |   +125.3° |   -125.3° | +0.0°
   2.0 |   -109.4° |   +109.4° | +0.0°

Proporcion |0> de H | t (ms) | longitud transversal de P
                1.00 |    0.5 | 1.000
                1.00 |    1.0 | 1.000
                0.85 |    0.5 | 0.778
                0.85 |    1.0 | 0.817
                0.50 |    0.5 | 0.460
                0.50 |    1.0 | 0.578

t (ms) | pulso pi sobre | fase de P (con J) | fase de P (J = 0) | desacople
   1.0 | ninguno        |           +125.3° |             +0.0° | no
   1.0 | H              |             +0.0° |             +0.0° | si
```

```
  1.0 | P             |            +0.0° |            +0.0° | si
  1.0 | H y P         |          -125.3° |            +0.0° | no
  2.0 | ninguno       |          -109.4° |            +0.0° | no
  2.0 | H             |            +0.0° |            +0.0° | si
  2.0 | P             |            +0.0° |            +0.0° | si
  2.0 | H y P         |          +109.4° |            +0.0° | no
```

Es determinista.

24.7. Interpretación

La fase de P vale $\pm\pi Jt$: 62.6° a 0.5 ms y 125.3° a 1 ms, con signo opuesto según el estado de H. Con H en superposición 50/50, la longitud transversal de P baja a 0.460 (0.5 ms) y a 0.578 (1 ms), que coincide con $|\cos \pi Jt|$. Con un pulso π sobre H o sobre P a mitad de la evolución, la fase de P es 0°, igual que sin acoplamiento: **hay desacople**. Con π sobre ambos, la fase queda en $\mp 125.3°$ (a 1 ms): **no hay desacople**.

24.8. Ejercicios

1. Cambia la posición del pulso π (por ejemplo, a $t/4$) y observa si hay desacople.
2. Con H en superposición 50/50, aplica un pulso π sobre H a mitad y mide la longitud transversal de P.
3. Prueba con $J = 300$ Hz y con $t = 2$ ms: ¿cambia la conclusión?

24.9. Errores comunes

- Creer que invertir los dos espines ayuda: como $\sigma_z \otimes \sigma_z$ no cambia, el acoplamiento sigue.
- Olvidar comparar con el caso $J = 0$: el pulso π también cambia la fase de P por sí mismo.

25. Pulsos optimizados con GRAPE: control como problema de optimización

25.1. Objetivo

Diseñar con un algoritmo numérico (GRAPE) la forma de un pulso de radiofrecuencia que prepare un estado de Bell en dos espines acoplados, y comprobar cómo se comporta frente a errores de frecuencia y de amplitud.

25.2. Conceptos previos

Concepto previo: control con Hamiltoniano, fidelidad y gradiente

Ver capítulo 12 (control con pulsos y acoplamiento), capítulo 13 (fidelidad) y capítulo 19 (optimización con gradiente). La *fidelidad* con un estado objetivo puro $|\phi\rangle$ es $F = |\langle\phi|\psi\rangle|^2$ (vale 1 si son iguales).

25.3. Fundamento teórico

Hasta ahora los pulsos tenían forma rectangular (*pulsos duros*: amplitud constante durante cierto tiempo). Pero la forma de un pulso se puede elegir. Según las notas del curso, los *pulsos con forma* (gaussianos, de Hermite) reducen los errores de excitación fuera de banda, y los algoritmos de optimización numérica como **GRAPE** y **SMP** ajustan la forma de los pulsos de control para maximizar la fidelidad de una operación dada: tratan el control como un problema de optimización, no de diseño manual. La guía de la práctica pide preparar un estado de Bell con GRAPE y hacer el pulso resistente a errores de ±10 Hz en la frecuencia y ±5 % en la amplitud.

El problema. Dos espines con acoplamiento $J = 696$ Hz y control de radiofrecuencia en x e y sobre cada uno:

$$H(t) = \frac{2\pi J}{4}\sigma_z \otimes \sigma_z + \frac{2\pi}{2}\sum_{j=1}^{4} u_j(t)\,\sigma_j, \tag{41}$$

con cuatro controles $u_j(t)$ (en Hz): σ_x y σ_y de cada espín. Se busca la forma de $u_j(t)$ que lleve $|00\rangle$ a $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$.

GRAPE. (*Gradient ascent pulse engineering.*) La duración total T se divide en N pasos de amplitud constante, y la amplitud de cada control en cada paso es un parámetro. Se maximiza

$$F = \left|\langle\Phi^+|\,U_N \cdots U_2 U_1\,|00\rangle\,\right|^2, \qquad U_k = e^{-iH_k\,\Delta t}. \tag{42}$$

El gradiente respecto de cada amplitud se obtiene propagando el estado hacia adelante y el objetivo hacia atrás (en primera aproximación en Δt), y un optimizador (aquí L-BFGS-B) lo usa para mejorar los parámetros.

Robustez. Para que el pulso resista errores se optimiza el *promedio* de la fidelidad sobre varias condiciones de error a la vez: desintonías $\pm\Delta$ en ambos espines y factores de escala de amplitud.

Fuentes y alcance

Este capítulo se basa en el resumen de las notas (qué son GRAPE y SMP) y en la guía de la práctica. Los archivos de teoría de pulsos con forma y de optimización numérica son escaneos sin texto legible. Por eso aquí **solo se implementa GRAPE** con el estado de Bell; no se implementan SMP, los pulsos duros de comparación ni los pulsos gaussianos o

de Hermite.

25.4. Código (completamente comentado)

```python
# %% [markdown]
# # 23 - Pulsos optimizados con GRAPE: preparar un estado de Bell con pulsos de forma libre

# %% Celda 1: importaciones
import numpy as np
from scipy.linalg import expm
from scipy.optimize import minimize

I2 = np.eye(2, dtype=complex)
sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)

# %% Celda 2: el problema de control
# Dos espines acoplados (J = 696 Hz, como en el capitulo 12). Se controlan con
# campos de radiofrecuencia en x e y sobre cada espin; la amplitud de cada campo
# (en Hz) puede cambiar en el tiempo. El Hamiltoniano es
#     H(t) = 2*pi*J*(sz (x) sz)/4  +  2*pi * suma_j  u_j(t) * sigma_j / 2,
# con cuatro controles: sx y sy en cada espin. Se busca la forma de u_j(t) que lleva
# |00> al estado de Bell (|00> + |11>)/sqrt2.
J = 696.0
H_deriva = 2 * np.pi * J * np.kron(sz, sz) / 4
OPERADORES = [np.kron(sx, I2), np.kron(sy, I2), np.kron(I2, sx), np.kron(I2, sy)]
H_controles = [2 * np.pi * op / 2 for op in OPERADORES]

T = 1.0e-3                        # duracion total del pulso: 1 ms
N = 20                            # numero de pasos de amplitud constante
dt = T / N
U_MAX = 5000.0                    # amplitud maxima de cada control: 5 kHz

inicial = np.array([1, 0, 0, 0], dtype=complex)                    # |00>
objetivo = np.array([1, 0, 0, 1], dtype=complex) / np.sqrt(2)      # Bell


def H_con_errores(desint=0.0, escala=1.0):
    """Deriva con un error de frecuencia (desint, en Hz) en ambos espines; los controles se escalan por 'escala'."""
    error = 2 * np.pi * desint * (np.kron(sz, I2) + np.kron(I2, sz)) / 2
    return H_deriva + error, escala


# %% Celda 3: GRAPE
# GRAPE (gradient ascent pulse engineering) trata el pulso como un problema de
# optimizacion: la forma del pulso se divide en N pasos y la amplitud de cada
# control en cada paso es un parametro. Se maximiza la fidelidad
#     F = |<objetivo| U(T) |inicial>|^2.
# El gradiente respecto de cada amplitud se obtiene propagando el estado hacia
# adelante y el objetivo hacia atras (aproximacion de primer orden en dt).
def propagadores(u, H0, escala):
    return [expm(-1j * (H0 + escala * sum(u[k, j] * H_controles[j] for j in range(4))) * dt) for k in range(N)]


def fidelidad_y_gradiente(u, H0, escala):
    U = propagadores(u, H0, escala)
    adelante = [inicial]
    for Uk in U:
        adelante.append(Uk @ adelante[-1])
```

```
    atras = [None] * (N + 1)
    atras[N] = objetivo
    for k in range(N - 1, -1, -1):
        atras[k] = U[k].conj().T @ atras[k + 1]
    c = np.vdot(objetivo, adelante[N])
    grad = np.zeros_like(u)
    for k in range(N):
        for j in range(4):
            dc = np.vdot(atras[k + 1], (-1j * dt * escala * H_controles[j]) @ adelante[k + 1])
            grad[k, j] = 2 * np.real(np.conj(c) * dc)
    return abs(c) ** 2, grad


def optimizar(errores, semilla=0, intentos=4):
    """Maximiza la fidelidad PROMEDIO sobre una lista de errores (desintonia, escala)."""
    mejor = None
    for s in range(intentos):
        rng = np.random.default_rng(semilla + s)
        u0 = rng.uniform(-1000, 1000, (N, 4))

        def costo(x):
            u = x.reshape(N, 4)
            F, G = 0.0, np.zeros_like(u)
            for desint, escala in errores:
                H0, esc = H_con_errores(desint, escala)
                f, g = fidelidad_y_gradiente(u, H0, esc)
                F += f / len(errores)
                G += g / len(errores)
            return -F, -G.ravel()

        res = minimize(costo, u0.ravel(), jac=True, method="L-BFGS-B",
                       bounds=[(-U_MAX, U_MAX)] * (N * 4), options={"maxiter": 400})
        if mejor is None or res.fun < mejor.fun:
            mejor = res
    return mejor.x.reshape(N, 4), -mejor.fun


# %% Celda 4: pulso optimizado para el caso ideal (sin errores)
u_ideal, F_ideal = optimizar([(0.0, 1.0)])
print(f"Fidelidad del pulso GRAPE (caso ideal): {F_ideal:.4f}")
print("Amplitud maxima usada:", round(float(np.abs(u_ideal).max())), "Hz (limite", int(U_MAX), "
    ↪ Hz)")

# %% Celda 5: ese pulso frente a errores
# En un equipo real la frecuencia del pulso puede estar algo desafinada y la
# amplitud puede no ser exacta. Se evalua el pulso con desintonias de +-10 Hz
# y errores de amplitud de +-5 %.
CASOS = [(d, e) for d in (-10.0, 0.0, 10.0) for e in (0.95, 1.0, 1.05)]


def evaluar(u):
    fid = [fidelidad_y_gradiente(u, *H_con_errores(d, e))[0] for d, e in CASOS]
    return np.array(fid)


f1 = evaluar(u_ideal)
print(f"Sin robustez: fidelidad promedio = {f1.mean():.4f}, minima = {f1.min():.4f}")

# %% Celda 6: pulso robusto
# Para hacer el pulso resistente, se optimiza el PROMEDIO de la fidelidad sobre
# todos los errores a la vez (9 combinaciones de desintonia y amplitud).
u_robusto, _ = optimizar(CASOS)
f2 = evaluar(u_robusto)
```

```
print(f"Con robustez: fidelidad promedio = {f2.mean():.4f}, minima = {f2.min():.4f}")
print(f"En el caso ideal (sin errores) el pulso robusto da {evaluar(u_robusto)[CASOS.index((0.0, 1.0))]:.4f}")

# %% Celda 7: errores mayores
# Con los errores del experimento (+-10 Hz, +-5 %) el pulso sin robustez ya se
# comporta bien, porque el pulso dura solo 1 ms. Con errores mas grandes la
# diferencia se nota: desintonias de +-300 Hz y errores de amplitud de +-20 %.
CASOS_GRANDES = [(d, e) for d in (-300.0, 0.0, 300.0) for e in (0.8, 1.0, 1.2)]


def evaluar_grandes(u):
    return np.array([fidelidad_y_gradiente(u, *H_con_errores(d, e))[0] for d, e in CASOS_GRANDES])


u_robusto_grande, _ = optimizar(CASOS_GRANDES)
for nombre, u in (("sin robustez", u_ideal), ("robusto (errores grandes)", u_robusto_grande)):
    f = evaluar_grandes(u)
    print(f"{nombre:26s}: fidelidad promedio = {f.mean():.4f}, minima = {f.min():.4f}")
```

25.5. Explicación celda por celda

Celda 2 Define H con deriva, controles, $T = 1$ ms, $N = 20$ pasos, amplitud máxima de 5 kHz y el estado objetivo.

Celda 3 GRAPE: propagadores, fidelidad, gradiente y la función `optimizar`, con cuatro intentos desde puntos iniciales distintos.

Celda 4 Optimiza el pulso del caso ideal (sin errores).

Celda 5 Evalúa ese pulso con desintonías de ± 10 Hz y errores de amplitud de ± 5 % (nueve combinaciones).

Celda 6 Optimiza el promedio sobre esas nueve combinaciones.

Celda 7 Repite con errores mayores (± 300 Hz y ± 20 %).

25.6. Salida obtenida

```
Fidelidad del pulso GRAPE (caso ideal): 0.9979
Amplitud maxima usada: 1694 Hz (limite 5000 Hz)
Sin robustez: fidelidad promedio = 0.9939, minima = 0.9889
Con robustez: fidelidad promedio = 0.9927, minima = 0.9902
En el caso ideal (sin errores) el pulso robusto da 0.9966
sin robustez              : fidelidad promedio = 0.4130, minima = 0.0809
robusto (errores grandes) : fidelidad promedio = 0.9520, minima = 0.8751
```

Es determinista (semillas fijas).

25.7. Interpretación

El pulso GRAPE prepara el estado de Bell con fidelidad 0.9979, usando como máximo 1694 Hz de amplitud. Con los errores de la guía (± 10 Hz y ± 5 %), el pulso sin robustez ya mantiene una fidelidad promedio de 0.9939 (mínima 0.9889), y el robusto 0.9927 (mínima 0.9902): **casi no hay diferencia**, porque el pulso dura solo 1 ms y esos errores son pequeños. Con errores mayores (± 300 Hz y ± 20 %) la diferencia es grande: la fidelidad promedio del pulso sin robustez cae a 0.4130 (mínima 0.0809), mientras que el pulso optimizado para esos errores mantiene 0.9520 (mínima 0.8751). Es el costo típico: el pulso robusto es un poco menos preciso en el caso ideal (0.9966 frente a 0.9979).

25.8. Ejercicios

1. Aumenta T a 2 ms y mira cómo cambia la fidelidad y la robustez.
2. Reduce la amplitud máxima a 1 kHz: ¿se sigue alcanzando el Bell?
3. Cambia el estado objetivo a $|\Psi^+\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$.

25.9. Errores comunes

- Creer que el optimizador siempre halla el máximo global: GRAPE puede quedarse en un máximo local, por eso se prueban varios puntos iniciales.
- Olvidar que la fidelidad reportada se calcula con el propagador exacto, mientras que el gradiente usa una aproximación de primer orden.

Referencias

- M. A. Nielsen e I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press, 2010. (Referencia general; cap. 1 y 2 para los fundamentos.)
- Documentación de Qiskit y el curso *Basics of quantum information* de IBM Quantum Learning (verificar la versión vigente de la API antes de usarla: cambia entre versiones).
- M. Born, "Zur Quantenmechanik der Stoßvorgänge", *Zeitschrift für Physik* 37, 863–867, 1926. (Regla de Born.)
- J. S. Bell, "On the Einstein Podolsky Rosen paradox", *Physics* 1, 195–200, 1964.
- D. M. Greenberger, M. A. Horne y A. Zeilinger, "Going beyond Bell's theorem", en *Bell's Theorem, Quantum Theory and Conceptions of the Universe*, Kluwer, 1989. (Estados GHZ.)
- L. K. Grover, "A fast quantum mechanical algorithm for database search", *Proc. 28th ACM STOC*, 212–219, 1996.
- P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring", *Proc. 35th IEEE FOCS*, 124–134, 1994.
- J. Preskill, "Quantum Computing in the NISQ era and beyond", *Quantum* 2, 79, 2018.
- Notas del curso *Quantum Computing Experimental Platform* (materiales de la carpeta del Laboratorio de Programación Cuántica UNA), prácticas 1.3 *Rabi Oscillations*, 1.4 *Quantum Bit*, 1.6 *Quantum Control* y 1.9 *Quantum State Tomography*. (Base de los capítulos 10 a 13.)
- Notas del mismo curso, prácticas 2.1 *Introduction to Quantum Algorithm*, 2.2 *Deutsch Algorithm*, 2.3 *Bernstein–Vazirani Algorithm* y 2.5 *Quantum Fourier Transformation and Quantum Phase Estimation*. (Base de los capítulos 14 a 17.)
- Notas del mismo curso, prácticas 2.6 *HHL Algorithm*, 2.7 *Variational Quantum Eigensolver* y 2.8 *Quantum Approximate Optimization Algorithm*. (Base de los capítulos 18 a 20.) Las notas citan a Harrow, Hassidim y Lloyd (2009) para HHL y a Farhi, Goldstone y Gutmann (2014) para QAOA.
- Notas del mismo curso: *Spin Echo* (teoría completa), y las guías de las prácticas *Quantum Decoherence*, *Dynamic Decoupling*, *Shape Pulse* y *Numerical Optimization Pulse*, junto con el resumen de las notas (definiciones de T_1, T_2, GRAPE y SMP). (Base de los capítulos 21 a 23; los archivos de teoría de decoherencia, desacople dinámico y pulsos son escaneos sin texto legible.)
- Variante del algoritmo de Bernstein–Vazirani usada en el capítulo 16: *Phys. Rev. A* 64, 042306 (citada en las notas del curso).